



$$\psi_k = \sum_{\mu} c_{\mu k} \chi_{\mu} \quad (\text{LCAO})$$

$$\hat{F} \varphi_i = \varepsilon_i \varphi_i$$

$$S_{\mu\nu} = \int \chi_{\mu}^*(\mathbf{r}) \chi_{\nu}(\mathbf{r}) d\mathbf{r}$$

С. М. Пономаренко

Квантово-механічні методи обчислення Використання PySCF

$$\left[-\frac{1}{2} \nabla_i^2 + \sum_{j \neq i} \frac{1}{r_{ij}} + V_{\text{nuc}}(i) \right] \varphi_i = \varepsilon_i \varphi_i$$
$$\Psi = \det \begin{pmatrix} \varphi_1(1) & \varphi_2(1) & \cdots & \varphi_N(1) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi_1(N) & \varphi_2(N) & \cdots & \varphi_N(N) \end{pmatrix}$$

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

С. М. Пономаренко

Квантово-механічні методи обчислення Використання PySCF

*Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського як
навчальний посібник для здобувачів ступеня магістра за спеціальностями
Е6 «Прикладна фізика та наноматеріали»*

КИЇВ
КПІ ім. Ігоря Сікорського
2025

Зміст

1	Знайомство з PySCF	8
1.1	Що таке PySCF: можливості та переваги	8
1.1.1	Основні можливості PySCF	8
1.1.2	Переваги використання PySCF	8
1.1.3	Порівняння з іншими програмами	9
1.2	Встановлення PySCF	9
1.2.1	Системні вимоги	9
1.2.2	Встановлення в Linux	10
1.2.3	Встановлення в Windows	10
1.2.4	Встановлення в macOS	10
1.2.5	Перевірка встановлення	11
1.3	Базова структура програми на PySCF	11
1.3.1	Загальна схема розрахунку	11
1.3.2	Мінімальний приклад	11
1.3.3	Структура для атома Гелію	12
1.3.4	Розрахунок з аналізом результатів	12
1.4	Система одиниць та конвенції	12
1.4.1	Атомні одиниці	12
1.4.2	Конвертація одиниць	13
1.4.3	Конвенції для атомів	13
1.5	Документація та ресурси	14
1.5.1	Офіційна документація	14
1.5.2	Основні публікації	14
1.6	Резюме	14
2	Базові об'єкти PySCF	15
2.1	Молекулярний об'єкт (Mole)	15
2.1.1	Клас Mole — серце PySCF	15
2.1.2	Створення об'єкта Mole	15
2.1.3	Основні атрибути об'єкта Mole	16
2.1.4	Визначення атома: різні способи	17
2.1.5	Робота з іонами	19
2.1.6	Налаштування вербальності виводу	19
2.1.7	Ланцюжок викликів методів (Method Chaining)	20

2.2	Базисні набори в PySCF	21
2.2.1	Доступні бібліотечні базиси	21
2.2.2	Задання власного базису	21
2.2.3	Рекомендації	22
2.2.4	Власні базисні набори	22
2.2.5	Комбінування базисів	22
2.2.6	Інформація про базисний набір	23
2.3	Симетрія в атомних розрахунках	24
2.3.1	Точкові групи симетрії атомів	24
2.3.2	Використання симетрії	24
2.3.3	Симетрія орбіталей	25
2.3.4	Коли вимикати симетрію	26
2.4	Спін та мультиплетність	26
2.4.1	Визначення спінового стану	26
2.4.2	Розподіл альфа- та бета-електронів	27
2.4.3	Вибір правильного спіну	28
2.4.4	Енергетичне підтвердження спінових станів	28
2.4.5	Практичні зауваження	29
2.4.6	Резюме	30
2.4.7	Контрольні запитання	30
2.4.8	Завдання для самостійної роботи	30
3	Метод Хартрі-Фока	31
3.1	Теоретичні основи методу Хартрі-Фока	31
3.1.1	Рівняння Хартрі-Фока	31
3.1.2	Енергія Хартрі-Фока	32
3.1.3	Рівняння Хартрі-Фока в базисному представленні	32
3.1.4	SFC-процедура	37
3.1.5	Початкове наближення для матриці густини $\mathbf{D}^{(0)}$	39
3.1.6	Які бувають базиси	41
3.1.7	Варіанти методу Хартрі-Фока	42
3.2	Метод UHF та спінове забруднення	45
3.2.1	Спінове забруднення	45
3.3	Прості атомні системи	48
3.3.1	Атом Гідрогену	48
3.3.2	Атом Гелію	51
3.3.3	Збуджені стани в межах методу Хартрі-Фока	53
3.3.4	Висновки	54
3.4	Атоми другого періоду (Li–Ne)	54
3.4.1	Літій (Li, $Z=3$)	54
3.4.2	Берилій (Be, $Z=4$)	56
3.4.3	Бор–Неон: систематичне дослідження	57
3.4.4	Заселеності орбіталей і принцип заповнення	58
3.4.5	Енергії іонізації	61

3.4.6	Електронна густина	62
3.4.7	Порівняння електронних густин різних атомів	64
3.4.8	Зв'язок із електронними оболонками	64
3.4.9	Практичні завдання	65
3.4.10	Методичні рекомендації	66
3.5	Складні випадки та збіжність	66
3.5.1	Причини поганої збіжності	66
3.5.2	Методи стабілізації збіжності	66
3.5.3	Вироджені орбіталі та дробові заповнення	68
3.5.4	Перехідні метали — приклад складної системи	68
3.5.5	Загальна стратегія досягнення збіжності	69
4	Метод Хартрі-Фока для простих молекул	72
4.1	Орбіталі і електронні стани молекули	72
4.1.1	Класифікація молекулярних орбіталей	72
4.1.2	Класифікація електронних станів молекули	73
4.1.3	Від атомів до молекул: базисні функції та ЛКАО	75
4.2	Початкові модельні молекули: H_2^+ та H_2	76
4.3	Іон молекули водню H_2^+	77
4.3.1	Теоретичні основи	77
4.3.2	Визначення молекули в PySCF	78
4.3.3	Інтерпретація результатів	79
4.3.4	Крива потенційної енергії (PES)	80
4.3.5	Оптимізація геометрії	83
4.3.6	Молекулярні орбіталі та принцип ЛКАО	85
4.3.7	Вплив базисного набору	85
4.4	Молекула водню H_2	88
4.4.1	Двоелектронна система	88
4.4.2	RHF розрахунок	89
4.5	Проблема дисоціації в методі Хартрі-Фока	91
4.6	Дипольний момент молекули	93
4.7	Резюме	97
4.7.1	Основні висновки	97
4.7.2	Що далі?	98
5	Пост-Хартрі-Фоківські методи	99
5.1	Вступ до електронної кореляції	99
5.1.1	Що таке електронна кореляція?	99
5.1.2	Типи електронної кореляції	99
5.1.3	Кореляційна енергія атомів	100
5.1.4	Коли потрібні post-HF методи?	100
5.1.5	Концепція одно- та багатоконфігураційних методів	101
5.2	Теорія збурень Møller-Plesset (MP2)	103
5.2.1	Теоретичні основи MP2	103

5.2.2	Формула енергії MP2	103
5.2.3	Фізичний зміст MP2-корекції	104
5.2.4	Практичне застосування	104
5.2.5	Практичний гайд: розрахунок MP2 у PySCF	104
5.2.6	Практичні аспекти та обчислювальна складність MP2	105
5.2.7	Приклади MP2-розрахунків у PySCF	108
5.2.8	Переваги та недоліки MP2	108
5.2.9	Рекомендації по використанню MP2	108
5.3	Configuration Interaction (CI) та Full CI	109
5.3.1	Основи Configuration Interaction	109
5.3.2	Full CI — точний розв’язок	110
5.3.3	CISD розрахунки	111
5.4	Coupled Cluster методи (CCSD, CCSD(T))	114
5.4.1	Теоретичні основи Coupled Cluster	114
5.4.2	CCSD розрахунок атома Гелію	116
5.4.3	CCSD для відкритих оболонок (UCCSD)	116
5.4.4	Перевірка size-extensivity: дисоціація LiH	116
5.4.5	Порівняння ієрархії методів	117
5.4.6	T ₁ -діагностика та багатоконфігураційний характер	117
5.5	CASSCF — багатоконфігураційний метод	117
5.5.1	Ідея Complete Active Space Self-Consistent Field	117
5.5.2	Позначення CASSCF(n,m)	118
5.5.3	CASSCF розрахунок атома Берилію	118
5.5.4	CASSCF для перехідних металів	118
5.5.5	Вибір активного простору	118
5.5.6	State-averaged CASSCF	119
5.5.7	CASPT2 — динамічна кореляція поверх CASSCF	119
5.5.8	Переваги та недоліки CASSCF	119
5.6	Практичні завдання	119
5.7	Резюме	122
5.7.1	Основні методи та їх характеристики	122
5.7.2	Ключові висновки	122
5.7.3	Рекомендації по використанню	123
5.7.4	Типові помилки	124
5.7.5	Практичні поради	124
5.7.6	Корисні посилання	124

6 Теорія функціоналу густини (DFT) 125

6.1	Основи теорії функціоналу густини	126
6.1.1	Теореми Хоенберга–Кона	126
6.1.2	Рівняння Кона–Шема	126
6.1.3	Обмінно-кореляційна енергія	127
6.1.4	Порівняння HF та DFT	127
6.2	Функціонали обміну-кореляції	127

6.2.1	Класифікація функціоналів: драбина Якова	127
6.2.2	LDA та LSDA функціонали	127
6.2.3	GGA функціонали	128
6.2.4	Meta-GGA функціонали	129
6.2.5	Гібридні функціонали	129
6.2.6	Порівняння різних рівнів теорії	130
6.2.7	Вибір функціоналу: рекомендації	130
6.2.8	Практичний приклад: вплив функціоналу на властивості	131
6.3	DFT розрахунки атомів	131
6.3.1	Базова структура DFT розрахунку	131
6.3.2	Систематичний розрахунок атомів другого періоду . .	131
6.3.3	Розрахунок атомів перехідних металів	131
6.3.4	Порівняння спінових станів	131
6.3.5	Аналіз d-орбіталей перехідних металів	131
6.3.6	Розрахунок важких атомів	131
6.3.7	Числові сітки в DFT	131
6.3.8	Паралелізація DFT розрахунків	131
6.4	Порівняння HF та DFT результатів	132
6.4.1	Систематичне порівняння енергій	132
6.4.2	Порівняння енергій іонізації	132
6.4.3	Електронна спорідненість	132
6.4.4	Порівняння орбітальних енергій	132
6.4.5	Забруднення спіном: HF vs DFT	132
6.4.6	Час обчислень: HF vs DFT	132
6.4.7	Загальні висновки HF vs DFT	132
6.5	Вибір функціоналу для атомних систем	133
6.5.1	Тестовий набір даних	133
6.5.2	Рекомендації для різних елементів	133
6.6	Практичні завдання	133
6.6.1	Завдання 1: Систематичне дослідження	133
6.6.2	Завдання 2: Функціональна залежність	133
6.6.3	Завдання 3: Конвергенція до базисної межі	134
6.6.4	Завдання 4: Перехідні метали	134
6.7	Резюме	134
6.7.1	Ключові висновки	134
6.7.2	Типові помилки	134
6.7.3	Корисні посилання	135
6.8	DFT для діатомних молекул	135
6.8.1	Базові DFT розрахунки молекул	135
6.8.2	Оптимізація геометрії	135
6.8.3	Криві потенційної енергії	135
6.8.4	Порівняння функціоналів для зв'язків	135
6.8.5	Спектроскопічні константи	135

6.8.6	Гетероядерні молекули	136
6.8.7	Молекули з кратними зв'язками	136
6.8.8	Порівняння HF та DFT для молекул	136
6.8.9	Молекули перехідних металів	136
6.8.10	Відкритошкаралупні молекули	136
6.8.11	Дисперсійні корекції для слабких зв'язків	136
6.8.12	Аналіз хвильових функцій	136
6.8.13	Вплив базисного набору	136
6.8.14	Дисоціація зв'язків	136
6.8.15	Збуджені стани: TD-DFT	137
6.8.16	Порівняльна таблиця результатів	137
6.8.17	Практичні рекомендації	137
6.8.18	Типові помилки при розрахунках молекул	138
6.8.19	Задачі для самостійної роботи	138
6.8.20	Типові помилки при розрахунках молекул	138
6.8.21	Систематичне дослідження галогенів	139
6.8.22	Benchmark: час обчислень	139
6.8.23	Тестовий набір G2	139
6.8.24	Використання симетрії	140
6.8.25	Резюме секції	140
7	Властивості молекул	141
7.1	Електронні переходи та УФ-видимі спектри	147
7.1.1	Теорія збуджених станів	147
7.1.2	TDDFT розрахунок для формальдегіду	149
7.1.3	Вибір функціоналу для TDDFT	149
7.1.4	Візуалізація спектру	150
7.1.5	Аналіз характеру переходів	150
	Література	151

1

Знайомство з PySCF

1.1. Що таке PySCF: можливості та переваги

PySCF (Python-based Simulations of Chemistry Framework) — це сучасний програмний пакет з відкритим вихідним кодом для квантово-хімічних розрахунків, написаний переважно на мові Python з критичними для продуктивності частинами на C. Розроблений групою під керівництвом професора Garnet Chan, PySCF надає потужний та гнучкий інструментарій для проведення *ab initio* розрахунків молекулярних та атомних систем.

1.1.1. Основні можливості PySCF

PySCF підтримує широкий спектр квантово-хімічних методів:

- *Метод Хартрі-Фока (HF)*: restricted (RHF), unrestricted (UHF), та restricted open-shell (ROHF) варіанти
- *Теорія функціоналу густини (DFT)*: з великою бібліотекою функціоналів обміну-кореляції через інтеграцію з `libxc`
- *Post-Hartree-Fock методи*: MP2, CCSD, CCSD(T), CASSCF, CASPT2, NEVPT2
- *Configuration Interaction (CI)*: CISD, full CI для малих систем
- *Явна кореляція*: F12 методи
- *Багаторівневі методи*: ONIOM, embedding
- *Періодичні системи*: через модуль PBC (Periodic Boundary Conditions)

1.1.2. Переваги використання PySCF

Відкритий код та безкоштовність. PySCF розповсюджується під ліцензією Apache 2.0, що робить його повністю безкоштовним для академічного та комерційного використання. Доступ до вихідного коду дозволяє глибоко зрозуміти реалізацію методів та за потреби модифікувати програму.

Гнучкість та розширюваність. Завдяки використанню Python як основної мови, PySCF надзвичайно гнучкий. Користувачі можуть легко:

- Комбінувати різні методи в одному скрипті.

- Створювати власні workflow для складних розрахунків.
- Інтегрувати PySCF з іншими Python бібліотеками (NumPy, SciPy, matplotlib).
- Розробляти власні модулі та методи.

Сучасна архітектура. PySCF використовує об'єктно-орієнтований підхід, що робить код зрозумілим та легким для модифікації. Модульна структура дозволяє використовувати тільки потрібні компоненти.

Активна розробка. Проект активно розвивається, регулярно додаються нові функції та покращується продуктивність. Спільнота користувачів надає підтримку через GitHub та форуми.

Продуктивність. Незважаючи на використання Python, критичні обчислювальні частини написані на C та оптимізовані. PySCF ефективно використовує сучасні бібліотеки лінійної алгебри (BLAS, LAPACK) та може працювати на багатоядерних системах.

1.1.3. Порівняння з іншими програмами

У таблиці 1.1 наведено порівняння PySCF з іншими популярними квантово-хімічними пакетами.

Таблиця 1.1. Порівняння квантово-хімічних програм

Програма	Ліцензія	Мова	Гнучкість	Навчання
PySCF	Відкрита	Python/C	Висока	Середнє
Gaussian	Комерційна	Fortran	Низька	Легке
ORCA	Академічна	C++	Середня	Легке
Q-Chem	Комерційна	C/C++	Середня	Середнє
Psi4	Відкрита	C++/Python	Висока	Середнє

1.2. Встановлення PySCF

1.2.1. Системні вимоги

Для роботи з PySCF необхідно:

- Python 3.7 або новіший
- NumPy версії 1.13.0 або новішої
- SciPy версії 1.0.0 або новішої
- H5py (для збереження великих масивів даних)
- 4–8 ГБ оперативної пам'яті (мінімум)

1. Знайомство з PySCF

- Сучасний процесор (бажано багатоядерний)

1.2.2. Встановлення в Linux

Найпростіший спосіб встановлення PySCF у Linux — використання `pip`:

```
# Оновлення pip
pip install --upgrade pip

# Встановлення PySCF
pip install pyscf
```

Для встановлення з додатковими можливостями:

```
# з підтримкою XC функціоналів
pip install pyscf[geomopt,dftd3,dmrgscf]
```

Альтернативно, можна встановити з вихідного коду:

```
git clone https://github.com/pyscf/pyscf.git
cd pyscf
pip install -e .
```

1.2.3. Встановлення в Windows

Для Windows рекомендується використання Anaconda:

```
# Створення нового середовища
conda create -n pyscf_env python=3.10
conda activate pyscf_env

# Встановлення PySCF
conda install -c pyscf pyscf
```

1.2.4. Встановлення в macOS

Для macOS процес аналогічний до Linux:

```
# Встановлення через pip
pip3 install pyscf

# Або через Homebrew + pip
brew install python3
pip3 install pyscf
```

1.2.5. Перевірка встановлення

Після встановлення перевіримо, чи PySCF працює коректно:

```
import pyscf
# Вивід версії
print(pyscf.__version__)
# Простий тест
from pyscf import gto, scf
mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
mf = scf.RHF(mol)
energy = mf.kernel()
print(f'Енергія H2: {energy:.6f} Ha')
```

Якщо програма виводить версію та обчислює енергію молекули H_2 , встановлення виконано успішно.

1.3. Базова структура програми на PySCF

1.3.1. Загальна схема розрахунку

Типовий розрахунок у PySCF складається з трьох основних етапів:

1. **Створення молекулярного об'єкта** — визначення геометрії системи, базисного набору, заряду та спіну
2. **Вибір та налаштування методу** — вибір квантово-хімічного методу (HF, DFT, CC тощо)
3. **Виконання розрахунку** — запуск обчислень та отримання результатів

1.3.2. Мінімальний приклад

Розглянемо найпростіший приклад розрахунку атома Гідрогену:

```
from pyscf import gto, scf
# Етап 1: Створення молекулярного об'єкта
mol = gto.M(
    atom='H 0 0 0',      # Координати атома
    basis='sto-3g',       # Базисний набір
    spin=1                # 2S (1 неспарений електрон)
)
# Етап 2: Вибір методу (UHF для відкритої оболонки)
mf = scf.UHF(mol)
# Етап 3: Виконання розрахунку
energy = mf.kernel()
# Виведення результатів
print(f'Енергія атома H: {energy:.8f} Hartree')
print(f'Енергія атома H: {energy * 27.211386:.6f} eV')
```

1.3.3. Структура для атома Гелію

Для атома з двома електронами у замкненій оболонці:

```
from pyscf import gto, scf
# Атом Гелію
mol = gto.M(
    atom='He 0 0 0',
    basis='6-31g',
    spin=0,                # Всі електрони спарені
    charge=0               # Нейтральний атом
)
# RHF для замкненої оболонки
mf = scf.RHF(mol)
energy = mf.kernel()
print(f'Енергія He: {energy:.8f} Ha')
```

1.3.4. Розрахунок з аналізом результатів

Більш детальний приклад з виведенням додаткової інформації:

```
from pyscf import gto, scf

mol = gto.M(atom='Li 0 0 0', basis='cc-pvdz', spin=1)

mf = scf.UHF(mol)
mf.verbose = 4          # Детальний вивід
energy = mf.kernel()
```

1.4. Система одиниць та конвенції

1.4.1. Атомні одиниці

PySCF використовує атомну систему одиниць (atomic units, a.u.), де:

$$\begin{aligned}\hbar &= 1 \\ m_e &= 1 \quad (\text{маса електрона}) \\ e &= 1 \quad (\text{елементарний заряд}) \\ 4\pi\epsilon_0 &= 1 \quad (\text{електрична константа})\end{aligned}$$

У цій системі:

- Енергія вимірюється в Hartree (Ha): $1 \text{ Ha} = 27.211386 \text{ eV} = 627.509 \text{ kcal/mol}$
- Відстань вимірюється в Bohr (a_0): $1 \text{ Bohr} = 0.529177 \text{ \AA}$
- Час вимірюється у а.о.: $1 \text{ a.u.} = 2.4189 \times 10^{-17} \text{ s}$

1.4.2. Конвертація одиниць

PySCF надає [модуль для конвертації](#) одиниць вимірювання та фізичних констант:

```
from pyscf.data import nist

# Конвертація енергії
energy_ha = -2.85516
energy_ev = energy_ha * nist.HARTREE2EV
energy_kcal = energy_ha * nist.HARTREE2KCAL

# Конвертація відстані
dist_bohr = 2.0
dist_angstrom = dist_bohr * nist.BOHR

# Фізичні константи
print(f'Швидкість світла: {nist.LIGHT_SPEED} a.u.')
print(f'Константа Планка: {nist.PLANCK} J·s')
print(f'Число Авогадро: {nist.AVOGADRO} mol-1)
```

1.4.3. Конвенції для атомів

Координати. Координати задаються у формі рядка або списку. За замовчуванням використовуються ангстрєми (Ångström), але можна вказати одиниці:

```
# Координати в Ångström (за замовчуванням)
mol = gto.M(atom='C 0 0 0')
# Явне вказання одиниць
mol = gto.M(atom='C 0 0 0', unit='Angstrom')
# Координати в Bohr
mol = gto.M(atom='C 0 0 0', unit='Bohr')
```

Спін. Параметр `spin` визначає $2S = N_\alpha - N_\beta$, де N_α та N_β — кількість альфа та бета електронів:

```
# Атом H (1 неспарений електрон): S=1/2, 2S=1
mol = gto.M(atom='H 0 0 0', spin=1)
# Атом He (всі спарені): S=0, 2S=0
mol = gto.M(atom='He 0 0 0', spin=0)
# Атом O у триплетному стані: S=1, 2S=2
mol = gto.M(atom='O 0 0 0', spin=2)
```

Симетрія. За замовчуванням PySCF використовує повну точкову симетрію системи:

```
# Автоматичне визначення симетрії
mol = gto.M(atom='Ne 0 0 0', symmetry=True)
print(f'Точкова група: {mol.groupname}')
# Вимкнення симетрії
mol = gto.M(atom='Ne 0 0 0', symmetry=False)
```

1.5. Документація та ресурси

1.5.1. Офіційна документація

Ресурс	URL
Головний сайт	https://pyscf.org
GitHub репозиторій	https://github.com/pyscf/pyscf
Документація API	https://pyscf.org/pyscf_api_docs/pyscf.html
Приклади коду	https://github.com/pyscf/pyscf/tree/master/examples

1.5.2. Основні публікації

Ключові статті для цитування при використанні PySCF:

1. PySCF: the Python-based simulations of chemistry framework / Q. Sun [et al.] // Wiley Interdisciplinary Reviews: Computational Molecular Science. — 2018. — Vol. 8, no. 1. — e1340. — DOI: [10.1002/wcms.1340](https://doi.org/10.1002/wcms.1340).
2. Recent developments in the PySCF program package / Q. Sun [et al.] // Journal of Chemical Physics. — 2020. — Vol. 153, no. 2. — P. 024109. — DOI: [10.1063/5.0006074](https://doi.org/10.1063/5.0006074).

1.6. Резюме

У цьому розділі ми познайомились з PySCF — потужним інструментом для квантово-хімічних розрахунків. Основні моменти:

- PySCF є сучасним, відкритим та гнучким програмним пакетом.
- Встановлення здійснюється просто через **pip** або **conda**.
- Базова структура програми включає три етапи: створення системи, вибір методу, виконання розрахунку
- PySCF використовує атомну систему одиниць.
- Доступна велика кількість документації та навчальних матеріалів.

У наступному розділі ми детально розглянемо базові об'єкти PySCF та навчимося налаштовувати параметри розрахунків для атомних систем.

2

Базові об'єкти PySCF

2.1. Молекулярний об'єкт (Mole)

2.1.1. Клас Mole — серце PySCF

Клас `gto.Mole` є центральним об'єктом у бібліотеці **PySCF** (Python-based Simulations of Chemistry Framework). Цей клас визначає і зберігає всю інформацію про атомну або молекулярну систему, яка необхідна для квантово-хімічних розрахунків. Саме з нього починається будь-який проєкт у PySCF, адже всі інші модулі (SCF, DFT, MP2, CCSD тощо) працюють із вже побудованим об'єктом `Mole`.

Об'єкт `Mole` містить такі основні дані:

- **Геометрію системи** — координати атомів у просторі (в ангстремах або борах);
- **Базисний набір** — набір функцій, на яких розкладаються молекулярні орбіталі;
- **Заряд системи та спин** (мультиплетність);
- **Інформацію про симетрію** (за потреби);
- **Одиниці вимірювання** (ангстрем, бори).

Таким чином, `Mole` виконує роль «серця» всієї програми — воно зберігає стан квантової системи і передає цю інформацію до інших підсистем для проведення обчислень.

2.1.2. Створення об'єкта Mole

Існує два основні способи створення молекулярного об'єкта: або за допомогою скороченого конструктора, або покроково. Обидва підходи рівноцінні за результатом, однак відрізняються стилем запису.

Метод 1: Використання конструктора `Mole()` Цей спосіб є найзручнішим для простих систем, коли всі параметри можна вказати безпосередньо при створенні об'єкта:

```
from pyscf import gto
```

```
# Простий спосіб
mol = gto.Mole(
    atom='Li 0 0 0',
    basis='6-31g',
    charge=0,
    spin=1
)
```

Тут:

- `atom='Li 0 0 0'` — визначає атом літію в координатах (0, 0, 0);
- `basis='6-31g'` — вказує базисний набір для розрахунків;
- `charge=0` — нейтральний атом;
- `spin=1` — означає, що система має $2S = 1$, тобто один неспарений електрон.

Функція `gto.Mole()` автоматично створює об'єкт, налаштовує всі параметри та викликає `.build()`.

Метод 2: Покрокове налаштування Цей спосіб зручний для складних систем, де потрібно задати багато параметрів або змінювати їх під час роботи:

```
from pyscf import gto

# Створення порожнього об'єкта
mol = gto.Mole()

# Налаштування параметрів
mol.atom = 'C 0 0 0'
mol.basis = 'cc-pvdz'
mol.charge = 0
mol.spin = 2 # Два неспарені електрони

# Завершення побудови (важливо!)
mol.build()
```

Важливо: При використанні другого методу обов'язково викликати команду `mol.build()`, інакше PySCF не згенерує внутрішні структури (матриці, орбіталі, таблиці інтегралів тощо), необхідні для подальших розрахунків.

2.1.3. Основні атрибути об'єкта Mole

Після побудови об'єкта можна отримати детальну інформацію про систему. Ось приклад, який демонструє найпоширеніші атрибути:

```
from pyscf import gto

mol = gto.M(atom='O 0 0 0', basis='6-31g', spin=2)

# Інформація про систему
print(f'Кількість електронів: {mol.nelectron}')
```

```

print(f'Заряд: {mol.charge}')
print(f'Спін (2S): {mol.spin}')
print(f'Кількість базисних функцій: {mol.nao_nr()}')

# Альфа та бета електрони
print(f'Альфа електронів: {mol.nelec[0]}')
print(f'Бета електронів: {mol.nelec[1]}')

# Атомна структура
print(f'Кількість атомів: {mol.natm}')
print(f'Заряди ядер: {mol.atom_charges()}')

# Базисний набір
print(f'Назва базису: {mol.basis}')

```

Ці атрибути особливо важливі для перевірки, чи правильно задані всі вхідні параметри перед запуском складних обчислень.

- `mol.nelectron` — кількість електронів у системі;
- `mol.charge` — сумарний заряд системи;
- `mol.spin` — подвоєне значення спіну (2S);
- `mol.nao_nr()` — кількість базисних орбіталей (число функцій, які будуть використані в обчисленнях);
- `mol.nelec` — кортеж `(n_alpha, n_beta)` з кількістю альфа- та бета-електронів;
- `mol.natm` — кількість атомів у системі;
- `mol.atom_charges()` — масив ядерних зарядів;
- `mol.basis` — опис обраного базисного набору.

Типовий вивід програми для атома кисню:

```

Кількість електронів: 8
Заряд: 0
Спін (2S): 2
Кількість базисних функцій: 18
Альфа електронів: 5
Бета електронів: 3
Кількість атомів: 1
Заряди ядер: [8.]
Назва базису: 6-31g

```

Такий вивід дозволяє переконатися, що система побудована коректно, а параметри відповідають фізичному змісту задачі. Наприклад, у цьому випадку маємо нейтральний атом кисню ($Z = 8$, $N_e = 8$), у якого спін $S = 1$ (два неспарені електрони).

Після побудови об'єкта `Mole` його можна передавати до будь-яких методів PySCF — від найпростіших `SCF` до корельованих методів `CCSD`, `CASCI` та інших. Тому розуміння структури і властивостей цього класу є ключем до ефективного використання всієї бібліотеки.

2.1.4. Визначення атома: різні способи

Визначення атома або атомів — це перший крок при побудові квантово-хімічної моделі. У PySCF координати та типи атомів можна задавати кіль-

кома способами. Усі вони приводять до одного й того ж результату, тому вибір форми запису залежить лише від зручності.

Спосіб 1: Рядок Цей варіант є найпростішим і найчастіше використовується для одиночних атомів або малих систем. Формат: `<атом> x y z`, де координати задаються в ангстремах за замовчуванням (або в борах, якщо `unit='Bohr'`).

```
# Один атом
mol = gto.M(atom='Ne 0 0 0', basis='def2-svp')

# Можна використовувати атомний номер
mol = gto.M(atom='10 0 0 0', basis='def2-svp') # 10 = Ne
```

Як видно, PySCF дозволяє вказувати елемент як за його хімічним символом, так і за атомним номером. У другому випадку він автоматично розпізнає елемент періодичної системи.

Спосіб 2: Список або кортеж Якщо потрібно задати кілька атомів, або якщо координати отримуються програмно (наприклад, з масиву), зручно використовувати структуру даних типу `list` або `tuple`. Кожен атом описується як пара: `[ім'я, (x, y, z)]`.

```
# Використання списку
mol = gto.M(
    atom=[['Na', (0.0, 0.0, 0.0)]],
    basis='aug-cc-pvdz'
)

# Для декількох атомів (наприклад, для порівняння)
mol = gto.M(
    atom=[
        ['H', (0.0, 0.0, 0.0)],
        ['He', (5.0, 0.0, 0.0)] # далеко один від одного
    ],
    basis='sto-3g'
)
```

Така форма є більш універсальною — вона дозволяє легко зчитувати геометрію з файлів, генерувати координати циклом або будувати складні молекули. Наприклад, молекула водню H_2 може бути задана як:

```
atom = [['H', (0, 0, 0)], ['H', (0, 0, 0.74)]]
```

де відстань у 0.74 Å відповідає типовій довжині зв'язку Н–Н.

2.1.5. Робота з іонами

Клас `Mole` дозволяє легко моделювати заряджені системи — катіони та аніони. Для цього достатньо змінити параметр `charge`, який визначає сумарний заряд системи:

$$Q = Z_{\text{ядер}} - N_{\text{електронів}}$$

Крім того, слід відповідно скоригувати `spin`, який задає подвоєне значення спіну $2S$. Для нейтральних атомів значення спіну зазвичай відповідає їхній електронній конфігурації.

```
from pyscf import gto, scf

# Нейтральний атом Li
li_atom = gto.M(atom='Li 0 0 0', basis='6-31g', charge=0, spin=1)

# Катіон Li+ (втрачено 1 електрон)
li_cation = gto.M(atom='Li 0 0 0', basis='6-31g', charge=1, spin=0)

# Аніон Li- (додано 1 електрон)
li_anion = gto.M(atom='Li 0 0 0', basis='6-31g', charge=-1, spin=1)

print(f'Li: {li_atom.nelectron} електронів')
print(f'Li+: {li_cation.nelectron} електронів')
print(f'Li-: {li_anion.nelectron} електронів')
```

У цьому прикладі можна побачити, як зміна заряду впливає на кількість електронів:

- Нейтральний атом Li має 3 електрони;
- Катіон Li — 2 електрони (втратив один);
- Аніон Li⁻ — 4 електрони (отримав додатковий).

Такі прості зміни параметрів дозволяють вивчати іонізаційні енергії, електронну спорідненість, а також порівнювати властивості нейтральних і заряджених систем.

2.1.6. Налаштування вербальності виводу

PySCF має вбудований механізм керування докладністю текстового виводу (тобто «балакучістю» програми). Це зручно, коли потрібно або бачити повну діагностику, або, навпаки, приховати проміжні повідомлення при пакетних розрахунках.

Рівень керується параметром `verbose`:

0 = тихо, 4 = стандартно, 9 = детально (debug)

```
# verbose контролює кількість виводу
# 0 = мінімум, 4 = максимум (за замовчуванням), 9 = дебаг
```

```
mol = gto.M(
    atom='F 0 0 0',
    basis='cc-pvdz',
    verbose=4 # Детальний вивід
)

# Або налаштувати пізніше
mol.verbose = 0 # Тихий режим
mol.build()
```

У практиці рекомендується:

- Використовувати `verbose=4` для навчальних цілей, щоб бачити хід побудови базису та інтегралів;
- Використовувати `verbose=0` або `1` при серійному запуску розрахунків на кластері, щоб уникнути перевантаження логів;
- Для діагностики чи налагодження коду — `verbose=7-9`, що дає максимально деталізований вивід.

Таким чином, параметр `verbose` дозволяє зручно регулювати ступінь деталізації повідомлень під час обчислень, роблячи роботу з PySCF більш контрольованою і зручною як для навчання, так і для автоматизованих досліджень.

2.1.7. Ланцюжок викликів методів (Method Chaining)

PySCF підтримує зручний синтаксис **ланцюжка викликів** (method chaining), який дозволяє компактно записувати послідовність операцій. Замість того, щоб створювати проміжні змінні для кожного кроку, можна «з'єднати» виклики методів у один вираз.

Традиційний підхід vs Chaining

Спочатку розглянемо звичайний спосіб написання коду:

```
from pyscf import gto, scf

# Крок 1: Створюємо молекулу
mol = gto.M(
    atom = 'H 0 0 0; H 0 0 0.74',
    basis = 'cc-pvdz'
)

# Крок 2: Створюємо об'єкт розрахунку
mf = scf.RHF(mol)

# Крок 3: Налаштовуємо параметри
mf.conv_tol = 1e-8
mf.max_cycle = 100
mf.verbose = 4

# Крок 4: Запускаємо розрахунок
mf.kernel()
```

```
# Крок 5: Отримуємо результат
energy = mf.e_tot
print(f'Енергія: {energy:.8f} Ha')
```

Тепер той самий код через **chaining**:

```
from pyscf import gto, scf

# Все в одному виразі
energy = (gto.M(atom='H 0 0 0; H 0 0 0.74', basis='cc-pvdz')
          .RHF()
          .set(conv_tol=1e-8, max_cycle=100, verbose=4)
          .run()
          .e_tot)

print(f'Енергія: {energy:.8f} Ha')
```

2.2. Базисні набори в PySCF

У пакеті PySCF базисний набір задається через параметр `basis` при створенні об'єкта `Mol`.

```
from pyscf import gto

mol = gto.M(atom='H 0 0 0; O 0 0 1', basis='6-31g')
print('Кількість базисних функцій:', mol.nao_nr())
```

2.2.1. Доступні бібліотечні базиси

PySCF містить велику вбудовану бібліотеку базисних наборів: ST0-3G, 6-31G, cc-pVDZ, def2-TZVP, aug-cc-pVTZ тощо. Повний список доступний у модулі `pyscf.gto.basis` або онлайн: pyscf.org/basis.html.

```
from pyscf.gto import basis
print(basis.basis_names.keys()) # усі доступні назви базисів
```

2.2.2. Задання власного базису

Користувач може визначити власні базисні функції безпосередньо у коді. Наприклад, простий базис для атома гелію можна записати так:

```
custom_basis = {
    'He': [
        [0, [ (13.6267, 0.1752), (1.99935, 0.8935) ]], # s-функції
    ]
}

mol = gto.M(atom='He 0 0 0', basis=custom_basis)
```

Також можна поєднувати бібліотечні базиси з власними:

```
mol = gto.M(atom='Li 0 0 0; H 0 0 1.6',
            basis={'Li': 'def2-SVP', 'H': custom_basis})
```

2.2.3. Рекомендації

Для тестових розрахунків підходить ST0-3G. Для точніших результатів — 6-31G* або cc-pVDZ. У розрахунках DFT зазвичай застосовують def2-TZVP або def2-SVP.

Порада

Бібліотечні базиси PySCF зберігаються у модулі `pyscf.gto.basis`. Їх можна переглядати, змінювати чи додавати власні файли у форматі Python-словників.

2.2.4. Власні базисні набори

Іноді потрібно створити власний базис (наприклад, для навчання або тестування гібридних моделей). PySCF дозволяє явно задавати коефіцієнти та експоненти базисних функцій через `gto.basis.parse()`.

CustomBasisExample.py

```
from pyscf import gto

# Визначення базису для H (тільки s-функції)
mol = gto.M(
    atom='H 0 0 0',
    basis={
        'H': gto.basis.parse('''
            H      S
                13.00773    0.019685
                1.962079    0.137977
                0.444529    0.478148
            H      S
                0.1219492    1.000000
            ''')
    })

print(f'Власний базис: {mol.nao_nr()} функцій')
```

У цьому прикладі явно визначено два набори *s*-функцій із різними коефіцієнтами. Так можна створювати спеціалізовані базиси для навчання або розширення стандартних наборів.

2.2.5. Комбінування базисів

PySCF підтримує **гібридні базиси** — різні для різних атомів у тій самій системі. Це корисно, коли потрібно зменшити обчислювальні витрати, наприклад, використовуючи спрощений базис для водню, а розширений — для важчих атомів.

```
# Для складних систем можна використовувати різні базиси
# (хоча для атомів це рідко потрібно)
mol = gto.M(
    atom='''
        H 0 0 0
        He 5 0 0
    ''',
    basis={
        'H': 'sto-3g',
        'He': 'cc-pvdz'
    }
)
```

У результаті PySCF автоматично комбінує обидва базиси при побудові молекулярних орбіталей.

2.2.6. Інформація про базисний набір

Іноді потрібно дослідити, які саме функції використовуються у вибраному базисі — їхні типи, кількість та мітки. Для цього можна скористатися внутрішніми структурами об'єкта `Mole`.

BasisSetInfo.py

```
from pyscf import gto

mol = gto.M(atom='C 0 0 0', basis='6-31g')

# Детальна інформація
print('Базисні функції по типу:')
for atom_idx in range(mol.natm):
    symbol = mol.atom_symbol(atom_idx)
    print(f'\nАтом {symbol}:')

    # Кількість функцій кожного типу
    basis_atom = mol._basis[symbol]
    for shell in basis_atom:
        l_quantum = shell[0] # Азимутальне квантове число
        n_contractions = len(shell[1:])

        shell_type = ['s', 'p', 'd', 'f', 'g'][l_quantum]
        print(f' {shell_type}-тип: {n_contractions} contracted')

# Діапазони базисних функцій для атома
ao_labels = mol.ao_labels()
print(f'\nМітки базисних функцій:')
for i, label in enumerate(ao_labels):
    print(f'{i}: {label}')
```

Так можна отримати повну структуру базису — скільки функцій кожного типу (*s*, *p*, *d*), які індекси мають їхні атомні орбіталі, та як вони нумеруються в PySCF. Ця інформація важлива при аналізі орбіталей, побудові щільності або інтегралів.

2.3. Симетрія в атомних розрахунках

Симетрія відіграє ключову роль у квантово-хімічних розрахунках. Вона дозволяє:

- скоротити обчислювальні витрати (менше інтегралів та менші матриці),
- розпізнавати вироджені орбіталі та їх симетрійні властивості,
- аналізувати структуру електронних станів через іррепи (незвідні представлення),
- уникати «зайвих» розв'язків при самоузгодженні.

У PySCF симетрія автоматично враховується при побудові молекулярного об'єкта, якщо активувати опцію `symmetry=True`. Для атомів сферична симетрія апроксимується підгрупами типу `D2h`, що сумісні з декартовими координатами.

2.3.1. Точкові групи симетрії атомів

Ізольований атом у строгому сенсі має повну сферичну симетрію $SO(3)$. Однак у PySCF, через використання декартових базисних функцій, використовується одна з підгруп, зазвичай `D2h`. Це дає змогу використовувати блокову структуру матриць і автоматично класифікувати орбіталі за іррепами.

SymmetryPointGroupsExample.py

```
from pyscf import gto

# Автоматичне визначення симетрії
mol = gto.M(
    atom='Ne 0 0 0',
    basis='cc-pvdz',
    symmetry=True # Автоматичне визначення точкової групи
)

print(f'Точкова група: {mol.groupname}')
print(f'Топологічна група: {mol.topgroup}')

# Для атома результат, як правило: D2h або подібна підгрупа
```

Пояснення:

- `groupname` — ідентифікатор підгрупи (наприклад, `D2h`, `C2v`, `Cs`);
- `topgroup` — вища група симетрії, до якої належить дана підгрупа.

2.3.2. Використання симетрії

Симетрія допомагає прискорити розрахунок, оскільки гамільтоніан та інші оператори блокуються відповідно до іррепів. Тобто інтеграли між функціями з різних симетрій не обчислюються — що значно зменшує обсяг роботи.

UsingSymmetryExample.py

```

from pyscf import gto, scf

# 3 симетрією (швидше, менше пам'яті)
mol_sym = gto.M(
    atom='Ar 0 0 0',
    basis='cc-pvdz',
    symmetry=True
)
mf_sym = scf.RHF(mol_sym)
e_sym = mf_sym.kernel()

# Без симетрії
mol_nosym = gto.M(
    atom='Ar 0 0 0',
    basis='cc-pvdz',
    symmetry=False
)
mf_nosym = scf.RHF(mol_nosym)
e_nosym = mf_nosym.kernel()

print(f'3 симетрією: {e_sym:.8f} Ha')
print(f'Без симетрії: {e_nosym:.8f} Ha')
print(f'Різниця: {abs(e_sym - e_nosym):.2e} Ha')

```

Коментар

Різниця в енергіях практично нульова (обидва методи еквівалентні з фізичної точки зору), але симетрійний розрахунок потребує значно менше часу і пам'яті.

2.3.3. Симетрія орбіталей

PySCF автоматично визначає симетрійні властивості орбіталей після розрахунку. Це особливо корисно для аналізу заповнених і віртуальних рівнів, побудови діаграм енергетичних рівнів та порівняння з експериментом.

OrbitalSymmetryExample.py

```

from pyscf import gto, scf

mol = gto.M(
    atom='N 0 0 0',
    basis='cc-pvdz',
    spin=3, # Основний стан 4S
    symmetry=True
)

mf = scf.UHF(mol)
mf.kernel()

# Орбітальна симетрія
if mol.symmetry:
    orbsym = scf.hf_symm.get_orbsym(mol, mf.mo_coeff[0])
    from pyscf.symm import label_orb_symm

    labels = label_orb_symm(mol, mol.irrep_name, mol.symm_orb,
                           mf.mo_coeff[0])

    print('\nСиметрія заповнених орбіталей (альфа):')
    for i in range(mol.nelec[0]):

```

```
print(f' MO {i+1}: {labels[i]}')
```

Коментарі:

- `mol.irrep_name` — список назв іррепів¹;
- `mol.symm_orb` — матриці симетричних орбіталей;
- `label_orb_symm()` — функція, що відображає орбіталі у відповідні іррепи;
- `get_orbsym()` — отримує симетрії молекулярних орбіталей з урахуванням побудованих коефіцієнтів.

Це дає змогу, наприклад, визначити які орбіталі мають однакову симетрію, а які не взаємодіють між собою.

2.3.4. Коли вимикати симетрію

Хоча симетрія зазвичай корисна, існують ситуації, коли її слід **вимкнути**:

- При моделюванні **збуджених станів**, які порушують симетрію основного стану.
- При побудові **поверхонь потенційної енергії (PES)**, де геометрія змінюється і симетрія зникає.
- Якщо виникають **проблеми з конвергенцією** — часто через жорсткі симетрійні обмеження.
- Коли необхідно вручну **змінювати орбіталі** або аналізувати змішування між різними симетріями.

У таких випадках достатньо задати:

```
mol = gto.M(atom='...', basis='...', symmetry=False)
```

Підсумок: Використання симетрії — це не лише «оптимізація швидкості», а й **фізично обґрунтований підхід**, що дозволяє зрозуміти структуру електронних рівнів, виродження та природу хімічних зв'язків.

2.4. Спін та мультиплетність

2.4.1. Визначення спінового стану

У квантовій хімії поняття спіну має фундаментальне значення. Кожен електрон характеризується спіном $s = \frac{1}{2}$, тобто він може перебувати в одному

¹Незвідне представлення (irrep) — фундаментальне поняття теорії груп, що описує, як функція (зокрема молекулярна орбіталь) трансформується під дією операцій симетрії молекули. Кожне ірредуцибельне представлення відповідає певному типу симетрії: наприклад, для групи C_{2v} це A_1, A_2, B_1, B_2 , а для групи D_{2h} — $A_g, B_{1g}, B_{2g}, B_{3g}, A_u, B_{1u}, B_{2u}, B_{3u}$. Всі орбіталі, що належать до одного й того ж ірреп, мають однакову симетрію і можуть змішуватися між собою, тоді як орбіталі різних ірреп не взаємодіють.

з двох можливих спінових станів — «вгору» (α) або «вниз» (β).

Для багаточастинкової системи повний спін S визначається як векторна сума спінів усіх електронів. У більшості практичних розрахунків нас цікавить тільки величина S (а не його напрям).

В PySCF параметр `spin` задає величину $2S$, тобто різницю між кількістю α - та β -електронів:

$$2S = N_{\alpha} - N_{\beta} \quad (2.1)$$

Звідси:

$$S = \frac{N_{\alpha} - N_{\beta}}{2}, \quad M = 2S + 1$$

де M — мультиплетність (кількість можливих орієнтацій спіну системи в магнітному полі).

- $S = 0 \Rightarrow$ синглет ($M = 1$)
- $S = \frac{1}{2} \Rightarrow$ дублет ($M = 2$)
- $S = 1 \Rightarrow$ триплет ($M = 3$)
- $S = \frac{3}{2} \Rightarrow$ квартет ($M = 4$)

SpinExamples.py

```
from pyscf import gto

# Атом H: S=1/2, 2S=1, дублет (M=2)
h = gto.M(atom='H 0 0 0', basis='cc-pvdz', spin=1)

# Атом He: S=0, 2S=0, синглет (M=1)
he = gto.M(atom='He 0 0 0', basis='cc-pvdz', spin=0)

# Атом O: основний стан 3P, S=1, 2S=2, триплет (M=3)
o = gto.M(atom='O 0 0 0', basis='cc-pvdz', spin=2)

# Атом N: основний стан 4S, S=3/2, 2S=3, квартет (M=4)
n = gto.M(atom='N 0 0 0', basis='cc-pvdz', spin=3)

print(f'H: {h.nelec}, 2S={h.spin}, M={h.spin+1}')
print(f'He: {he.nelec}, 2S={he.spin}, M={he.spin+1}')
print(f'O: {o.nelec}, 2S={o.spin}, M={o.spin+1}')
print(f'N: {n.nelec}, 2S={n.spin}, M={n.spin+1}')
```

Цей код створює атомні молекули з різними значеннями параметра `spin`. В об'єкті `mol.nelec` PySCF зберігає кортеж (N_{α}, N_{β}) , що дозволяє контролювати спінову структуру системи.

2.4.2. Розподіл альфа- та бета-електронів

У багатоспінових системах важливо знати, як електрони розподілені за спіновими підрівнями. В PySCF ця інформація визначається автоматично на основі параметра `spin`, однак її можна перевірити вручну.

PrintElectronConfig.py

```
from pyscf import gto
```

```
def print_electron_config(atom_symbol, charge=0, spin=0):
    mol = gto.M(
        atom=f'{atom_symbol} 0 0 0',
        basis='sto-3g',
        charge=charge,
        spin=spin
    )

    n_alpha, n_beta = mol.nelec
    print(f'{atom_symbol} (charge={charge}, 2S={spin}):')
    print(f'  Всього електронів: {mol.nelectron}')
    print(f'  α-електронів: {n_alpha}')
    print(f'  β-електронів: {n_beta}')
    print(f'  Неспарених: {n_alpha - n_beta}\n')

# Приклади
print_electron_config('Li', charge=0, spin=1)    # Li (2s1)
print_electron_config('C', charge=0, spin=2)    # C (³P)
print_electron_config('O', charge=0, spin=2)    # O (³P)
print_electron_config('O', charge=-1, spin=1)   # O⁻ (дублет)
```

Ця функція показує, як саме PySCF обчислює кількість α - та β -електронів. Наприклад, для нейтрального атома кисню ($Z = 8$) маємо 8 електронів. При $2S = 2$ отримаємо:

$$N_{\alpha} = 5, \quad N_{\beta} = 3$$

тобто два неспарені електрони — саме це відповідає триплетному стану (3P).

2.4.3. Вибір правильного спіну

Необхідно завжди задавати правильний спін для основного стану атома або молекули, інакше SCF-процедура може не збігатися або дати фізично некоректний результат.

Правильне значення S визначається на основі *правил Хунда*:

1. Електрони займають орбіталі так, щоб сумарний спін S був максимальним.
2. Для даного S максимізується орбітальний момент L .
3. Для менш ніж напівзаповненої оболонки найнижчий рівень має $J = |L - S|$, а для більш ніж напівзаповненої — $J = L + S$.

Для атомів другого періоду ці правила дають такі основні стани:

2.4.4. Енергетичне підтвердження спінових станів

Нижче наведено код, який обчислює енергії атомів другого періоду для їхніх правильних спінових станів. Для відкрито-оболонкових систем (`spin > 0`) використовується UHF (неспарені електрони), а для синглетів (`spin = 0`) — RHF.

EnergyConfirmation2ndPeriod.py

```
from pyscf import gto, scf
```

Таблиця 2.1. Основні спінові стани атомів другого періоду

Атом	Конфігурація	Терм	S	2S
Li	[He] 2s ¹	² S	1/2	1
Be	[He] 2s ²	¹ S	0	0
B	[He] 2s ² 2p ¹	² P	1/2	1
C	[He] 2s ² 2p ²	³ P	1	2
N	[He] 2s ² 2p ³	⁴ S	3/2	3
O	[He] 2s ² 2p ⁴	³ P	1	2
F	[He] 2s ² 2p ⁵	² P	1/2	1
Ne	[He] 2s ² 2p ⁶	¹ S	0	0

```
# Правильні спінові стани для другого періоду
atoms_2nd_period = [
    ('Li', 1), ('Be', 0), ('B', 1), ('C', 2),
    ('N', 3), ('O', 2), ('F', 1), ('Ne', 0)
]

print('Основні стани атомів другого періоду:\n')
for symbol, spin in atoms_2nd_period:
    mol = gto.M(
        atom=f'{symbol} 0 0 0',
        basis='6-31g',
        spin=spin
    )
    mf = scf.UHF(mol) if spin > 0 else scf.RHF(mol)
    mf.verbose = 0
    energy = mf.kernel()

    mult = spin + 1
    print(f'{symbol:2s}: 2S={spin}, M={mult}, E={energy:12.6f} Ha')
```

Результати дозволяють перевірити, що обраний спіновий стан дійсно відповідає мінімуму енергії для даного атома. Якщо спробувати змінити `spin`, наприклад для атома азоту задати `spin=1`, отримаємо енергію вищу на кілька десятків міліГартрі — тобто триплет виявиться збудженим станом відносно правильного к्वартету.

2.4.5. Практичні зауваження

- У PySCF спін задається *на всю систему*, тому при моделюванні молекул потрібно враховувати сумарний спін усіх атомів.
- Для молекул із непарним числом електронів (odd number of electrons) `spin` завжди має бути непарним.
- Неправильно заданий спін часто призводить до SCF not converged.
- При використанні DFT (наприклад, `scf.UKS`) параметр `spin` також впливає на спінову поляризацію густини.

2.4.6. Резюме

- **Клас Mole** — центральний об'єкт, що містить всю інформацію про систему.
- **Базисні набори** — від мінімальних (STO-3G) до великих (cc-pVQZ); вибір залежить від задачі та компромісу точність/вартість.
- **Симетрія** — прискорює розрахунки, але може обмежувати гнучкість при спонтанному руйнуванні симетрії.
- **Спін** — критичний параметр для правильного опису основного стану відкрито-оболонкових систем.

2.4.7. Контрольні запитання

1. Яка різниця між методами створення об'єкта Mole через `gto.M()` та покроково?
2. Чому для атома Карбону у основному стані використовується `spin=2`?
3. Коли слід використовувати дифузні функції (aug- базиси)?
4. Що означає параметр `conv_tol` і як він впливає на точність?
5. Які методи початкового наближення найкраще підходять для атомів?

2.4.8. Завдання для самостійної роботи

1. Створіть функцію, яка автоматично визначає правильний спін для атомів першого та другого періодів.
2. Порівняйте енергії атома Неону з різними базисними наборами (STO-3G, 6-31G, cc-pVDZ, cc-pVTZ) та побудуйте графік залежності енергії від розміру базису.
3. Дослідіть вплив симетрії на швидкість розрахунку для атома Аргону: порівняйте час виконання з `symmetry=True` та `symmetry=False`.
4. Обчисліть енергію іонізації атома Літію як різницю енергій Li та Li⁺.
5. Для атома Заліза (Fe) з різними спіновими станами (2S = 2, 4, 6) знайдіть, який стан має найнижчу енергію.

У наступному розділі ми детально розглянемо метод Хартрі-Фока та його застосування до розрахунків атомів.

3

Метод Хартрі-Фока

3.1. Теоретичні основи методу Хартрі-Фока

Метод Хартрі-Фока (HF) — це фундаментальне наближення для розв’язання рівняння Шредінгера багатоелектронних систем, яке вводить концепцію «середнього поля» для спрощення взаємодії електронів. Воно лежить в основі більшості квантово-хімічних обчислень і слугує відправною точкою для DFT чи пост-HF методів.

Ключовим теоретичним підґрунтям методу є **варіаційний принцип**, згідно з яким наближена хвильова функція вибирається так, щоб мінімізувати середню енергію системи:

$$E[\Psi] = \frac{\langle \Psi | \hat{H} | \Psi \rangle}{\langle \Psi | \Psi \rangle},$$

що гарантує, що отримане значення E_{HF} є *верхньою межею* до точної енергії основного стану. Це означає, що наша наближена енергія завжди буде вищою за справжню енергію, і чим точніше Ψ , тим ближче ми наближаємося до неї.

3.1.1. Рівняння Хартрі-Фока

У межах наближення Хартрі-Фока хвильова функція представляється одним детермінантом Слейтера, який забезпечує антисиметричність і враховує принцип Паулі:

$$\Psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) = \frac{1}{\sqrt{N!}} \begin{vmatrix} \psi_1(\mathbf{r}_1) & \psi_2(\mathbf{r}_1) & \cdots & \psi_N(\mathbf{r}_1) \\ \psi_1(\mathbf{r}_2) & \psi_2(\mathbf{r}_2) & \cdots & \psi_N(\mathbf{r}_2) \\ \vdots & \vdots & \ddots & \vdots \\ \psi_1(\mathbf{r}_N) & \psi_2(\mathbf{r}_N) & \cdots & \psi_N(\mathbf{r}_N) \end{vmatrix}, \quad (3.1)$$

де $\psi_i(\mathbf{r})$ — спін-орбіталі (добуток просторової орбіталі $\varphi_i(\mathbf{r})$ та спінової функції α або β). Просторова орбіталь описує розподіл електронної густини $|\varphi_i(\mathbf{r})|^2$ — імовірність знайти електрон у точці $\mathbf{r}$. Для фізичної однозначності і зручності обчислень спін-орбіталі беруть ортонормованими.

Унаслідок варіаційної мінімізації функціонала енергії (3.1) отримують **рівняння Хартрі-Фока** — систему одноелектронних рівнянь для спін-орбіталей:

$$\hat{F}\psi_i = \varepsilon_i\psi_i, \quad (3.2)$$

де оператор Фока $\hat{F} = \hat{h} + \sum_{j=1}^N (\hat{J}_j - \hat{K}_j)$. Тут $\hat{h}$ — одноелектронний гамільтоніан (кінетична енергія + притягання до ядра), $\hat{J}_j$ — кулонівський оператор (електрон-електронне притягання в середньому полі), $\hat{K}_j$ — обмінний оператор (враховує антисиметрію).

Ця форма перетворює багатоелектронну задачу на набір одноелектронних рівнянь, де кожен електрон «бачить» фіксоване поле від інших.

3.1.2. Енергія Хартрі-Фока

Повна енергія в HF-наближенні виражається через одно- та дво електронні інтеграли:

$$E_{\text{HF}} = \sum_{i=1}^N h_{ii} + \frac{1}{2} \sum_{i,j}^N (J_{ij} - K_{ij}) + V_{\text{NN}}, \quad (3.3)$$

де h_{ii} — одноелектронні інтеграли, J_{ij} та K_{ij} — кулонівський та обмінний інтеграли, V_{NN} — між'ядерне відштовхування (для атомів = 0).

Для перевірки коректності розв'язку (стаціонарність) використовується **вірйальне співвідношення**:

$$2\langle T \rangle + \langle V \rangle = 0 \quad \Rightarrow \quad \frac{\langle V \rangle}{\langle T \rangle} = -2,$$

де T та V — кінетична та потенціальна енергія. Це співвідношення перевіряє баланс між кінетичною та потенціальною енергією, і його виконання свідчить про коректність SCF-збіжності (див. підр. 3.5).

3.1.3. Рівняння Хартрі-Фока в базисному представленні

Розглянемо представлення спін-орбіталей ψ_i (одноелектронних функцій у рівнянні Хартрі-Фока (3.2)). Якщо не робити жодних припущень про їх вигляд, то неможливо виконувати розв'язок рівнянь (3.2). Однак, якщо кожному орбіталі подати як лінійну комбінацію заздалегідь вибраних *базисних функцій* — функцій, схожих до воднеподібних, що описують поведінку електрона поблизу ядра, — рівняння можна звести до матричної форми, а процедуру розв'язку до чисельної процедури, придатної для реалізації на комп'ютері.

Ці базисні функції утворюють *базисний набір* розмірності M :

$$\{\chi_\mu\}_{\mu=1}^M.$$

Тоді спин-орбіталь виражається як

$$\psi_i = \gamma(\sigma) \sum_{\mu=1}^M C_{\mu i} \chi_{\mu},$$

де $\gamma(\sigma)$ — спінова функція, що визначає проєкцію спіну електрона (α або β), а $\chi_{\mu}(\mathbf{r})$ — залежать лише від просторових координат.

Поняття *базису* дозволяє звести задачу до обчислення коефіцієнтів $C_{\mu i}$, що робить метод Хартрі-Фока чисельним.

Ключовим поняттям є **матриця густини**:

$$D_{\mu\nu} = \sum_i^N C_{\mu i} C_{\nu i}, \quad (3.4)$$

де N — число електронів. Матриця $\mathbf{D}$ зберігає всю інформацію про електронну густину:

$$\rho(\mathbf{r}) = \sum_{\mu\nu} D_{\mu\nu} \chi_{\mu} \chi_{\nu}.$$

і дозволяє обчислювати середні значення спостережуваної O :

$$\langle \hat{O} \rangle = \text{Tr}(\mathbf{D}\mathbf{O}) = \sum_{\mu\nu} D_{\mu\nu} O_{\mu\nu}. \quad (3.5)$$

Використовуючи базис, можна розрахувати матрицю одноелектронного гамільтоніану: $h_{\mu\nu} = T_{\mu\nu} + V_{\mu\nu}^{\text{eN}}$, з інтегралами

$$T_{\mu\nu} = \int \chi_{\mu} \left(-\frac{1}{2} \nabla^2 \right) \chi_{\nu} d\mathbf{r}, \quad V_{\mu\nu}^{\text{nuc}} = - \sum_A Z_A \int \frac{\chi_{\mu} \chi_{\nu}}{|\mathbf{r} - \mathbf{R}_A|} d\mathbf{r}.$$

Матриці двоелектронних інтегралів (або інтегралів відштовхування (*eri* — *electron repulsion integrals*)) є ключовими для врахування взаємодії між електронами в методі HF. Вони записуються в базисі атомних орбіталей $\{\chi_{\mu}\}$ у фізичній нотації (Chemist's notation) як:

$$(\mu\nu|\lambda\sigma) = \iint \frac{\chi_{\mu}^*(\mathbf{r}_1) \chi_{\nu}(\mathbf{r}_1) \chi_{\lambda}^*(\mathbf{r}_2) \chi_{\sigma}(\mathbf{r}_2)}{|\mathbf{r}_1 - \mathbf{r}_2|} d\mathbf{r}_1 d\mathbf{r}_2, \quad (3.6)$$

де $|\mathbf{r}_1 - \mathbf{r}_2|^{-1}$ — кулонівський оператор відштовхування. Для реальних базисних функцій (як у стандартних HF-обчисленнях) комплексне спряження опускається: $\chi_{\mu}^* \rightarrow \chi_{\mu}$.

Ці інтеграли симетричні: $(\mu\nu|\lambda\sigma) = (\nu\mu|\lambda\sigma) = (\lambda\sigma|\mu\nu) = (\mu\nu|\sigma\lambda)$. Кількість таких інтегралів для базису розміром K — $O(K^4)$, що робить їх обчислення обчислювально витратним (використовуються апроксимації, як RI або Cholesky-розклад).

Кулонівський і обмінний оператори у просторі атомних орбіталей описуються відповідними матрицями $\mathbf{J}$ та $\mathbf{K}$, елементи яких визначаються як:

$$J_{\mu\nu} = \sum_{\lambda\sigma} D_{\lambda\sigma}(\mu\nu|\lambda\sigma), \quad K_{\mu\nu} = \sum_{\lambda\sigma} D_{\lambda\sigma}(\mu\lambda|\nu\sigma),$$

а сама матриця виглядає Фока виглядає як:

$$F_{\mu\nu} = h_{\mu\nu} + \sum_{\lambda\sigma} D_{\lambda\sigma} \left[(\mu\nu|\lambda\sigma) - \frac{1}{2}(\mu\lambda|\nu\sigma) \right]. \quad (3.7)$$

У матричній формі канонічні рівняння Хартрі-Фока для молекулярних орбіталей записуються як узагальнена задача на власні значення:

$$\sum_{\nu=1}^M F_{\mu\nu} C_{\nu i} = \varepsilon_i \sum_{\nu=1}^M S_{\mu\nu} C_{\nu i}, \quad \mu = 1, \dots, M; \quad i = 1, \dots, N, \quad (3.8)$$

де M — розмір базису, N — кількість електронів, $S_{\mu\nu}$ — елементи матриці перекриття:

$$S_{\mu\nu} = \int \chi_{\mu}^*(\mathbf{r}) \chi_{\nu}(\mathbf{r}) d\mathbf{r},$$

$C_{\nu i}$ — коефіцієнти розкладу, ε_i — орбітальні енергії.

Ця система N рівнянь для кожної орбіталі i еквівалентна узагальненій матричній формі (де стовпці матриці $\mathbf{C}$ — вектори коефіцієнтів орбіталей):

$$\mathbf{FC} = \mathbf{SC}\varepsilon. \quad (3.9)$$

Розмірності матриць в рівняннях Хартрі-Фока

Матриця Фока $\mathbf{F}$ в базисі атомних орбіталей має розмірність $M \times M$. Така сама розмірність і в матриці $\mathbf{S}$ інтегралів перекривання атомних функцій. Матриця $\mathbf{C}$ коефіцієнтів розкладання молекулярних орбіталей за атомним базисом має розмірність $M \times N$. І нарешті, діагональна матриця орбітальних енергій має розмірність $N \times N$:

$$\underbrace{\left(\begin{array}{ccccc} \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \mathbf{F} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{array} \right)}_M \underbrace{\left(\begin{array}{ccc} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{array} \right)}_N = \underbrace{\left(\begin{array}{ccccc} \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \mathbf{S} & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{array} \right)}_M \underbrace{\left(\begin{array}{ccc} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{array} \right)}_N \underbrace{\left(\begin{array}{ccc} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{array} \right)}_N \varepsilon$$

Матриці $T_{\mu\nu}$, $V_{\mu\nu}^{\text{nuc}}$, $\left[(\mu\nu|\lambda\sigma) - \frac{1}{2}(\mu\lambda|\nu\sigma) \right]$ та $S_{\mu\nu}$, обчислюється один раз на початку і використовується в усіх SCF-ітераціях. PySCF вищевказані інтеграли обчислює автоматично:

```
from pyscf import gto
import numpy as np

mol = gto.M(atom='H 0 0 0; H 0 0 1.0', basis='sto-3g')
K = mol. nao # Розмір базису K=2 для H2 STO-3G
```

```
# Одноелектронні інтеграли (K x K матриці)
T = mol.intor('int1e_kin') # Кінетична енергія
V = mol.intor('int1e_nuc') # Притягання до ядер
S = mol.intor('int1e_ovlp') # Матриця перекриття

# Двоелектронні інтеграли (K x K x K x K тензор)
eri = mol.intor('int2e') # (μν|λσ) у chemist's notation
```

Матриця густини та матриця фока обчислюються вже після запуску SCF-процедури:

```
from pyscf import gto, scf
import numpy as np

mol = gto.M(atom='H 0 0 0; H 0 0 1.0', basis='sto-3g')

# Запуск SCF процедури
mf = scf.RHF(mol).run()

# Обчислення матриці густини
D = mf.make_rdm1()

# Одноелектронний гамільтоніан h
h = mf.get_hcore()

# Двоелектронні інтеграли
eri = mol.intor('int2e')

# Обчислення J та K (залежні від D!)
# np.einsum() - реалізація правила Ейнштейна для сумування індексів у бібліотеці NumPy.
J = np.einsum('ijkl,kl->ij', eri, D) # J_μν = ∑_{λσ} D_{λσ} (μν|λσ)
K = np.einsum('iljk,kj->ij', eri, D) # K_μν = ∑_{λσ} D_{λσ} (μλ|νσ)

# Матриця Фока для RHF (вручну для ілюстрації)
F_manual = h + J - 0.5 * K

# Матриця Фока з PySCF (автоматично обчислена в SCF)
F_pyscf = mf.get_fock()

# Орбітальні енергії
orbital_energies = mf.mo_energy

# Вивід для перевірки
print('h:\n', np.round(h, 6))
print('J:\n', np.round(J, 6))
print('K:\n', np.round(K, 6))
print('F (вручну):\n', np.round(F_manual, 6))
print('F (з PySCF):\n', np.round(F_pyscf, 6))
```

Компоненти енергії системи, як спостережувані величини, виражається через матрицю густини (див. (3.5)):

$$T = \sum_{\mu\nu} D_{\mu\nu} T_{\mu\nu}, \quad V_{eN} = \sum_{\mu\nu} D_{\mu\nu} V_{\mu\nu}^{\text{nuc}}, \quad V_{ee} = \frac{1}{2} \sum_{\mu\nu} D_{\mu\nu} (J_{\mu\nu} - K_{\mu\nu}),$$

де T — кінетична енергія електронів, V_{eN} — потенціальна енергія взаємодії електронів із ядрами, а V_{ee} — енергія електрон-електронної взаємодії, яка є

ефективним середнім описом кулонівського відштовхування між електронами з урахуванням обмінного (квантового) внеску, що виникає внаслідок антисиметрії хвильової функції. Сама енергія дорівнює:

$$E_{\text{HF}} = \sum_{\mu\nu} D_{\mu\nu} h_{\mu\nu} + \frac{1}{2} \sum_{\mu\nu} D_{\mu\nu} (J_{\mu\nu} - K_{\mu\nu}) + V_{\text{NN}}, \quad (3.10)$$

Розрахунок енергії та компонент в PySCF:

```
from pyscf import gto, scf
import numpy as np

# Створення молекули (H2, STO-3G)
mol = gto.M(atom='H 0 0 0; H 0 0 1.0', basis='sto-3g')

# Інтеграли, які обчислюються на основі базисних функцій
T_int = mol.intor('int1e_kin') # Кінетична енергія
V_eN_int = mol.intor('int1e_nuc') # Притягання до ядер
eri = mol.intor('int2e') # Двоелектронні інтеграли ( $\mu\nu|\lambda\sigma$ )

# Між'ядерне відштовхування
V_NN = mol.energy_nuc()

# Запуск SCF процедури (RHF)
mf = scf.RHF(mol).run(verbose=0)

# Матриця густини
D = mf.make_rdm1()

veff = mf.get_veff(mol, D)

# Матриця одностинкових інтегралів
h = mf.get_hcore() # h = T_int + V_eN_int

# Компоненти енергії через поелементне множення
T = (D * T_int).sum() # Кінетична енергія <T> = Tr(D T)
V_eN = (D * V_eN_int).sum() # Притягання електрона до ядра <V_eN> = Tr(D V_eN)
V_ee = 1/2 * (D * veff).sum() # Енергія електрон-електронної взаємодії

# HF енергія системи
E_tot = T + V_eN + V_ee + V_NN # Повна HF-енергія
# Вбудований спосіб
# E_tot = mf.e_tot

# Вивід
print('≡≡≡ Компоненти енергії (hartree) ≡≡≡')
print(f'T = {T:.6f}')
print(f'V_eN = {V_eN:.6f}')
print(f'V_ee = {V_ee:.6f}')
print(f'V_NN = {V_NN:.6f}')
print(f'E_HF = T + V_eN + V_ee + V_NN = {T:.6f} {V_eN:.6f} + {V_ee:.6f} + {V_NN:.6f} =')
print(f'↪ {E_tot:.6f}')
```

У PySCF усі складові енергії системи — кінетична, електрон–ядерна, електрон–електронна, між'ядерна та повна енергія Хартрі–Фока — можуть бути отримані безпосередньо через набір спеціалізованих API-методів

класу `scf.hf.SCF`. Для кожного з них передбачено окремі функції, наприклад, `energy_elec()` для електронної частини, `energy_nuc()` для ядерної взаємодії, `energy_tot()` для повної енергії, а також допоміжні процедури `get_hcore()`, `get_jk()`, `get_veff()` та `get_fock()` для побудови окремих операторів і потенціалів, що входять до виразу енергії. Завдяки цьому енергію можна отримати кількома способами: безпосередньо з атрибутів об'єкта SCF після виконання `mf.run()`, або вручну, обчислюючи внески через інтеграли та матриці густини. Повний список методів наведено у вихідному коді PYSCF у модулі `pyscf/scf/hf.py`, де всі API-команди задокументовано з прикладами використання.

3.1.4. SFC-процедура

SCF-процедура — ітераційна: з початкового наближення $\mathbf{D}^{(0)}$ послідовно будується $\mathbf{F}^{(k)}$, розв'язується (3.9) для $\mathbf{C}^{(k+1)}$, оновлюється $\mathbf{D}^{(k+1)} = 2 \sum_i^{\text{occ}} \mathbf{C}_i (\mathbf{C}_i)^T$, і цикл триває до збіжності $\|\mathbf{D}^{(k+1)} - \mathbf{D}^{(k)}\| < 10^{-8}$. Тоді обчислюється остаточно E_{HF} та перевіряється віріальне співвідношення. В програмі `c HF_procedure.py` показуються всі етапи RHF розрахунку, а на (рис. 3.1) показана блок-схема процесу.

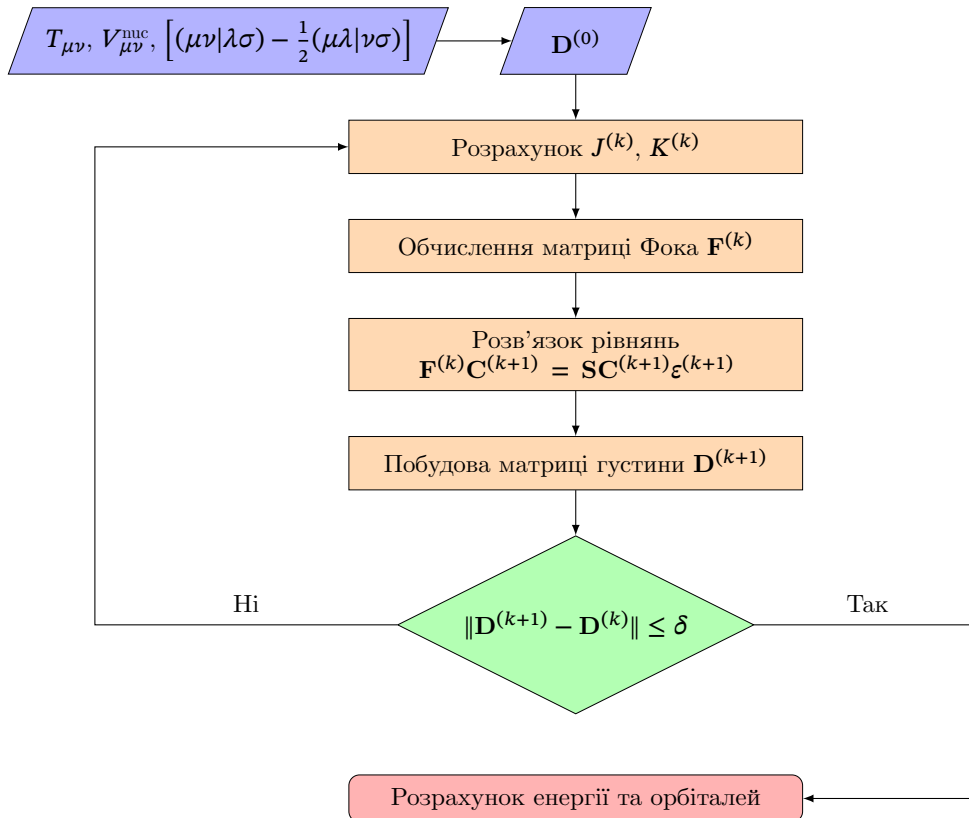


Рис. 3.1. Блок-схема SFC-процедури.

Метод DIIS — прискорення збіжності SCF

У стандартному SCF-алгоритмі ми на кожній ітерації будуємо нову матрицю Фока $\mathbf{F}^{(n)}$ з поточної матриці густини $\mathbf{D}^{(n-1)}$, розв'язуємо задачу на власні значення та отримуємо нову матрицю густини $\mathbf{D}^{(n)}$. Цей процес повторюється до збіжності.

Проблема у тому, що такий простий ітераційний процес може збігатися дуже повільно, особливо для складних систем. Іноді енергія навіть осцилює між кількома значеннями і не досягає стабільного результату.

Метод **DIIS** (Direct Inversion in the Iterative Subspace — пряма інверсія в ітераційному підпросторі), запропонований Пітером Пулаєм у 1980 році, вирішує цю проблему наступним чином:

Замість того, щоб використовувати лише матрицю Фока з поточної ітерації, DIIS *запам'ятовує* кілька попередніх матриць Фока

$$\mathbf{F}^{(n-k)}, \mathbf{F}^{(n-k+1)}, \dots, \mathbf{F}^{(n-1)}, \mathbf{F}^{(n)}$$

та будує оптимальну лінійну комбінацію:

$$\mathbf{F}_{\text{DIIS}}^{(n)} = \sum_{i=1}^m c_i \mathbf{F}^{(n-m+i)} \quad (3.11)$$

де коефіцієнти c_i підбираються так, щоб мінімізувати похибку самоузгодженості.

Що таке похибка самоузгодженості?

Якщо рішення ідеально самоузгоджене, то виконується комутаційне співвідношення:

$$[\mathbf{F}, \mathbf{DS}] = \mathbf{FDS} - \mathbf{SDF} = \mathbf{0} \quad (3.12)$$

Похибка (residual) на i -тій ітерації — це відхилення від цієї умови:

$$\mathbf{e}^{(i)} = \mathbf{F}^{(i)}\mathbf{D}^{(i)}\mathbf{S} - \mathbf{SD}^{(i)}\mathbf{F}^{(i)} \quad (3.13)$$

DIIS шукає такі коефіцієнти c_i , щоб похибка комбінованої матриці Фока була мінімальною.

SCF-процес умовно можна уявити як «рух по енергетичному ландшафту» до мінімуму. Без DIIS процедура робить кроки, які можуть «перестрибувати» через мінімум туди-сюди. DIIS аналізує кілька попередніх кроків і робить «розумну» екстраполяцію, яка швидше веде до мінімуму.

Типова схема роботи:

1. **Ітерації 1–2:** звичайний SCF без DIIS (потрібно накопичити історію)
2. **Ітерація 3 і далі:** DIIS активується автоматично, зберігає 6–8 останніх матриць Фока та похибок
3. На кожній ітерації будується оптимальна комбінація, яка прискорює збіжність

4. Процес продовжується до досягнення заданої точності (наприклад, $\Delta E < 10^{-6}$ Hartree)

Детальний розбір DIIS та що робити, коли він не працює, наведено в розділі 3.5.

3.1.5. Початкове наближення для матриці густини $\mathbf{D}^{(0)}$.

Початкове наближення матриці густини $\mathbf{D}^{(0)}$ є ключовим для швидкості та стабільності збіжності SCF-процедури. Розглянемо чотири методи від найпростішого до складнішого: Zero, S-based, Hcore та SAD (Superposition of Atomic Densities). Кожен метод генерує $\mathbf{D}^{(0)}$ із умовою $\text{Tr}(\mathbf{D}^{(0)}\mathbf{S}) \approx N_e$.

1. Zero guess (нульове наближення). Найпростіший метод: матриця густини ініціалізується як нульова.

$$\mathbf{D}^{(0)} = \mathbf{0}.$$

- **Математика:** $\mathbf{F}^{(0)} = \mathbf{h}$ (одноелектронний гамільтоніан), перша ітерація SCF еквівалентна Hcore. Для He у STO-3G ($K = 1$): $\mathbf{D}^{(0)} = [\mathbf{0}]$, $\text{Tr}(\mathbf{D}^{(0)}\mathbf{S}) = 0$.
- **Переваги:** Абсолютна простота.
- **Недоліки:** Не враховує електронну структуру, дуже повільна збіжність (особливо для молекул).
- **Застосування:** Рідко використовується, хіба що для тестування.

2. S-based guess (на основі матриці перекриття) Використовує обернену матрицю перекриття $\mathbf{S}^{-1}$ для розподілу електронів.

$$\mathbf{D}^{(0)} = \frac{N_e}{\text{Tr}(\mathbf{S}^{-1})} \mathbf{S}^{-1}.$$

- **Математика:** $\mathbf{S}_{\mu\nu} = \int \chi_\mu \chi_\nu d\mathbf{r}$. Нормалізація забезпечує $\text{Tr}(\mathbf{D}^{(0)}\mathbf{S}) = N_e$. Для He у STO-3G: $\mathbf{S} = [1]$, $\mathbf{S}^{-1} = [1]$, $\text{Tr}(\mathbf{S}^{-1}) = 1$, тож $\mathbf{D}^{(0)} = \frac{2}{1} \cdot [1] = [2]$.
- **Переваги:** Просте обчислення, враховує перекриття базисних функцій.
- **Недоліки:** Не враховує хімічну природу чи зв'язки, менш точне для молекул.
- **Застосування:** Некритичні випадки або для малих базисів.

3. Hcore guess (діагоналізація одноелектронного гамільтоніана) Базується на розв'язку одноелектронного гамільтоніана $\mathbf{h} = \mathbf{T} + \mathbf{V}^{\text{nuc}}$.

$$\mathbf{h}\mathbf{C}^{(0)} = \mathbf{S}\mathbf{C}^{(0)}\boldsymbol{\varepsilon}^{(0)}, \quad \mathbf{D}^{(0)} = 2 \sum_{i=1}^{N_{\text{occ}}} \mathbf{C}_i^{(0)}(\mathbf{C}_i^{(0)})^T,$$

де $N_{\text{occ}} = N_e/2$.

- **Математика:** Для He у STO-3G ($K = 1$): $\mathbf{h} = [h_{11}]$, $\mathbf{S} = [1]$, $\mathbf{C}^{(0)} = [1]$, $\mathbf{D}^{(0)} = 2 \cdot [1] \cdot [1]^T = [2]$, $\text{Tr}(\mathbf{D}^{(0)}\mathbf{S}) = 2$.
- **Переваги:** Фізично обґрунтований (електрони в полі ядер), швидка реалізація.
- **Недоліки:** Ігнорує двоелектронну взаємодію, може бути повільним для молекул.
- **Застосування:** Стандартний метод у багатьох програмах.

4. SAD (Superposition of Atomic Densities) Сума атомних густин, обчислених для ізольованих атомів у цільовому базисі.

$$\mathbf{D}^{(0)} = \sum_A \mathbf{D}^A, \quad \mathbf{D}^A = 2 \sum_{i=1}^{N_e^A/2} \mathbf{C}_i^A (\mathbf{C}_i^A)^T,$$

де $\mathbf{C}_i^A$ — з HF-розрахунку для атома A .

- **Математика:** Для He у STO-3G: $\mathbf{D}^{\text{He}} = [2]$ ($1s^2$), тож $\mathbf{D}^{(0)} = [2]$, $\text{Tr}(\mathbf{D}^{(0)}\mathbf{S}) = 2$. Для більших базисів (наприклад, cc-pVDZ) враховує лише $1s$.
- **Переваги:** Враховує атомну природу, ефективний для великих молекул.
- **Недоліки:** Ігнорує молекулярні зв'язки, може бути неточним для ковалентних систем.
- **Застосування:** Поширений для складних молекул.

5. MINAO (Minimal Atomic Orbital basis) Проекція атомних густин із мінімального базису (STO-3G) на цільовий.

$$\mathbf{D}^{(0)} = \sum_A \mathbf{P}^A \mathbf{D}_{\min}^A (\mathbf{P}^A)^T, \quad \mathbf{P}^A = \mathbf{S}_{\text{mol},\min}^A (\mathbf{S}_{\min}^A)^{-1}.$$

- **Математика:** Для He у STO-3G: $\mathbf{D}_{\min}^{\text{He}} = [2]$, $\mathbf{P}^{\text{He}} = [1]$, $\mathbf{D}^{(0)} = [2]$, $\text{Tr}(\mathbf{D}^{(0)}\mathbf{S}) = 2$. Для більших базисів (cc-pVDZ): $\mathbf{D}^{(0)}$ розподіляє густину по $1s/2s/2p$.
- **Переваги:** Враховує атомну природу, стабільний для великих базисів.
- **Недоліки:** Потребує попереднього обчислення $\mathbf{D}_{\min}^A$.
- **Застосування:** Складні молекули, великі базиси.

У PySCF вибір початкового наближення здійснюється за допомогою опції `init_guess='minao'`:

```
mf = scf.RHF(mol).run(init_guess='minao')
```

У PySCF реалізовано кілька варіантів початкового наближення, вони перелічені в документації [initial-guess](#).

3.1.6. Які бувають базиси

Для практичного розв'язання рівнянь Хартрі-Фока необхідно задати *базисний набір* — скінченну систему функцій, у просторі яких розкладаються молекулярні орбіталі. Якість і обчислювальна складність усієї задачі визначається саме вибором цього базису.

Атомні орбіталі типу STO. Природним вибором були б орбіталі, подібні до аналітичних розв'язків атома водню:

$$\chi_{\text{STO}}(r, \theta, \varphi) = N r^{n-1} e^{-\zeta r} Y_{\ell}^m(\theta, \varphi),$$

які називають **Slater-type orbitals (STO)**. Вони добре відтворюють поведінку електронної густини поблизу ядра ($r \rightarrow 0$) та правильний експоненційний спад при $r \rightarrow \infty$. Однак головна проблема — обчислення багатьох інтегралів для STO немає аналітичного виразу, що робить їх чисельно громіздкими.

Гаусові базисні функції (GTO). З цієї причини в більшості сучасних квантово-хімічних програм використовуються **Gaussian-type orbitals (GTO)**:

$$\chi_{\text{GTO}}(r, \theta, \varphi) = N r^{2n-2} e^{-\alpha r^2} Y_{\ell}^m(\theta, \varphi).$$

Попри те, що GTO гірше описують поведінку густини біля ядра, вони мають надзвичайно важливу властивість: добуток двох гаусових функцій з різними центрами знову є гаусом. Це дозволяє обчислювати всі одно- та двоелектронні інтеграли аналітично — ключова перевага для комп'ютерних розрахунків.

Зв'язок STO і GTO. Щоб поєднати фізичну точність STO і обчислювальну зручність GTO, кожен Slater-орбіталь апроксимують як лінійну комбінацію кількох GTO:

$$\chi_{\text{STO}} \approx \sum_{k=1}^{N_g} c_k \chi_{\text{GTO},k}.$$

Класичний приклад — базис **STO-3G**, у якому кожна STO замінюється трьома підібраними гаусами (звідси «3G»). Такі «мінімальні» базиси відтворюють лише заповнені атомні орбіталі та забезпечують найпростіше представлення молекули.

Для наочності розглянемо розклад 1s-орбіталі типу STO через три гаусові функції (**STO-3G**):

$$\chi_{1s}^{\text{STO-3G}}(r) = \sum_{i=1}^3 c_i N_i e^{-\alpha_i r^2},$$

де N_i — нормувальні константи, а коефіцієнти c_i і експоненти α_i підбираються так, щоб максимально наблизити форму до справжньої експоненти $e^{-\zeta r}$.

Для 1s-орбіталі (стандартна параметризація STO-3G):

i	1	2	3
c_i	0.444635	0.535328	0.154329
α_i	0.109818	0.405771	2.22766

Отже,

$$\chi_{1s}^{\text{STO-3G}}(r) = 0.444635 e^{-0.109818r^2} + 0.535328 e^{-0.405771r^2} + 0.154329 e^{-2.22766r^2}.$$

Ця комбінація дуже точно відтворює експоненційний спад Slater-орбіталі, але зберігає всі переваги аналітичного інтегрування для гаусових функцій.

Причини розширення базисів. Мінімальні базиси, як-от STO-3G, часто є надто обмеженими. Це означає, що вони містять лише стільки функцій, скільки потрібно для опису заповнених атомних орбіталей у ізольованих атомах, без можливості гнучко змінювати форму орбіталей у молекулі. Унаслідок цього такі базиси не враховують деформацію орбіталей під час утворення хімічного зв'язку, зміни електронної густини при поляризації, а також розмиття густини в слабко зв'язаних або збуджених станах.

Для точнішого опису електронної густини та поляризації орбіталей використовують розширені набори:

- **Double-zeta (DZ), triple-zeta (TZ)** — по дві чи три функції на кожен орбіталь;
- **Поляризаційні функції** (наприклад, d або f на легких елементах);
- **Дифузні функції** (з малими α) для опису розмитих електронних хмар.

Приклади таких базисів: 6-31G, 6-311G**, cc-pVDZ, aug-cc-pVTZ тощо.

Таким чином, вибір базисного набору — це компроміс між обчислювальними витратами та точністю відтворення хвильової функції.

3.1.7. Варіанти методу Хартрі-Фока

У попередній секції було розглянуто загальну форму рівнянь Хартрі-Фока. Однак конкретна реалізація залежить від *спінового стану системи* та того, чи є електронні оболонки заповненими або частково зайнятими. Залежно від цього використовуються різні варіанти HF-методу (табл. 3.1).

Restricted Hartree-Fock (RHF)

Цей варіант застосовується до систем із *замкненими оболонками*, де всі орбіталі заповнені парами електронів з протилежними спінами:

$$\psi_i^\alpha = \psi_i^\beta = \varphi_i.$$

Таблиця 3.1. Основні варіанти методу Хартрі-Фока

Метод	Спінова структура	Відкриті оболонки	Коментар
RHF	$\psi_i^\alpha = \psi_i^\beta$	Ні	Закриті оболонки (синглети)
UHF	$\psi_i^\alpha \neq \psi_i^\beta$	Так	Радикали, порушення $\langle \hat{S}^2 \rangle$
ROHF	частково спільні орбіталі	Так	Виправлення до UHF для правильного спіну
GHF	ψ_i — спінові спінори	Так	Повністю узагальнена форма, рідко використовується

Кожна просторова орбіталь φ_i містить два електрони (один зі спіном α , інший β). Тому матриця густини має вигляд

$$D_{\mu\nu} = 2 \sum_{i=1}^{N_{\text{occ}}} C_{\mu i} C_{\nu i},$$

а матриця Фока спрощується:

$$F_{\mu\nu} = h_{\mu\nu} + \sum_{\lambda\sigma} D_{\lambda\sigma} \left[(\mu\nu|\lambda\sigma) - \frac{1}{2}(\mu\lambda|\nu\sigma) \right].$$

RHF підходить для синглетних систем (He, Ne, H₂ у синглетному стані). У PySCF — це стандартна реалізація класу `scf.RHF`.

```
from pyscf import gto, scf
mol = gto.M(atom='H 0 0 0; H 0 0 1.4', basis='sto-3g')
mf = scf.RHF(mol).run()
print('E(RHF) =', mf.e_tot)
```

Unrestricted Hartree–Fock (UHF)

UHF дозволяє різні просторові орбіталі для електронів зі спінами α та β :

$$\psi_i^\alpha \neq \psi_i^\beta.$$

Таким чином, будується дві окремі матриці густини:

$$D_{\mu\nu}^\alpha = \sum_i^{N_\alpha} C_{\mu i}^\alpha C_{\nu i}^\alpha, \quad D_{\mu\nu}^\beta = \sum_i^{N_\beta} C_{\mu i}^\beta C_{\nu i}^\beta,$$

і відповідні матриці Фока:

$$F_{\mu\nu}^{\alpha} = h_{\mu\nu} + \sum_{\lambda\sigma} \left[(D_{\lambda\sigma}^{\alpha} + D_{\lambda\sigma}^{\beta})(\mu\nu|\lambda\sigma) - D_{\lambda\sigma}^{\alpha}(\mu\lambda|\nu\sigma) \right],$$

$$F_{\mu\nu}^{\beta} = h_{\mu\nu} + \sum_{\lambda\sigma} \left[(D_{\lambda\sigma}^{\alpha} + D_{\lambda\sigma}^{\beta})(\mu\nu|\lambda\sigma) - D_{\lambda\sigma}^{\beta}(\mu\lambda|\nu\sigma) \right].$$

UHF добре описує системи з непарною кількістю електронів (H, O, NO, радикали). Недолік: **порушення чистоти спіну**, тобто

$$\langle \hat{S}^2 \rangle \neq S(S+1),$$

через те, що хвильова функція не є власним станом оператора $\hat{S}^2$. У PySCF:

`scf.UHF(mol)`.

```
mf = scf.UHF(mol).run()
print('E(UHF) =', mf.e_tot)
print('<S^2> =', mf.spin_square()[0])
```

Restricted Open-Shell Hartree-Fock (ROHF)

ROHF — компроміс між RHF і UHF, який зберігає однакові орбіталі для парних електронів, але допускає різні для неспарених. Для замкнених орбіталей:

$$\psi_i^{\alpha} = \psi_i^{\beta} = \varphi_i,$$

для відкритих — лише α -електрони. Це забезпечує точне збереження $\langle \hat{S}^2 \rangle = S(S+1)$ при врахуванні відкритої конфігурації.

ROHF використовується для систем із *мультиплетними* станами (триплети, дублети), наприклад O₂, CH₂, NO. У PySCF реалізація через `scf.ROHF`.

```
mf = scf.ROHF(mol).run()
print('E(ROHF) =', mf.e_tot)
```

Generalized Hartree-Fock (GHF)

У загальному випадку хвильова функція записується через спин-спінори:

$$\psi_i(\mathbf{r}) = \begin{pmatrix} \varphi_i^{\alpha}(\mathbf{r}) \\ \varphi_i^{\beta}(\mathbf{r}) \end{pmatrix},$$

що дозволяє змішування спінових компонент у кожній орбіталі. GHF — найзагальніша форма, що не зберігає проєкцію спіну, але дає найгнучкіший опис у сильно корельованих системах (ферромагнетики, спин-кросовери). У PySCF — клас `scf.GHF(mol)`.

```
mf = scf.GHF(mol).run()
```

Порівняння методів

- RHF — найстабільніший, але обмежений замкненими оболонками.
- UHF — простий і точний для відкритих систем, але може порушувати чистоту спіну.
- ROHF — фізично коректний для мультиплетів, але складніший у реалізації.
- GHF — найзагальніший, проте використовується лише в спеціальних задачах.

3.2. Метод UHF та спінове забруднення

У квантово-хімічних розрахунках вибір типу наближення Гартрі-Фока залежить від спінового стану системи. Методи RHF, UHF та ROHF відрізняються тим, як вони поведуться із спіновими функціями електронів — тобто, чи допускають різні орбіталі для α - і β -електронів.

В методі UHF (Unrestricted Hartree-Fock) — α - і β -електронів займають різні просторові орбіталі. Цей метод придатний для відкритих оболонок (наприклад, атомів з неспареними електронами), однак може страждати від *спінового забруднення* (*spin contamination*), також UHF зазвичай дає трохи нижчу енергію, ніж RHF/ROHF, оскільки має більше варіаційної свободи. Різниця $\Delta E = E_{\text{UHF}} - E_{\text{ROHF}}$ незначна (менше кількох міліГартрі), однак у системах з сильним спіновим змішуванням може бути суттєвою.

Орбітальні енергії

UHF формує два набори орбіталей — для α - і β -електронів. Тому орбітальні енергії можуть відрізнитись:

$$\varepsilon_i^{(\alpha)} \neq \varepsilon_i^{(\beta)}.$$

У ROHF усі парні орбіталі спільні, тож орбітальні енергії чітко впорядковані відповідно до симетрії та спіну.

3.2.1. Спінове забруднення

У системах з відкритими оболонками метод UHF часто використовується як простіше наближення, що дозволяє займати незалежні орбіталі електронам зі спінами α та β . Однак відсутність суворого обмеження на спінову симетрію призводить до важливого недоліку — **спінового забруднення**.

Математична суть явища

UHF-хвильова функція не є власним станом оператора $\hat{S}^2$, хоч і залишається власним станом $\hat{S}_z$:

$$\hat{S}_z \Psi_{\text{UHF}} = M_S \Psi_{\text{UHF}}, \quad \hat{S}^2 \Psi_{\text{UHF}} \neq S(S+1) \Psi_{\text{UHF}}.$$

Отже, очікуване значення

$$\langle S^2 \rangle = \langle \Psi_{\text{UHF}} | \hat{S}^2 | \Psi_{\text{UHF}} \rangle$$

зазвичай перевищує істинне $S(S+1)$, тобто:

$$\Delta S^2 = \langle S^2 \rangle - S(S+1) > 0.$$

Фізичне тлумачення

UHF-хвильова функція може бути подана як суперпозиція чистих спінових станів:

$$\Psi_{\text{UHF}} = c_0 \Psi_S + c_1 \Psi_{S+1} + c_2 \Psi_{S+2} + \dots,$$

де присутність коефіцієнтів $c_i \neq 0$ для $i > 0$ означає змішування різних мультиплетів. Наприклад, для триплетного стану ($S = 1$) теоретичне значення $\langle S^2 \rangle = 2.0$, але якщо розрахунок UHF дає 2.25, це означає, що хвильова функція містить домішку п'ятичленного ($S = 2$) стану.

Вплив на результати

- **Енергія.** UHF може давати занадто низьку енергію через змішування спінів і штучне зменшення кулонівського відштовхування.
- **Електронна густина.** Спінова густина стає несиметричною, особливо поблизу атомів з неспареними електронами.

Як обчислюється спінове забруднення

Програми квантово-хімічних розрахунків (зокрема PySCF, Gaussian, ORCA) обчислюють спінове забруднення не безпосередньо через оператор $\hat{S}^2$, а через перекриття між орбіталями спінів α і β .

Основна формула для середнього значення $\langle S^2 \rangle$ у наближенні UHF має вигляд:

$$\langle S^2 \rangle = \frac{(N_\alpha - N_\beta)^2}{4} + \frac{N_\alpha + N_\beta}{2} - \sum_{i=1}^{N_\alpha} \sum_{j=1}^{N_\beta} |\langle \varphi_i^\alpha | \varphi_j^\beta \rangle|^2$$

де:

- N_α, N_β — кількість електронів зі спінами α та β відповідно;
- $\varphi_i^\alpha, \varphi_j^\beta$ — молекулярні(атомні) орбіталі для спінів α і β ;

- $\langle \varphi_i^\alpha | \varphi_j^\beta \rangle$ — перекриття між орбіталями різних спінів.

Якщо орбіталі α і β однакові, тобто $\varphi_i^\alpha = \varphi_i^\beta$, то:

$$\langle S^2 \rangle = \frac{(N_\alpha - N_\beta)^2}{4},$$

і для синглету ($N_\alpha = N_\beta$) це дорівнює нулю — спінове забруднення відсутнє.

Інтерпретація

Якщо орбіталі α і β подібні, перекриття $S_{\alpha\beta}$ велике, тому Σ наближається до N_β , і $\langle S^2 \rangle$ близьке до теоретичного $S(S+1)$ — тобто забруднення мінімальне. Якщо ж орбіталі сильно відрізняються (наприклад, при розриві зв'язку або в радикалах), Σ зменшується, і $\langle S^2 \rangle$ зростає — з'являється суттєве спінове забруднення.

Діагностика та критерії

Значення $\langle S^2 \rangle$ зазвичай друкується у вихідних даних програми. Практичні межі:

$$\Delta S^2 = \langle S^2 \rangle - S(S+1),$$

$$\begin{cases} \Delta S^2 < 0.05 & \text{— незначне забруднення, прийнятно;} \\ 0.05 < \Delta S^2 < 0.10 & \text{— помірне, бажано перевірити;} \\ \Delta S^2 > 0.10 & \text{— суттєве, слід перейти до ROHF або проєкційних методів.} \end{cases}$$

Приклад чисельного розрахунку

Для системи з $N_\alpha = 5$, $N_\beta = 4$ (очікувано $S = \frac{1}{2}$, $\langle S^2 \rangle = 0.75$):

- якщо $\Sigma = 3.9$, тоді

$$\langle S^2 \rangle = \frac{1}{4} + \frac{9}{2} - 3.9 = 0.85,$$

$$\Delta S^2 = 0.85 - 0.75 = 0.10,$$

тобто забруднення перебуває на межі суттєвого;

- якщо $\Sigma = 3.4$, тоді

$$\langle S^2 \rangle = \frac{1}{4} + \frac{9}{2} - 3.4 = 1.35,$$

$$\Delta S^2 = 1.35 - 0.75 = 0.60,$$

що свідчить про сильне спінове забруднення.

3.3. Прості атомні системи

Прості атомні системи — це найзручніша відправна точка для вивчення методів квантової хімії та самозгодженої процедури поля (SCF). На прикладі атомів Гідрогену та Гелію можна продемонструвати основні ідеї методу Гартрі-Фока, вплив базисного набору, збіжність енергії та базові аспекти орбітального аналізу.

3.3.1. Атом Гідрогену

Атом Гідрогену є найпростішим квантовим об'єктом: він складається лише з одного електрона, що рухається в кулонівському полі ядра. Його гамільтоніан має вигляд

$$\hat{H} = -\frac{1}{2}\nabla^2 - \frac{1}{r},$$

де перший доданок описує кінетичну енергію електрона, а другий — потенціал притягання до ядра. Ця система має аналітичне розв'язання, і енергія основного стану відома точно:

$$E_1 = -\frac{1}{2} \text{ Ha.}$$

Метод Гартрі-Фока для одноелектронних систем не містить апроксимацій — відсутнє електрон-електронне відштовхування, отже HF-енергія є точною для будь-якого заданого базису. Вся похибка зумовлена лише обмеженістю базисного набору.

H_SCF_calculation.py

```
# =====
# H_SCF_basis_analysis.py
# Енергетичний аналіз атома Гідрогену методом UHF
# для різних базисних наборів у PySCF.
#
# Для кожного базису обчислюються:
#   - Кінетична енергія (T)
#   - Потенціал ядро-електрон (V_nuc)
#   - Взаємодія електрон-електрон (V_ee = 0)
#   - Повна енергія (E_tot)
#   - Віральне співвідношення (V/T)
# =====

from pyscf import gto, scf
import numpy as np

def analyze_basis(basis_name):
    """Розрахунок і вивід таблиці для заданого базису"""
    # 1. Побудова молекули
    mol = gto.M(
        atom="H 0 0 0",
        basis=basis_name,
        spin=1,
        verbose=0
    )
```

```

# 2. Розрахунок методом UHF
mf = scf.UHF(mol)
E_tot = mf.kernel()

# 3. Отримання матриць інтегралів і густини
dm_a, dm_b = mf.make_rdm1()
dm = dm_a + dm_b
T = mol.intor("int1e_kin")
Vn = mol.intor("int1e_nuc")

# 4. Енергетичні компоненти
E_kin = np.einsum("ij,ji->", T, dm)
E_nuc = np.einsum("ij,ji->", Vn, dm)
E_ee = 0.0
V_total = E_nuc + E_ee
virial_ratio = V_total / E_kin

# 5. Форматований вивід
print("=" * 60)
print(f"      ЕНЕРГЕТИЧНИЙ АНАЛІЗ АТОМА H – БАЗИС: {basis_name}")
print("=" * 60)
print(f"{'Компонента':<35}{'Значення (Ha)':>20}")
print("-" * 60)
print(f"{'Кінетична енергія (T)':<35}{E_kin:>20.6f}")
print(f"{'Ядро-електрони (V_nuc)':<35}{E_nuc:>20.6f}")
print(f"{'Електрон-електрон (V_ee)':<35}{E_ee:>20.6f}")
print("-" * 60)
print(f"{'Повна енергія (E_tot)':<35}{E_tot:>20.6f}")
print("=" * 60)
print(f"{'Віральне співвідношення':<40}{'V/T'}")
print("-" * 60)
print(f"{'Розраховане значення':<35}{virial_ratio:>20.3f}")
print(f"{'Теоретичне значення':<35}{-2.000:>20.3f}")
print("=" * 60)
print()

# -----
# Основна частина
# -----
if __name__ == "__main__":
    basis_list = ["sto-3g", "3-21g", "6-31g", "cc-pVDZ"]
    for b in basis_list:
        analyze_basis(b)

```

Щоб побачити, як збільшується точність при розширенні базису, порівняймо кілька популярних наборів — від ST0-3G до cc-pV5Z. Код знаходиться в файлі `ch_atom_basis_convergence.py`, а графік, який виводить код подано на (рис. 3.2).

З графіка (рис. 3.2) видно, що при переході до великих кореляційно-узгоджених базисів (наприклад, cc-pVQZ, cc-pV5Z) енергія швидко збігається до повного базисного ліміту (CBS limit).

Орбіталі та матриця густини

Попри те, що в атома Гідрогену є лише одна заповнена орбіталь (1s), програма PySCF дозволяє аналізувати всі орбіталі базису, енергетичний

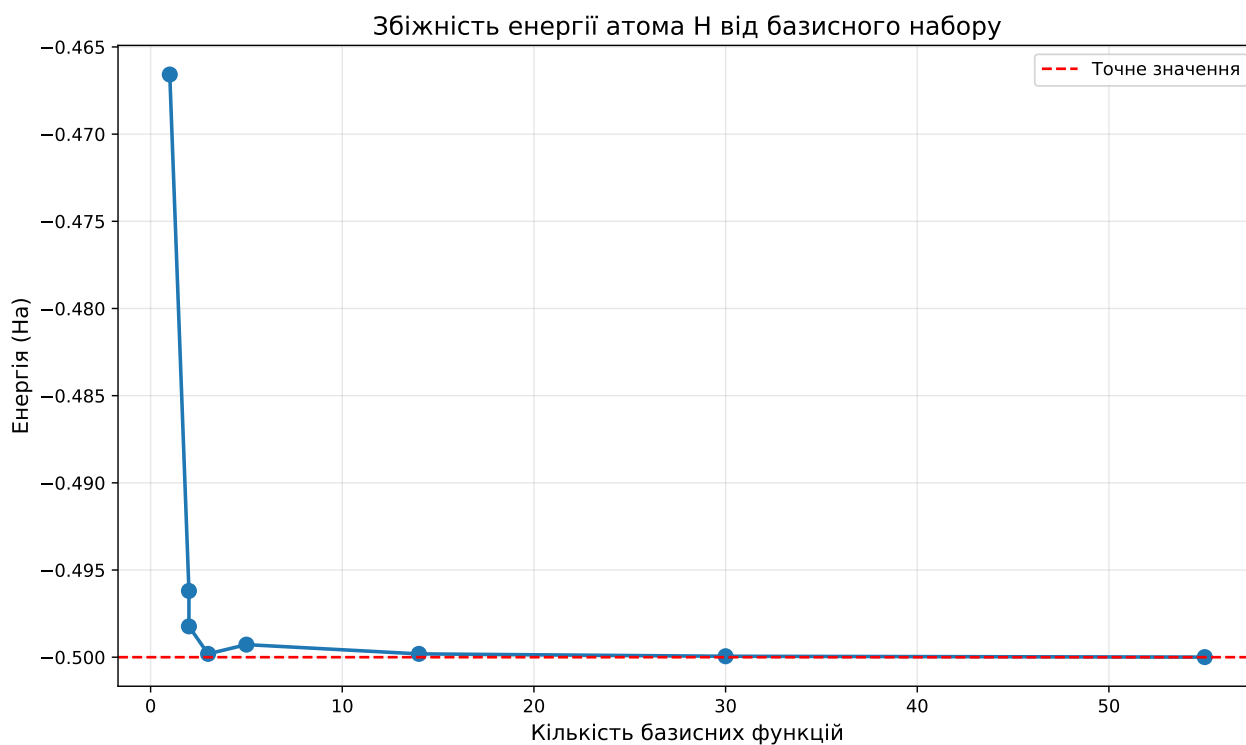


Рис. 3.2. Залежність енергії атома Н від вибору базису.
спектр і матрицю густини.

uhf_h_atom.py

```
# =====
# Файл: uhf_h_atom.py
# Автор: доктор Фейнман
# Опис: Розрахунок енергії атома Гідрогену методом UHF
#       з базисом cc-pVTZ у пакеті PySCF.
# =====

from pyscf import gto, scf
import numpy as np

mol = gto.M(atom="H 0 0 0", basis="cc-pvtz", spin=1)
mf = scf.UHF(mol)
energy = mf.kernel()

# Орбітальні енергії
print("\nОрбітальні енергії (альфа-спін):")
print("-" * 50)
for i, (e, label) in enumerate(zip(mf.mo_energy[0], mol.ao_labels())):
    occ = "(occ)" if i < mol.nelec[0] else "(virt)"
    print(f"{i + 1:2d}. {label:20s}: {e:10.6f} Ha {occ}")

print(f"\nЕнергія 1s орбіталі: {mf.mo_energy[0][0]:.8f} Ha")
print(f"Теоретична енергія: -0.50000000 Ha")

# Матриця густини
dm = mf.make_rdm1()
print(f"\nМатриця густини (альфа): {dm[0].shape}")
print(f"Слід dm: {np.trace(dm[0]):.1f} (має дорівнювати 1)")
```

Енергія 1s орбіталі: -0.49980981 Ha

Теоретична енергія: -0.50000000 Ha
Слід $dm(\alpha)$: 0.5

Слід матриці густини для альфа-спіну дорівнює 0.5, оскільки PySCF нормує густину на повну систему ($\alpha + \beta$). Для одноелектронного випадку:

$$\text{Tr}(dm_\alpha) = 0.5, \quad \text{Tr}(dm_\beta) = 0, \quad \text{Tr}(dm_\alpha + dm_\beta) = 1.$$

Отриманий результат ідеально відтворює точний розв'язок, ілюструючи коректність SCF-процедури для одноелектронних систем.

3.3.2. Атом Гелію

Атом гелію — найпростіша багатоелектронна система і перший приклад, де проявляється **електрон–електронна взаємодія**. Обидва електрони заповнюють орбіталь $1s$ з протилежними спінами, утворюючи замкнену оболонку (синглет, $S = 0$).

HeHF.py

```
from pyscf import gto, scf

# --- 1. Опис атома гелію ---
mol = gto.Mole()
mol.atom = 'He 0 0 0'
mol.basis = 'cc-pVTZ'
mol.build()

# --- 2. Розрахунок RHF ---
mf = scf.RHF(mol)
E_tot = mf.kernel()

# --- 3. Матриця густини та інтеграли ---
dm = mf.make_rdm1()
T_int = mol.intor('int1e_kin') # оператор кінетичної енергії
V_nuc_int = mol.intor('int1e_nuc') # оператор потенціальної енергії ядро-електрон
vhf = mf.get_veff(mol, dm) # ефективний потенціал (Кулон + обмін)

# --- 4. Енергетичні складові ---
E_kin = (dm * T_int).sum() # <T>
E_nuc = (dm * V_nuc_int).sum() # <V_nuc>
E_ee = 0.5 * (dm * vhf).sum() # <V_ee> (Кулон + обмін)
E_tot_check = E_kin + E_nuc + E_ee # перевірка

# --- 5. Віральне співвідношення ---
V_total = E_nuc + E_ee
virial_ratio = V_total / E_kin

# --- 6. Вивід у вигляді таблиці ---
print("\n" + "="*60)
print(" "15 + "ЕНЕРГЕТИЧНИЙ АНАЛІЗ АТОМА He")
print("="*60)
print(f"{'Компонента':<30} {'Значення (Ha)':>20}")
print("-"*60)
print(f"{'Кінетична енергія (T)':<30} {E_kin:>20.6f}")
print(f"{'Ядро-електрони (V_nuc)':<30} {E_nuc:>20.6f}")
print(f"{'Електрон-електрон (V_ee)':<30} {E_ee:>20.6f}")
print("-"*60)
```

```
print(f"{'Повна енергія (E_tot)':<30} {E_tot:>20.6f}")
print(f"{'Перевірка суми (E_sum)':<30} {E_tot_check:>20.6f}")
print("="*60)
print(f"\n{'Віральне співвідношення':<30} {'V/T':>20.2f}")
print("="*60)
print(f"{'Розраховане значення':<30} {virial_ratio:>20.3f}")
print(f"{'Теоретичне значення':<30} {-2.000:>20.3f}")
print("="*60 + "\n")
```

Після збіжності SCF-розрахунку можна отримати компоненти енергії з одно- та двоелектронних інтегралів і перевірити віральне співвідношення:

$$\frac{\langle V \rangle}{\langle T \rangle} \approx -2.00.$$

Відхилення від цього значення свідчить про неточність базису або проблеми збіжності.

Порівняння RHF та UHF

Оскільки гелій має рівну кількість електронів обох спінів ($N_\alpha = N_\beta$), обидва варіанти методу — RHF і UHF — дають однакові результати.

rhf_vs_uhf_he.py

```
# =====
# File: rhf_vs_uhf_he.py
# Purpose: Порівняння RHF та UHF енергій для атома Гелію (PySCF)
# =====

from pyscf import gto, scf

# --- Побудова молекули ---
mol = gto.M(atom="He 0 0 0", basis="6-31g", spin=0)

# --- RHF розрахунок ---
mf_rhf = scf.RHF(mol)
mf_rhf.verbose = 0
e_rhf = mf_rhf.kernel()

# --- UHF розрахунок ---
mf_uhf = scf.UHF(mol)
mf_uhf.verbose = 0
e_uhf = mf_uhf.kernel()

# --- Вивід результатів ---
print(f"RHF енергія: {e_rhf:.10f} Ha")
print(f"UHF енергія: {e_uhf:.10f} Ha")
print(f"Різниця: {abs(e_rhf - e_uhf):.2e} Ha")

# --- Перевірка симетрії спіну ---
s2_uhf = mf_uhf.spin_square()
print(f"\n<S^2> (UHF): {s2_uhf[0]:.6f}")
print("<S^2> (точно): 0.000000")
```

Коментар

Для відкрито-оболонкових систем метод UHF може порушувати спінову симетрію (*spin contamination*). У випадку гелію цього не відбувається: $S^2 = 0$, тому RHF $\equiv$ UHF.

3.3.3. Збуджені стани в межах методу Хартрі-Фока

Метод Хартрі-Фока у своїй стандартній формі призначений для опису **основного стану** системи — стану з найменшою можливою енергією. Під час самозгодженої процедури (SCF) електрони заповнюють орбіталі починаючи з найнижчих енергій, і програма автоматично знаходить конфігурацію, що мінімізує повну енергію:

$$E_0 = \min_{\Phi} \langle \Phi | \hat{H} | \Phi \rangle,$$

де Φ — це одно-детермінантна хвильова функція (детермінант Слейтера). Отже, звичайний HF-розрахунок завжди повертає *основний стан*.

Для опису **збуджених станів** потрібно враховувати мультиплетність системи та іноді примусово фіксувати електронну конфігурацію:

Триплетні стани. Для атома гелію збуджений триплетний стан $1s2s^3S$ можна отримати простим встановленням параметра `spin = 2`, що відповідає двом неспареним електронам. Оскільки триплетна конфігурація вимагає, щоб електрони мали паралельні спіни і знаходилися на різних орбіталях, SCF-процедура автоматично знаходить конфігурацію $1s2s$ без додаткових втручань.

Синглетні збуджені стани. Для збудженого синглету $1s2s^1S$ ситуація складніша. Параметр `spin = 0` означає лише, що кількість альфа- та бета-електронів однакова, але не визначає, *які саме орбіталі* вони займають. Тому звичайна SCF-процедура знову поверне основний стан $1s^2$, оскільки він енергетично вигідніший.

Щоб отримати збуджений синглет, необхідно **зафіксувати електронну конфігурацію** і не дозволити системі сповзти до основного стану. Для цього використовується **метод MOM (Maximum Overlap Method)**, який на кожній ітерації SCF обирає орбіталі з найбільшим перекриттям з орбіталями попередньої ітерації:

$$O_{ij} = \left| \langle \varphi_i^{(n)} | \varphi_j^{(n-1)} \rangle \right|^2.$$

Завдяки цьому MOM утримує бажану конфігурацію (наприклад, $1s2s$) і не дозволяє хвильовій функції сповзати до основного стану ($1s^2$).

Отриманий результат називають **самозгодженим збудженим детермінантом** (self-consistent excited determinant). Такий детермінант *не є справжнім власним станом гамільтоніана $\hat{H}$* , оскільки він не задовольняє рівняння Шредінгера точно:

$$\hat{H}\Phi \neq E\Phi.$$

Метод Хартрі-Фока обмежує хвильову функцію лише одним детермінантом, тоді як справжній власний стан гамільтоніана — це, взагалі кажучи, комбінація багатьох детермінантів (багатоконфігураційна хвильова функція). Тому **енергії збуджених станів, як і основного, систематично завищені** порівняно з експериментом.

Проте такі стани мають практичну цінність. Їх енергії та орбіталі дозволяють:

- побачити, **де локалізований збуджений електрон** — наприклад, у гелію при переході $1s \rightarrow 2s$ один електрон залишається поблизу ядра, а інший зміщується далі, змінюючи розподіл електронної густини $\rho(\mathbf{r})$;
- оцінити **енергетичні інтервали між станами**

$$\Delta E = E_{\text{збудж}} - E_{\text{основ}},$$

які відповідають спостережуваним спектральним лініям у дослідах поглинання чи випромінювання.

Приклад реалізації МОМ-розрахунку для атома гелію наведено у файлі

`he_excited_states_mom.py`.

3.3.4. Висновки

- Атом Гідрогену — ідеальний тест для демонстрації роботи SCF-процедури в найпростішому випадку.
- Для одноелектронних систем HF є точним методом; похибка визначається лише базисом.
- Атом Гелію — перша система, де з'являється електронна кореляція; HF дає наближення, але не точне значення енергії.
- Розширення базису покращує збіжність до повного базисного ліміту (CBS limit).
- Аналіз орбіталей і густини в PySCF дає інтуїцію про структуру розв'язку рівнянь Хартрі-Фока.

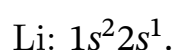
Таким чином, розгляд атомів H та He закладає основу для розуміння ітераційної природи SCF-процедури, ролі базисних функцій і меж застосовності методу Хартрі-Фока для багатоелектронних систем.

3.4. Атоми другого періоду (Li–Ne)

3.4.1. Літій (Li, Z=3)

Атом Літію — перший багатоелектронний атом, у якому проявляється неспарений електрон та спінова поляризація ($N_\alpha \neq N_\beta$, тобто густини ρ_α і ρ_β відрізняються).

Електронна конфігурація:



Два перші електрони утворюють замкнену оболонку (1s), а третій — одинокий у підоболонці 2s, що зумовлює мультиплетність $2S$ (спін $S = \frac{1}{2}$).

Через наявність неспареного електрона метод UHF є природним вибором. UHF дозволяє альфа- та бета-орбіталям відрізнятися, тобто хвильова функція не змушена мати однакову просторову форму для різних спінів.

Альтернативою є метод ROHF (Restricted Open-Shell Hartree-Fock), у якому

- усі замкнені орбіталі мають однакові просторові функції для α та β спінів;
- відкриті орбіталі (неспарені) можуть мати різну зайнятість, але однакову форму.

ROHF забезпечує правильне значення $\langle S^2 \rangle$ та зберігає спінову симетрію, проте формулювання рівнянь Фока є складнішим. UHF, навпаки, простіший у реалізації, але може призводити до *spin contamination*. У практиці обидва методи дають близькі енергії для атома Li, проте ROHF вважається формально більш коректним.

li_uhf_rohf_comparison.py

```
# =====
# Файл: li_uhf_rohf_comparison.py
# Автор: доктор Фейнман
# Опис: Порівняння результатів UHF і ROHF для атома літію (Li).
# Демонструє вплив спінового забруднення та різницю у варіаційній гнучкості методів.
# Розрахунок проводиться у базисі cc-pVDZ з використанням PySCF.
# =====

from pyscf import gto, scf

# --- 1. Визначення молекули ---
mol = gto.M(
    atom="Li 0 0 0",
    basis="cc-pvdz",
    spin=1,          # неспарений електрон (S=1/2)
    symmetry=True,
)

print("Літій (Li):")
print(" Конфігурація: [He] 2s1")
print(" Основний стан: 2S")
print(f" Електронів: {mol.nelectron}")
print()

# --- 2. UHF ---
uhf = scf.UHF(mol)
E_uhf = uhf.kernel()

print("=== UHF ===")
print(f"Енергія Li (UHF): {E_uhf:.8f} Ha")
print(f"Енергія Li (UHF): {E_uhf * 27.211386:.4f} eV")

s2_uhf = uhf.spin_square()
print(f"<S2> = {s2_uhf[0]:.6f} (очікується 0.75)\n")

# --- 3. ROHF ---
rohf = scf.ROHF(mol)
```

```

E_rohf = rohf.kernel()

print("=== ROHF ===")
print(f"Енергія Li (ROHF): {E_rohf:.8f} Ha")
print(f"Енергія Li (ROHF): {E_rohf * 27.211386:.4f} eV")

s2_rohf = rohf.spin_square()
print(f"<S²> = {s2_rohf[0]:.6f} (очікується 0.75)\n")

# --- 4. Порівняння ---
ΔE = (E_uhf - E_rohf) * 27.211386
print(f"Різниця (UHF - ROHF): {ΔE:.6f} eV")

# --- 5. Коментар ---
# UHF дозволяє різні орбіталі для α і β спінів,
# тому виникає спінове забруднення (<S²> > 0.75).
# ROHF зберігає правильну спінову симетрію, але менш гнучкий варіаційно.

```

Коментар

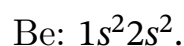
- Значення $\langle S^2 \rangle \approx 0.75$ свідчить про правильний спіновий стан подвійності (мультиплетність $2S + 1 = 2$).
- Наявність різниці між енергіями альфа- та бета-орбіталей демонструє спінову асиметрію.
- Метод UHF може частково порушувати симетрію (так звана *spin contamination*), але для Li це незначно.

Енергія, отримана методом HF, для Li становить близько -7.432 Ha у базисі *сс-pVDZ*, тоді як експериментальна енергія основного стану (повна) — близько -7.478 Ha. Таким чином, кореляційна похибка становить близько 0.046 Ha (приблизно 1.25 eV).

Цей приклад є першим, де проявляється *кореляція електронів*, яку HF не враховує. В подальших розділах буде показано, як методи MP2, CI та CC враховують ці ефекти.

3.4.2. Берилій (Be, Z=4)

Атом Берилію має електронну конфігурацію



Тут усі орбіталі парні, тому спінова поляризація відсутня, і зручно використовувати RHF (Restricted Hartree–Fock). Розрахунок атома наведено у файлі `c/be_rhf_energy.py`. Обидва електрони у підоболонці $2s$ мають протилежні спіни, тож система має мультиплетність $1S$ (синглетний стан).

Обговорення.

- Для Be HF-енергія виходить приблизно -14.573 Ha (у базисі *сс-pVTZ*), тоді як експериментальна — -14.667 Ha.
- Різниця близько 0.094 Ha (≈ 2.6 eV) — це **кореляційна енергія**, тобто внесок взаємодії електронів, не врахований у HF.

- У Берилія ефекти електронної кореляції вже досить значні, адже електрони в орбіталі $2s$ взаємодіють один з одним.
- Цей випадок є гарною ілюстрацією межі застосування методу Гартрі-Фока.

Висновки для атомів Li і Be.

- Li демонструє появу неспареного електрона і спінової поляризації (UHF необхідний).
- Be — перший приклад, де **електронна кореляція** дає помітну похибку енергії.
- PySCF дозволяє наочно дослідити вплив базису, спіну, симетрії та методів на точність енергії.

3.4.3. Бор-Неон: систематичне дослідження

Після розгляду окремих атомів Літію та Берилію доцільно перейти до систематичного аналізу всіх атомів другого періоду (від Бору до Неону). У цих атомах поступово заповнюється $2p$ -підрівень, і змінюється як спінова мультиплетність, так і форма електронної густини. Метод Гартрі-Фока дозволяє побачити, як змінюється енергія системи при збільшенні числа електронів, а також як проявляється спінова поляризація у відкритих оболонках.

Електронні конфігурації.

Li:	[He] $2s^1$	2S
Be:	[He] $2s^2$	1S
B:	[He] $2s^2 2p^1$	2P
C:	[He] $2s^2 2p^2$	3P
N:	[He] $2s^2 2p^3$	4S
O:	[He] $2s^2 2p^4$	3P
F:	[He] $2s^2 2p^5$	2P
Ne:	[He] $2s^2 2p^6$	1S

Як видно, кількість неспарених електронів збільшується від Бору до Нітрогену, а потім зменшується до Неону, що зумовлює зміну спінового стану та мультиплетності.

В файлі `c_second_period_hf.py` проведено розрахунок енергій Гартрі-Фока для атомів другого періоду в однаковому базисі **cc-pVDZ**, оцінено правильність спінового стану (через $\langle S^2 \rangle$) та проаналізовано систематичні тенденції.

Коментар

- Для кожного атома автоматично обирається тип SCF: RHF (для замкнених оболонок, $S = 0$) або UHF (для відкритих).
- Параметр `spin` у PySCF означає різницю між кількістю альфа- та бета-електронів: $\text{spin} = N_\alpha - N_\beta = 2S$.
- Розрахунок $\langle S^2 \rangle$ дозволяє перевірити правильність спінового стану. Для ідеального випадку значення має збігатися з теоретичним $S(S + 1)$.
- У файлі `second_period_hf.npz` зберігаються всі енергії для подальшого аналізу або побудови графіків.

Очікувані тенденції.

1. Повна енергія атома зменшується (стає більш негативною) із зростанням атомного номера Z .
2. Енергія спостерігає стрибки на межі заповнення оболонок: при переходах $\text{Be} \rightarrow \text{B}$, $\text{N} \rightarrow \text{O}$, $\text{F} \rightarrow \text{Ne}$.
3. Спінова мультиплетність відображає заповнення $2p$ -орбіталей згідно з **правилом Гунда** — максимальний спін у середині підоболонки (для N).
4. Значення $\langle S^2 \rangle$ для UHF повинні бути близькі до теоретичних, але можуть мати невеликі відхилення через *spin contamination*.

Фізичне узагальнення. Цей розрахунок демонструє фундаментальну властивість методу Гартрі-Фока: він добре описує загальну структуру енергетичних рівнів і тенденції в періодичній таблиці, але не враховує електронну кореляцію, через що абсолютні значення енергії мають систематичну похибку.

3.4.4. Заселеності орбіталей і принцип заповнення

Розв'язання рівнянь Хартрі-Фока дає повний набір одноелектронних орбіталей $\{\psi_i\}$, які утворюють ортонормований базис у просторі станів електронів. Однак не всі з них беруть участь у формуванні хвильової функції основного стану (детермінанта Слейтера (3.1)). Електрони займають лише найнижчі за енергією орбіталі — відповідно до принципу Паулі та правила Ауфбау (принципу послідовного заповнення). Такі орбіталі називаються **заселеними** (*occupied orbitals*), тоді як енергетично вищі рівні залишаються порожніми і утворюють підпростір *віртуальних орбіталей* (*virtual orbitals*). Формально повний набір орбіталей можна подати як об'єднання двох неперетинних множин:

$$\{\psi_i\} = \underbrace{\{\psi_1, \psi_2, \dots, \psi_{n_{\text{occ}}}\}}_{\text{заселені}} \cup \underbrace{\{\psi_{n_{\text{occ}}+1}, \dots, \psi_{n_{\text{bas}}}\}}_{\text{віртуальні}}.$$

Лише заселені орбіталі беруть участь у побудові детермінанта Слейтера, тоді як віртуальні використовуються для опису збуджених станів та врахування електронної кореляції в пост-Хартрі-Фоківських методах.

У випадку замкнених оболонок (RHF) кожна орбіталь до певного рівня заповнена двома електронами. Найвища з них називається **НОМО** (*Highest Occupied Molecular Orbital*) — найвища зайнята молекулярна орбіталь, а найближча вільна над нею — **LUMO** (*Lowest Unoccupied Molecular Orbital*) — найнижча незайнята орбіталь. Ці дві орбіталі визначають енергетичну «межу» між заселеними та віртуальними станами і відіграють ключову роль у переходах збудження, фотонному поглинанні та хімічній реакційній здатності.

Формально заселеності визначаються як:

$$n_i = \begin{cases} 2, & \varepsilon_i \leq \varepsilon_{\text{НОМО}}, \\ 0, & \varepsilon_i > \varepsilon_{\text{НОМО}}. \end{cases}$$

Така схема відповідає принципу Паулі та відображає послідовне заповнення орбіталей від нижчих до вищих енергій.

Для відкритих оболонок або при використанні необмеженого Гартрі-Фока (UHF) заселеності можуть відрізнятися для α - і β -спінів:

$$n_i^{(\alpha)} \neq n_i^{(\beta)}.$$

У цьому випадку електронна густина розділяється на дві компоненти:

$$\rho(\mathbf{r}) = \rho_{\alpha}(\mathbf{r}) + \rho_{\beta}(\mathbf{r}),$$

що дозволяє описати спінову поляризацію та магнітні моменти атомів.

Заселеності безпосередньо визначають енергетичний внесок кожної орбіталі:

$$E_{\text{orb}} = \sum_i n_i \varepsilon_i,$$

і тому є ключовими для аналізу хімічного зв'язку, реакційної здатності та переходів між електронними станами молекули.

Програмна реалізація. У пакеті PySCF заселеності можна отримати з об'єкта SCF:

```
from pyscf import gto, scf

# Створюємо молекулу літію з правильним спіном
mol = gto.M(atom="Li 0 0 0", basis="sto-3g", spin=1)

# RHF розрахунок
mf = scf.RHF(mol).run()
```

```
mo_occ = mf.mo_occ # масив заселеностей [2, 1, 0, 0, 0]
print(mo_occ)

# Для RHF mo_coeff – це ОДИН масив (не кортеж!)
mo_coeff = mf.mo_coeff
print(mo_coeff)
```

Вивід `mo_occ` показує кількість електронів у кожній орбіталі (0, 1 або 2). Це дозволяє побудувати таблицю заселеностей або енергетичну діаграму з позначенням заповнених рівнів.

Приклади аналізу орбіталей. У файлі `c li_orbital_analysis.py` продемонстровано, як у PySCF можна отримати узагальнену таблицю заселеностей орбіталей після RHF-розрахунку для атома Li. Виводяться енергії, спін, тип орбіталі (закрита, відкрита або віртуальна) та коротка статистика електронної конфігурації:

```
=====
ВІЗУАЛІЗАЦІЯ ЗАСЕЛЕНОСТІ ОРБІТАЛЕЙ (RHF)
=====
```

Орбіталь	Спини	Заселеність	Енергія (Ha)	Тип	Позначка
1	↑↓	2.0	-2.353491	закрита оболонка	
2	↑	1.0	-0.038960	відкрита оболонка	← HOMO
3	-	0.0	0.160521	віртуальна	← LUMO
4	-	0.0	0.160521	віртуальна	
5	-	0.0	0.160521	віртуальна	

```
=====

Статистика:
Подвійно заповнені (закриті): 1
Одинично заповнені (відкриті): 1
Віртуальні: 3
Всього електронів: 3
Спінова мультиплетність: 2 (дублет)

HOMO (орбіталь 2): -0.038960 Ha
LUMO (орбіталь 3): 0.160521 Ha
HOMO-LUMO gap: 0.199481 Ha (5.4282 eV)
=====
```

Такий формат дозволяє швидко оцінити структуру заповнення, мультиплетність і відсутність спінової контамінації.

У програмі `c print_orbitals.py` наведено докладніший аналіз: для кожної молекулярної орбіталі подаються її енергія, заселеність та коефіцієнти розкладу за атомними базисними функціями (Li 1s, Li 2s, Li 2p). Формат виводу подібний до таблиць молекулярних орбіталей у ORCA та інших квантово-хімічних пакетах і зручний для візуального аналізу орбітальної будови.

3.4.5. Енергії іонізації

Іонізаційна енергія I є однією з фундаментальних характеристик атома, що визначає мінімальні енерговитрати на видалення одного електрона із нейтрального атома в основному стані:

$$I = E(A^+) - E(A),$$

де $E(A)$ — повна енергія нейтрального атома, а $E(A^+)$ — енергія його однозарядного катіона. Ця величина безпосередньо вимірюється експериментально (у електронвольтах) і є важливим тестом якості будь-якого квантово-хімічного методу.

У межах методу Гартрі-Фока іонізаційна енергія може бути визначена двома способами.

Метод Δ SCF. Найбільш пряме обчислення полягає у незалежному знаходженні повних енергій нейтрального атома і катіона:

$$I_{\Delta\text{SCF}} = E(A^+) - E(A).$$

Такий підхід враховує релаксацію орбіталей після видалення електрона, тому дає точніші результати, ніж прості наближення.

Теорема Купманса. У межах наближення «заморожених орбіталей» (*frozen orbital approximation*) повна енергія системи вважається незмінною при видаленні одного електрона. У цьому випадку зміна енергії дорівнює орбітальній енергії найвищої зайнятої молекулярної орбіталі:

$$I_{\text{Koopmans}} \approx -\varepsilon_{\text{HOMO}}.$$

Це твердження відоме як **теорема Купманса**. Воно дозволяє оцінити енергії іонізації без повторного самозгодження, хоча і не враховує релаксаційні та кореляційні ефекти.

Порівняння підходів. В файлі [с KoopmansTheoremCheck.py](https://github.com/cheminformatics/KoopmansTheoremCheck.py) виконана перевірка теореми Купманса для атомів другого періоду. Для легких атомів (Li, Be, B, C, N, O, F, Ne) метод Купманса правильно відтворює тенденцію зростання енергії іонізації уздовж періоду, але систематично недооцінює абсолютні значення. Метод Δ SCF наближається до експерименту краще, оскільки включає релаксацію орбіталей.

Приклад: атом Літію. Для Li (конфігурація $1s^2 2s^1$):

$$I_{\text{Koopmans}} = -\varepsilon_{2s}, \quad I_{\Delta\text{SCF}} = E(\text{Li}^+) - E(\text{Li}), \quad I_{\text{exp}} = 5.39 \text{ eV}.$$

Енергія Купманса дещо завищена, а результат Δ SCF ближчий до експериментального, що демонструє вплив орбітальної релаксації.

Коментар

Розрахунок виводить таблицю порівняння енергій іонізації ($-\epsilon_{\text{HOMO}}$, ΔSCF , експеримент) для атомів другого періоду та відносну похибку у відсотках:

$$\delta = 100 \times \frac{I_{\text{calc}} - I_{\text{exp}}}{I_{\text{exp}}}.$$

Аналіз цих відхилень дозволяє оцінити якість базисного набору і межі застосовності теореми Купманса.

3.4.6. Електронна густина

Розподіл електронної густини $\rho(\mathbf{r})$ є однією з ключових величин у квантовій хімії. Саме ця функція визначає, де саме в просторі «перебувають» електрони атома або молекули, і, отже, пояснює більшість хімічних і спектроскопічних властивостей системи.

Згідно з однодетермінантним наближенням Гартрі-Фока, електронна густина визначається як

$$\rho(\mathbf{r}) = \sum_i^{\text{occ}} |\psi_i(\mathbf{r})|^2,$$

де $\psi_i(\mathbf{r})$ — орбіталі, а сума береться по всіх зайнятих орбіталях.

У рамках однодетермінантного наближення Хартрі-Фока електронна густина $\rho(\mathbf{r})$ може бути виражена через матрицю густини $D_{\mu\nu}$ та базисні функції $\chi_\mu(\mathbf{r})$:

$$\rho(\mathbf{r}) = \sum_{\mu\nu} D_{\mu\nu} \chi_\mu(\mathbf{r}) \chi_\nu(\mathbf{r}) = \chi^T(\mathbf{r}) \mathbf{D} \chi(\mathbf{r}). \quad (3.14)$$

Цей вираз відображає розподіл імовірності знайти будь-який з N електронів у точці простору $\mathbf{r}$. Нормування густини забезпечує тотальне число електронів:

$$\int \rho(\mathbf{r}) d\mathbf{r} = N. \quad (3.15)$$

Розподіл електронної густини

У програмному пакеті PySCF густина може бути обчислена як на сітці, так і в окремих точках простору, що дозволяє візуалізувати просторовий розподіл електронів.

Обчислення густини вздовж осі z

У програмі `c_electron_density_profile.py` реалізовано функцію `plot_density_profile`, що обчислює електронну густина вздовж осі z для атома. Це дозволяє побудувати одноосний «профіль густини» та простежити, як електронна густина спадає при віддаленні від ядра.

Для кожної точки $\mathbf{z}$ обчислюється матриця базисних функцій $\varphi_i(\mathbf{r})$, і далі густина:

$$\rho(\mathbf{r}) = \sum_{ij} \chi_i(\mathbf{r}) D_{ij} \chi_j(\mathbf{r}).$$

Результат зображається графічно: густина $\rho(0, 0, z)$ як функція відстані від ядра. Для нейтральних атомів крива має різкий максимум біля ядра, який швидко спадає експоненційно.

Інтерпретація результатів. На побудованому графіку $\rho(0, 0, z)$ спостерігаються чіткі області локалізації електронів: внутрішні електрони формують центральний пік, тоді як зовнішні оболонки проявляються у вигляді плавних плечей. Для важчих атомів густина стає більш «зосередженою» біля ядра через сильніше притягання кулонівським потенціалом.

Практична порада. При виборі базисного набору (cc-pvdz, 6-31G**, тощо) слід мати на увазі, що форма профілю густини істотно залежить від якості базису: мінімальні базиси дають лише грубу картину, тоді як розширені базиси (cc-pVTZ) відтворюють експоненційний спад більш точно.

Сферично усереднена густина

Для атомів, що мають сферичну симетрію, зручно розглядати усереднену по всіх напрямках густину:

$$\rho(r) = \frac{1}{4\pi} \int \rho(\mathbf{r}) d\Omega,$$

де $r = |\mathbf{r}|$ та $d\Omega$ — елемент тілесного кута.

Ця функція показує, як змінюється густина лише від радіуса r , незалежно від напрямку, і є основою для побудови радіальної функції розподілу

$$P(r) = 4\pi r^2 \rho(r),$$

що описує, яка частка електронів перебуває у сферичному шарі товщини dr на відстані r від ядра. В файлі `c_spherical_electron_density.py` реалізовано візуалізація сферично-усередненої густини.

Методика обчислення. Для кожного радіуса r вибираються випадкові напрями (кутові координати θ, φ) за методом Монте-Карло. У цих напрямках обчислюється густина $\rho(x, y, z)$, після чого береться середнє значення:

$$\rho(r) \approx \frac{1}{N} \sum_{k=1}^N \rho(r, \theta_k, \varphi_k).$$

Цей підхід забезпечує добру точність навіть при невеликій кількості напрямів $N \sim 100$.

Радіальна функція розподілу. На другому графіку зображується $4\pi r^2 \rho(r)$ — це функція, інтеграл від якої по r дає повну кількість електронів:

$$\int_0^\infty 4\pi r^2 \rho(r) dr = N_e.$$

Таким чином, можна перевірити нормування хвильової функції та коректність чисельного інтегрування.

Приклад інтерпретації. Для атома гелію максимум $4\pi r^2 \rho(r)$ спостерігається приблизно при $r \approx 0.3 \text{ Bohr}$, що відповідає найбільш ймовірній відстані електрона від ядра у $1s$ -стані. Для вуглецю або неону виникають додаткові піки, пов'язані з електронами на $2s$ та $2p$ оболонках.

Перевірка нормування. Наприкінці виконання функції виводиться значення

$$\int 4\pi r^2 \rho(r) dr,$$

яке має збігатися з кількістю електронів у системі, що є корисним тестом правильності побудованої густини.

3.4.7. Порівняння електронних густин різних атомів

Ми розглянули, як отримати електронну густину $\rho(\mathbf{r})$ та сферично усереднену густину $\rho(r)$ для окремого атома. Однак для повного розуміння періодичних закономірностей важливо порівняти, як змінюється форма та протяжність електронної хмари при переході від легких до важчих елементів.

Фізична мотивація

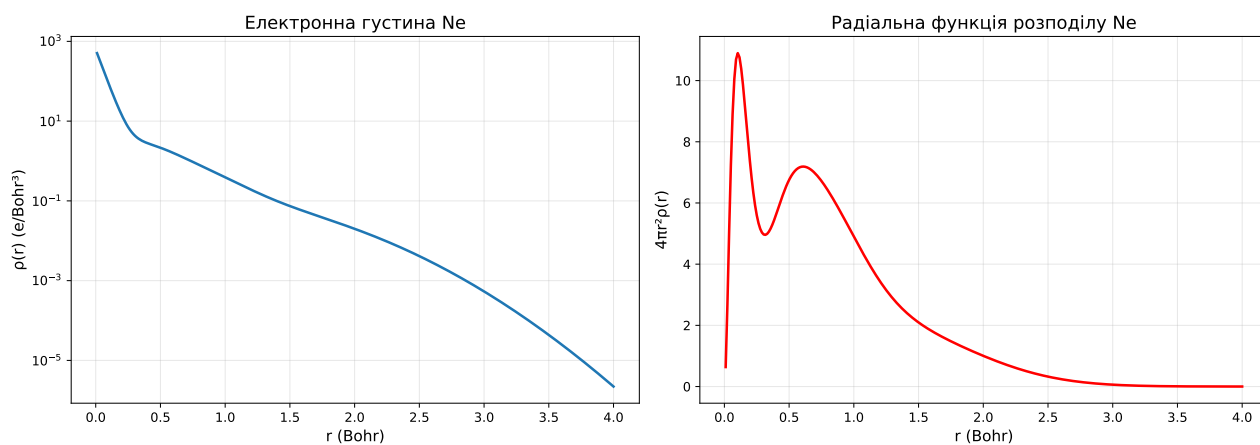
Порівняння електронних густин дозволяє:

- побачити, як із зростанням атомного номера Z ядро сильніше притягує електрони;
- проаналізувати зміну розмірів атома та ефективного радіуса;
- виявити закономірності заповнення електронних оболонок;
- візуально оцінити зв'язок між структурою густини та положенням елемента в періодичній системі.

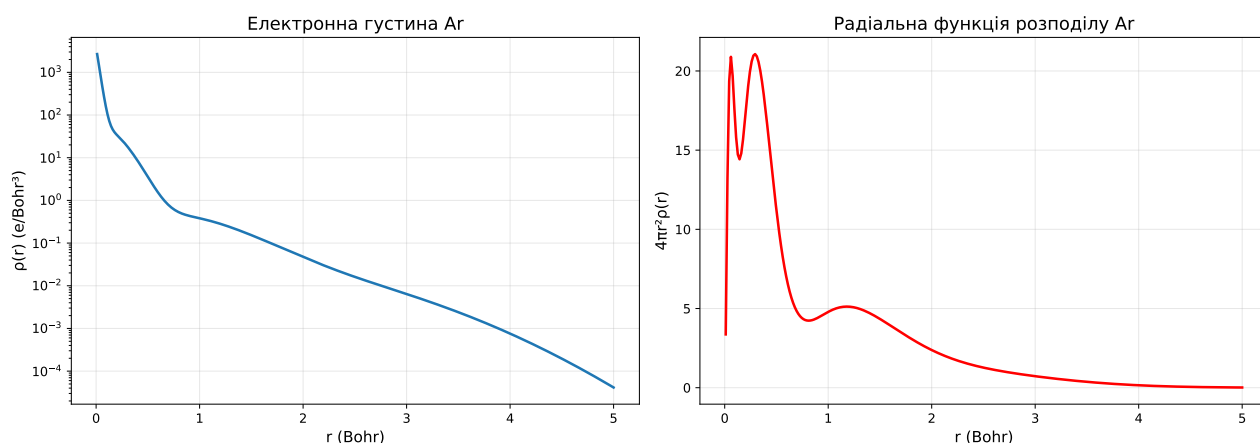
3.4.8. Зв'язок із електронними оболонками

Максимуми радіальної функції розподілу $4\pi r^2 \rho(r)$ відповідають областям найбільшої ймовірності перебування електронів певних орбіталей:

$$\begin{aligned} 1s &\rightarrow \text{перший максимум,} & 2s, 2p &\rightarrow \text{другий максимум,} \\ & & 3s, 3p, 3d &\rightarrow \text{третій тощо.} \end{aligned}$$



(а) АNe



(б) Атома Ar

Рис. 3.3. Електронна густина $\rho(r)$ (ліворуч) та радіальна функція розподілу $4\pi r^2 \rho(r)$ (праворуч) для атомів Ne та Ar. На графіках радіальна функція розподілу чітко видно окремі максимуми, що відповідають електронним оболонкам. Для неону (Ne) спостерігаються два основних максимуми, які відповідають заповненим підоболонкам $1s^2$ і $2s^2 2p^6$, тоді як для аргону (Ar) — три максимуми, що відображають заповнення оболонок $1s^2$, $2s^2 2p^6$ та $3s^2 3p^6$.

Таким чином, форма $4\pi r^2 \rho(r)$ дає прямий зв'язок між результатами квантово-хімічних розрахунків і традиційною атомною структурою, знайомою студентам з курсу атомної фізики. Візуальний аналіз таких графіків (рис. 3.4.7) дозволяє простежити закономірності побудови періодичної системи Менделєєва з точки зору електронної густини.

3.4.9. Практичні завдання

1. Побудуйте на одному графіку $\rho(r)$ для Li, Na, K. Як змінюється радіус атома?
2. Знайдіть положення максимумів $4\pi r^2 \rho(r)$ для атомів Ne, Ne, Ar. До яких оболонок вони належать?
3. Порівняйте радіальні функції для атомів C і O. Як змінюється густина зовнішньої оболонки?

3.4.10. Методичні рекомендації

1. Змінюючи базис (наприклад, ST0-3G, 6-31G, cc-pVTZ), можна дослідити, як збільшується точність відтворення радіального профілю.
2. Для відкритих оболонок (атомів з неспареними електронами) потрібно враховувати спінову поляризацію — використовується UHF.

Таким чином, наведені приклади демонструють практичний зв'язок між теоретичними поняттями електронної густини і їх чисельною реалізацією в пакеті PySCF.

3.5. Складні випадки та збіжність

Розрахунки у квантовій хімії не завжди проходять гладко. Навіть для простих систем ітераційна процедура самозгодженого поля (SCF) може не досягати стабільного рішення. Найчастіше проблеми збіжності виникають для перехідних металів, відкритих оболонок та систем із виродженими орбіталями. У цьому розділі розглянемо основні джерела труднощів та методи їх подолання.

3.5.1. Причини поганої збіжності

Типові джерела нестабільності SCF:

- сильна електронна кореляція у незаповнених *d*- і *f*-оболонках;
- мала різниця енергій між орбіталями (виродження чи квазивиродження);
- невдалий початковий вектор коефіцієнтів (погане стартове наближення);
- занадто жорсткі або занадто м'які критерії збіжності.

У таких випадках енергія може осцилювати або навіть зростати замість того, щоб збігатися. Для вирішення цих проблем існує кілька стратегій стабілізації.

3.5.2. Методи стабілізації збіжності

Коли стандартний SCF з DIIS-процедурою (див. 3.1.4) не працює, у PySCF доступні наступні інструменти:

Level shifting (зсув рівнів)

Метод штучно підвищує енергії віртуальних орбіталей на величину Δ :

$$\epsilon_{\text{virt}} \rightarrow \epsilon_{\text{virt}} + \Delta \quad (3.16)$$

Це запобігає коливанням заповнення орбіталей між ітераціями та стабілізує збіжність. Фізично це еквівалентно збільшенню енергетичного бар'єру для переходу електронів з заповнених на віртуальні орбіталі.

```
mf.level_shift = 0.5 # типові значення 0.2-1.0 Hartree
```

Альтернативні методи екстраполяції

EDIIS (Energy DIIS) мінімізує енергію замість похибки самоузгодженості. Це краще працює на початкових ітераціях, коли система далека від збіжності:

```
mf.DIIS = scf.EDIIS
```

ADIIS (Augmented DIIS) автоматично комбінує DIIS та EDIIS залежно від стану збіжності — використовує EDIIS на початку та переключається на DIIS ближче до мінімуму:

```
mf.DIIS = scf.ADIIS
```

Налаштування DIIS

Іноді корисно змінити параметри самого DIIS:

```
mf.diis_space = 4      # менший підпростір (4 замість 6-8)
mf.diis_start_cycle = 5 # пізніший старт DIIS
mf.diis = None         # повністю вимкнуті DIIS
```

Зменшення розміру DIIS-простору обмежує «пам'ять» алгоритму, що може допомогти у випадках, коли історія попередніх ітерацій вводить в оману.

Краще початкове наближення

Якість початкового наближення критично важлива для збіжності. PySCF пропонує кілька варіантів:

```
# Стандартні методи
mf = scf.UHF(mol)
mf.init_guess = 'minao' # мінімальний AO (за замовчуванням)
mf.init_guess = 'atom'  # атомні густини (краще для металів)
mf.init_guess = '1e'    # лише одноелектронний гамільтоніан

# Використання попереднього розрахунку
dm_init = mf_previous.make_rdm1()
mf.kernel(dm0=dm_init)
```

Варіант `'atom'` часто найкращий для перехідних металів та систем з відкритими оболонками.

Метод Ньютона–Рафсона

Для систем, які вже близькі до збіжності, але повільно сходяться, можна використати метод другого порядку:

```
mf = mf.newton() # перехід до Newton-Raphson
mf.kernel()
```

Це забезпечує квадратичну збіжність поблизу мінімуму, але вимагає додаткових обчислень гессіану.

3.5.3. Вироджені орбіталі та дробові заповнення

Якщо в системі наявні вироджені або майже вироджені орбіталі, SCF може «коливатись», не вибираючи між ними. Класичні приклади — атоми з частково заповненими *p*- або *d*-оболонками.

Для таких випадків ефективним є застосування *дробових заповнень орбіталей* (fractional occupations), що згладжують різницю між рівнями енергії.

Ідея полягає у введенні ефективного теплового розмазування Фермі–Дірака:

$$f_i = \frac{1}{1 + e^{(\varepsilon_i - \mu)/kT}},$$

де f_i — часткове заповнення орбіталі i , ε_i — її енергія, μ — хімічний потенціал, а kT — параметр розмазування (зазвичай ~ 0.1 – 0.3 Hartree).

Цей підхід дозволяє SCF уникнути нестійких перестрибувань між виродженими рівнями, імітуючи систему при скінченній температурі.

У PySCF для цього достатньо викликати:

```
from pyscf import scf
mf = scf.addons.frac_occ(mf)
```

Методика (`uhf_fractional_occupations.py`) добре працює для атомів (V, Cr), радикалів та малих кластерів перехідних металів.

3.5.4. Перехідні метали — приклад складної системи

Перехідні елементи (Fe, Co, Ni, Cr, Mn тощо) — класичні приклади систем із поганою збіжністю SCF. Вони поєднують декілька факторів складності одночасно.

```
from pyscf import gto, scf
mol = gto.M(atom="Ni 0 0 0", basis="sto-3g", spin=0)
```

```
mf = scf.UHF(mol).run()
print("Converged?", mf.converged) # часто False
```

Причини проблем:

- близькість енергетичних рівнів $3d$ і $4s$ -орбіталей;
- можливість декількох спінових станів, близьких за енергією;
- наявність кількох локальних мінімумів функціоналу енергії;
- сильна кореляція в незаповненій d -оболонці.

У PySCF це проявляється як:

- осциляції енергії між ітераціями;
- розбігання DIIS;
- стрибки значення $\langle S^2 \rangle$ між кроками (спін-забруднення).

В файлі `c Transition_Metals_UHF.py` наведено приклад універсальної функції для стабільного розрахунку атомів перехідних металів із урахуванням описаних вище прийомів:

- використовується UHF (для врахування спіну);
- застосовується зсув рівнів (`level_shift`);
- розширюється DIIS-простір (`diis_space`);
- використовується атомне початкове наближення (`init_guess='atom'`);
- у разі потреби вмикається уточнення Ньютона-Рафсона;
- контролюється спін-забруднення через $\langle S^2 \rangle$.

Такий комплексний підхід забезпечує стабільність навіть для важких атомів, де стандартний SCF часто не сходиться.

3.5.5. Загальна стратегія досягнення збіжності

Для надійного отримання фізично коректного рішення рекомендується послідовна стратегія — від простих прийомів до складніших:

1. Спробувати стандартний UHF/RHF;
2. Якщо не збігається — додати зсув рівнів: `level_shift=0.5`;
3. Змінити початкове наближення: `init_guess='atom'`;
4. Використати ADIIS: `mf.DIIS = scf.ADIIS`;
5. Збільшити DIIS-простір: `diis_space=10`;
6. Для вироджених систем — дробові заповнення: `frac_occ`;
7. Якщо близько до збіжності — метод Ньютона: `mf.newton()`;
8. Вимкнути симетрію: `symmetry=False` (як останній засіб).

В програмі `c uhf_convergence_strategies.py` продемонстровано різні стратегії покращення збіжності SCF на конкретних прикладах.

Практичні рекомендації:

- Завжди перевіряйте `mf.converged` після розрахунку;
- Для систем з відкритими оболонками контролюйте $\langle S^2 \rangle$ — велике спін-забруднення вказує на проблеми;

- Для великих систем на перших етапах можна зменшити точність:
`conv_tol=1e-5;`
- Якщо енергія осцилює — це сигнал використати level shifting або EDIIS;
- Не бійтесь експериментувати з комбінаціями методів — іноді неочевидна комбінація працює найкраще.

Таким чином, навіть якщо стандартна процедура SCF не збігається, послідовне застосування описаних прийомів практично гарантує стабільне та фізично обґрунтоване рішення. Ключ до успіху — розуміння природи проблеми та систематичний підхід до її вирішення.

Практичні завдання

Завдання 1: Систематичне дослідження

```
"""
ЗАВДАННЯ 1: Розрахуйте енергії всіх атомів першого періоду
(H, He) з різними базисними наборами. Побудуйте графік
збіжності енергії до базисної межі.
Базиси: sto-3g, 6-31g, cc-pvdz, cc-pvtz, cc-pvqz, cc-pv5z
"""

from pyscf import gto, scf
import matplotlib.pyplot as plt
# Ваш код тут
```

Завдання 2: Енергії іонізації

```
"""
ЗАВДАННЯ 2: Обчисліть першу та другу енергії іонізації
для атомів Li, Be, B, C, N, O, F, Ne. Порівняйте з
експериментальними значеннями.
 $IE_1 = E(A^+) - E(A)$ 
 $IE_2 = E(A^{2+}) - E(A^+)$ 
Використайте базис cc-pvtz
"""
# Ваш код тут
```

Завдання 3: Спектроскопічні константи

```
"""
ЗАВДАННЯ 3: Для атома Карбону розрахуйте енергетичну
різницю між основним триплетним станом ( $^3P$ ) та
збудженими станами ( $^1D$  та  $^1S$ ).
Підказка: використайте різні спінові конфігурації
"""
# Ваш код тут
```

Завдання 4: Залежність від базису

"""

ЗАВДАННЯ 4: Дослідіть, як додавання дифузних функцій
впливає на енергію та дипольну поляризованість атома
Кисню (O).

Порівняйте: cc-pvdz vs aug-cc-pvdz, cc-pvtz vs aug-cc-pvtz

"""

Ваш код тут

4

Метод Хартрі-Фока для простих молекул

Після вивчення атомних систем природним кроком є перехід до молекул. Найпростіші молекули — іон молекули водню H_2^+ та молекула водню H_2 — є ідеальними об'єктами для демонстрації можливостей та обмежень методу Хартрі-Фока.

4.1. Орбіталі і електронні стани молекул

Електронна структура двоатомних молекул описується двома рівнями:

1. класифікацією **молекулярних орбіталей (МО)**;
2. класифікацією **електронних станів молекули**, які утворюються внаслідок заповнення цих МО електронами.

4.1.1. Класифікація молекулярних орбіталей

Молекулярні орбіталі позначаються символами σ , π , δ , φ , ..., що відповідають проєкції орбітального моменту Λ на між'ядерну вісь:

$$\Lambda = 0, 1, 2, 3, \dots \Rightarrow \sigma, \pi, \delta, \varphi, \dots$$

Кожна орбіталь може бути:

- **зв'язувальною** або **антизв'язувальною** (позначається зірочкою *);
- **симетричною (gerade, g)** або **антисиметричною (ungerade, u)** відносно інверсії в центрі молекули.

Таким чином, типовий запис молекулярної орбіталі має вигляд:

$$\sigma_g, \quad \pi_u, \quad \sigma_u^*, \quad \pi_g^*, \dots$$

Зв'язок із симетрією: Парність g/u визначається поведінкою хвильової функції при інверсії координат:

$$\psi_g(\mathbf{r}) = +\psi_g(-\mathbf{r}), \quad \psi_u(\mathbf{r}) = -\psi_u(-\mathbf{r}).$$

4.1.2. Класифікація електронних станів молекули

Електронний терм — це позначення квантового стану молекули, який характеризується певними значеннями повного спіну S , проєкції орбітального моменту на між'ядерну вісь Λ , та симетріями відносно інверсії (g/u) і відбиття ($+/-$). Він має вигляд:

$$^{2S+1}\Lambda_{g/u}^{\pm}$$

де:

- S — повний спін системи;
- $2S + 1$ — **мультиплетність** (синглет, дублет, триплет тощо);
- Λ — **сумарна проєкція орбітального моменту** на між'ядерну вісь:

$$\Sigma (\Lambda = 0), \quad \Pi (\Lambda = 1), \quad \Delta (\Lambda = 2), \quad \Phi (\Lambda = 3), \dots$$

- верхній індекс $+$ або $-$ — **симетрія відносно площини, що містить вісь молекули** (лише для станів типу Σ);
- нижній індекс g та u (від *gerade* та *ungerade*) — **парність орбіталі** відносно інверсії в центрі молекули.

Типові терми двоатомних молекул

Терм	Мультиплетність	Симетрія	Фізичний зміст
$^1\Sigma_g^+$	Синглет	$g, +$	Основний стан молекули H_2 ; усі електрони спарені, повна симетрія
$^3\Sigma_u^+$	Триплет	$u, +$	Збуджений стан з паралельними спінами (наприклад, у O_2)
$^1\Pi_u$	Синглет	u	Орбітальний момент $\Lambda = 1$, без інверсійної симетрії
$^3\Pi_g$	Триплет	g	Орбітальний момент $\Lambda = 1$, паралельні спіни
$^1\Delta_g$	Синглет	g	Орбітальний момент $\Lambda = 2$, спарені електрони
$^2\Sigma_u^+$	Дублет	$u, +$	Один неспарений електрон, як у радикалах типу NO

Нижче наведено покроковий алгоритм визначення терму для заданої електронної конфігурації.

Алгоритм

1. **Записати електронну конфігурацію.** Визначити, які орбіталі заповнені, частково заповнені чи вакантні:

$$(\sigma_g)^2(\sigma_u^*)^2(\pi_u)^4(\pi_g^*)^2, \dots$$

2. **Визначити тип орбіталей і відповідні проєкції орбітального моменту.** Для кожної орбіталі встановити:

$$\sigma \rightarrow \Lambda_i = 0, \quad \pi \rightarrow \Lambda_i = 1, \quad \delta \rightarrow \Lambda_i = 2, \quad \text{тощо.}$$

3. **Знайти можливі комбінації орбітальних моментів.** Для активних (частково заповнених) орбіталей обчислити всі можливі значення:

$$\Lambda = |\Lambda_1 + \Lambda_2|, |\Lambda_1 - \Lambda_2|, \dots$$

Це дає можливі типи станів: $\Sigma, \Pi, \Delta, \dots$

4. **Комбінувати спіни електронів.** Для кожного електрона $S_i = \frac{1}{2}$. Обчислити всі можливі значення повного спіну:

$$S = |S_1 + S_2|, |S_1 - S_2|, \dots$$

Відповідно визначається мультиплетність $2S+1$ (синглет, дублет, триплет тощо).

5. **Перевірити принцип Паулі.** Загальна хвильова функція має бути антисиметричною при перестановці двох електронів:

$$\Psi_{\text{заг}} = \Psi_{\text{просторова}} \Psi_{\text{спінова}} \Psi_{\text{симетрії}}.$$

Якщо просторова частина симетрична $\Rightarrow S = 0$ (синглет), якщо антисиметрична $\Rightarrow S = 1$ (триплет).

6. **Визначити симетрії g/u і $+/-$.**

- Індекс g/u показує парність хвильової функції при інверсії в центрі молекули.
- Для станів Σ додається верхній індекс $+/-$, що вказує на симетрію при відбитті у площині, яка містить між'ядерну вісь.

7. **Записати всі можливі терми.** Комбінуючи знайдені $S, \Lambda, g/u$ та $+/-$, записуємо:

$$^{2S+1}\Lambda_{g/u}^{\pm}.$$

8. **Визначити основний терм за правилами Гунда.**

- 8.1. Найнижчу енергію має стан з **максимальним спіном S** .
- 8.2. Для однакового S — стан з **найбільшим Λ** .
- 8.3. Для даних S, Λ — визначається парність (g/u) і знак ($+/-$) з урахуванням симетрії конфігурації.

Приклад. Для молекули водню H_2 основний електронний стан має конфігурацію σ_g^2 . Оскільки обидва електрони спарені ($S = 0$), повний терм записується як:

$$2S+1\Lambda_g^+ = {}^1\Sigma_g^+.$$

Це **синглетний стан** ($S = 0$), симетричний відносно інверсії (індекс g) і з позитивною симетрією відносно площини, що містить між'ядерну вісь (+).

4.1.3. Від атомів до молекул: базисні функції та ЛКАО

Що ми вже знаємо з атомних розрахунків?

При розрахунку атомів ми використовували **базисні набори** — набори математичних функцій (зазвичай гаусіани), які описують форму електронної хмари навколо ядра. Наприклад, для атома Н у базисі STO-3G ми мали:

- **Базисні функції:** χ_μ (гаусіани, центровані на ядрі)
- **Орбіталі атома:** $\psi_i = \sum_\mu C_{\mu i} \chi_\mu$ (результат HF)
- **Енергії орбіталей:** ε_i (власні значення рівняння Фока)

Іншими словами, метод Хартрі-Фока знаходить коефіцієнти $C_{\mu i}$, які визначають, як базисні функції комбінуються у атомні орбіталі.

Що змінюється для молекул?

Для молекул ми використовуємо *той самий підхід*, але базисні функції тепер центровані на *різних ядрах*:

	Атом Н	Молекула H_2
Базисні функції	$\chi_1, \chi_2, \dots$ (центр: атом Н)	$\chi_A^{(1)}, \chi_A^{(2)}, \dots, \chi_B^{(1)}, \chi_B^{(2)}, \dots$ (центри: атом А і атом В)
Орбіталі	$\psi_i = \sum_\mu C_{\mu i} \chi_\mu$ (атомні орбіталі)	$\psi_i = \sum_\mu C_{\mu i} \chi_\mu$ (молекулярні орбіталі)
Метод	Hartree-Fock	Hartree-Fock

Ключовий момент: Математично — це одна й та сама процедура! Різниця лише в тому, що базисні функції розташовані на різних атомах.

Принцип ЛКАО

Оскільки базисні функції в молекулі природно групуються за атомами (функції на атомі А, функції на атомі В), молекулярну орбіталь можна записати як:

$$\psi_i^{(\text{MO})} = \sum_{\mu \in A} C_{\mu i} \chi_\mu^{(A)} + \sum_{\nu \in B} C_{\nu i} \chi_\nu^{(B)} + \dots \quad (4.1)$$

Це і є *принцип лінійної комбінації атомних орбіталей (ЛКАО)*: молекулярна орбіталь складається з внесків базисних функцій, центрованих на різних атомах.

Термінологічне уточнення

У контексті молекул термін «атомна орбіталь» (АО) часто означає *базисну функцію, центровану на атомі*, а не результат атомного HF-розрахунку.

Щоб уникнути плутанини, ми будемо використовувати:

- **Базисні функції χ_μ** — математичні функції (гаусіани)
- **Атомні орбіталі (АО)** — базисні функції у контексті ЛКАО
- **Молекулярні орбіталі (МО)** — результат HF для молекули

Далі ми розглянемо розрахунок молекулярного іону H_2^+ . У мінімальному базисі (по одній функції на атом) маємо:

- **Базис:** χ_A (1s на атомі А), χ_B (1s на атомі В)
- **Молекулярні орбіталі:** лінійні комбінації цих функцій

Метод Хартрі-Фока знаходить два розв'язки:

$$\psi_{\sigma_g} = \frac{1}{\sqrt{2(1+S)}}(\chi_A + \chi_B) \quad (\text{зв'язувальна}) \quad (4.2)$$

$$\psi_{\sigma_u^*} = \frac{1}{\sqrt{2(1-S)}}(\chi_A - \chi_B) \quad (\text{антизв'язувальна}) \quad (4.3)$$

де $S = \langle \chi_A | \chi_B \rangle$ — інтеграл перекриття базисних функцій.

- ψ_{σ_g} — **зв'язувальна МО**: обидві базисні функції додаються $\Rightarrow$ густина між ядрами зростає $\Rightarrow$ стабілізація
- $\psi_{\sigma_u^*}$ — **антизв'язувальна МО**: функції віднімаються $\Rightarrow$ густина між ядрами зменшується $\Rightarrow$ дестабілізація

Один електрон H_2^+ займає нижчу орбіталь σ_g , що призводить до утворення зв'язку. Верхня орбіталь σ_u^* залишається **віртуальною** (незайнятою).

Висновок:

1. Базисні функції — це математичні «цеглинки», з яких будуються орбіталі
2. Для атомів: всі функції на одному центрі $\Rightarrow$ атомні орбіталі
3. Для молекул: функції на різних центрах $\Rightarrow$ молекулярні орбіталі (ЛКАО)
4. Метод Хартрі-Фока — єдиний для обох випадків!

Таким чином, перехід від атомів до молекул не вимагає нових концепцій — лише розширення базису на кілька центрів.

4.2. Початкові модельні молекули: H_2^+ та H_2

Першим кроком у квантово-хімічному описі молекул є вивчення найпростіших систем, для яких можна чітко простежити природу хімічного зв'язку. Молекули H_2^+ та H_2 є **модельними** у тому сенсі, що вони:

- мають найменшу можливу кількість електронів (один і два відповідно);
- дозволяють побудувати хвильові функції, що безпосередньо демонструють утворення зв'язувальних і розривних (антизв'язувальних) орбіталей;
- відтворюють основні риси електронної структури складніших молекул, але без зайвих ускладнень багаточастинкової взаємодії;
- зручні для порівняння точних, наближених і експериментальних результатів;
- використовуються як тестові системи для перевірки методів квантової хімії.
- H_2^+ — найпростіша двоатомна молекула, що містить один електрон. Її можна розв'язати аналітично в еліптичних координатах, тому вона є класичним прикладом формування зв'язувальної орбіталі σ_g ;
- H_2 — найпростіша двоелектронна молекула, у якій вперше проявляється електронна кореляція та обмеження методу Хартрі-Фока.

Таким чином, системи H_2^+ і H_2 є **відправною точкою** для розуміння природи хімічного зв'язку, ролі симетрії, утворення молекулярних орбіталей і спінових станів.

4.3. Іон молекули водню H_2^+

4.3.1. Теоретичні основи

Іон H_2^+ — це найпростіша молекулярна система, яка складається з двох протонів та одного електрона. Незважаючи на її простоту, саме на прикладі H_2^+ вперше стає очевидною суть хімічного зв'язку як результату квантового перекривання хвильових функцій.

У нерелятивістському наближенні (та з використанням атомних одиниць) гамільтоніан системи має вигляд:

$$\hat{H} = -\frac{1}{2}\nabla^2 - \frac{1}{r_A} - \frac{1}{r_B} + \frac{1}{R},$$

де r_A , r_B — відстані електрона до ядер A і B , а R — між'ядерна відстань.

Перші три члени описують кінетичну енергію електрона та його притягання до двох ядер, а останній доданок — кулонівське відштовхування між ядрами.

Оскільки у системі лише один електрон, тут *відсутні електрон-електронні взаємодії*. Тому метод Хартрі-Фока не містить ніяких додаткових апроксимацій (крім обмежень базису), і його результат збігається з точною розв'язною енергією при нескінченному базисі. Таким чином, H_2^+ — це ідеальний тест для перевірки коректності реалізації ХФ-методу в програмних пакетах.

4.3.2. Визначення молекули в PySCF

Для визначення молекули в PySCF використовується рядкова нотація:

`h2plus_single.py`

```

"""
Розрахунок іона молекули водню H2+ при фіксованій відстані
та аналіз молекулярних орбіталей
"""

import numpy as np
import matplotlib.pyplot as plt

from pyscf import gto, scf

# -----
# Визначення молекули
# -----
mol = gto.Mole()
mol.atom = """
    H  0.0  0.0  0.0
    H  0.0  0.0  0.74
"""
mol.unit = "Angstrom"
mol.charge = 1      # заряд +1 (один електрон)
mol.spin = 1        # 2S = 1 (дублет)
mol.basis = "sto-3g"
mol.build()

# -----
# UHF-розрахунок
# -----
mf = scf.UHF(mol)
energy = mf.kernel()

# -----
# Вивід результатів
# -----
print("\n" + "=" * 60)
print("Іон молекули водню H2+")
print("=" * 60)
print(f"Міжядерна відстань R = 0.74 Å = {0.74 / 0.529177:.3f} bohr")
print(f"Базисний набір: {mol.basis}")
print(f"Повна енергія: {energy:.6f} Ha")
print(f"Кількість електронів: α={mol.nelec[0]}, β={mol.nelec[1]}")

# -----
# Орбітальні енергії
# -----
eps = mf.mo_energy[0] # α-спінові енергії
homo, lumo = eps[0], eps[1]
gap = lumo - homo

print("\nОрбітальні енергії (Ha):")
print(f"  HOMO (зв'язуюча σ): {homo:.4f}")
print(f"  LUMO (антизв'язуюча σ*): {lumo:.4f}")
print(f"  HOMO-LUMO gap = {gap:.4f} Ha = {gap * 27.211:.2f} eV")

# -----
# Спіновий стан
# -----
s2, mult = mf.spin_square()
print(f"\n<S^2> = {s2:.4f} (очікується 0.75 для S=1/2)")
print(f"Мультиплетність: 2S+1 = {mult}")

```

```
print("=" * 60)
```

Пояснення коду:

- `atom = 'H 0 0 0; H 0 0 0.74'` — координати атомів (Ангстремми за замовчуванням)
- `basis = 'sto-3g'` — мінімальний базис
- `charge = 1` — заряд системи (+1 для H_2^+)
- `spin = 1` — $2S = N_\alpha - N_\beta = 1$
- `scf.UHF` — необмежений ХФ (один непарний електрон)

4.3.3. Інтерпретація результатів

Вивід програми містить основні характеристики розрахованої системи H_2^+ при між'ядерній відстані

$$R = 0.74 \text{ \AA} = 1.399 a_0,$$

у базисному наборі STO-3G.

- **Повна енергія:**

$$E = -0.538205 \text{ Ha},$$

що є нижчою за енергію вільного атома водню (-0.5 Ha). Це свідчить про стабілізацію системи внаслідок утворення молекулярного зв'язку — електрон делокалізується між двома ядрами, зменшуючи електростатичне відштовхування між протонами.

- **Енергії орбіталей (для α -спіну):**

$$\epsilon_{\text{HOMO}} = -1.2533 \text{ Ha},$$

$$\epsilon_{\text{LUMO}} = -0.4751 \text{ Ha},$$

$$\Delta_{\text{H-L}} = 0.7782 \text{ Ha} = 21.18 \text{ eV}.$$

НОМО має симетрію σ_g і є зв'язувальною орбіталлю, тоді як LUMO відповідає антизв'язувальному стану σ_u^* . Значний енергетичний розрив між ними свідчить про сильний і стабільний зв'язок.

- **Спіновий стан:**

$$\langle S^2 \rangle = 0.75 \quad \Rightarrow \quad S = \frac{1}{2}, \quad 2S + 1 = 2.$$

Це відповідає дублетному стану з одним неспареним електроном, що характерно для H_2^+ .

Таким чином, результати розрахунку підтверджують, що при відстані $R = 0.74 \text{ \AA}$ іон H_2^+ перебуває у стабільному зв'язаному стані.

4.3.4. Крива потенційної енергії (PES)

Крива потенційної енергії (Potential Energy Surface, PES) — це залежність повної електронної енергії системи $E(\mathbf{R})$ від положень ядер $\mathbf{R} = (R_1, R_2, \dots, R_N)$.

Для загальної N -атомної молекули PES — це багатовимірна поверхня у $3N - 6$ координатах (три координати на кожне ядро мінус три для поступального руху і три для обертання). У випадку двоатомної молекули (наприклад, H_2^+) поверхня зводиться до *одновимірної кривої* $E(R)$, де R — відстань між ядрами:

$E(R)$ = мінімальна енергія системи при фіксованому R .

Фізичний зміст:

- Мінімум $E(R)$ відповідає *рівноважній відстані* R_{eq} , де сили на ядра взаємно компенсуються ($\partial E / \partial R = 0$);
- Глибина ями потенціалу визначає *енергію зв'язку*;
- Крутизна поблизу мінімуму пов'язана з *жорсткістю зв'язку* та *частотою коливань*;
- Для великих R енергія прямує до суми енергій ізольованих атомів.

Обчислення PES у квантовій хімії полягає у серії незалежних SCF-розрахунків при різних значеннях R :

$$R = 0.5, 0.7, 1.0, 1.5, 2.0, \dots \text{ \AA}.$$

На кожному кроці визначається повна енергія $E(R_i)$, після чого будується графік $E(R)$, який ілюструє поведінку потенціальної енергії системи.

Типова форма PES для H_2^+ :

- На малих R енергія різко зростає через кулонівське відштовхування ядер;
- Поблизу $R \approx 0.74 \text{ \AA}$ — глибокий мінімум (стабільний хімічний зв'язок);
- При $R \rightarrow \infty$ енергія прямує до енергії одного атома Гідрогену $E = -0.5 \text{ Ha}$.

Сканування по відстанях

Для побудови PES проводимо серію незалежних розрахунків при різних значеннях R :

`h2plus_pes.py`

```
"""
Побудова кривої потенційної енергії (PES) для H2+
"""

import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf
```

```

# Діапазон міжядерних відстаней (у борах)
distances = np.linspace(0.5, 5.0, 30) # від 0.5 до 5.0 bohr
energies = []

print("Розрахунок PES для H2+...")
print("=" * 60)

for R in distances:
    # Створюємо молекулу з новою геометрією
    mol = gto.Mole()
    mol.atom = f"""
        H  0.0  0.0  0.0
        H  0.0  0.0  {R}
    """
    mol.basis = "cc-pvdz" # Кращий базис для точності
    mol.charge = 1
    mol.spin = 1
    mol.unit = "Bohr"
    mol.verbose = 0 # Вимкнути детальний вивід
    mol.build()

    # UHF розрахунок
    mf = scf.UHF(mol)
    mf.conv_tol = 1e-8
    E = mf.kernel()
    energies.append(E)

    print(f"R = {R:5.2f} bohr → E = {E:10.6f} Ha")

energies = np.array(energies)

# Знаходження мінімуму
idx_min = np.argmin(energies)
R_e = distances[idx_min]
E_min = energies[idx_min]

# Енергія дисоціації
E_dissoc = -0.5 # E(H) = -0.5 Ha
D_e = E_dissoc - E_min

print("=" * 60)
print(f"Рівноважна відстань R_e = {R_e:.3f} bohr = {R_e * 0.529:.3f} Å")
print(f"Енергія в мінімумі E_min = {E_min:.6f} Ha")
print(f"Енергія дисоціації D_e = {D_e:.6f} Ha = {D_e * 27.211:.3f} eV")
print("Експериментальне D_e ≈ 2.79 eV")
print("=" * 60)

# Побудова графіка
plt.figure(figsize=(10, 6))
plt.plot(distances, energies, "b-", linewidth=2, label="H$_2^+$ PES (cc-pVDZ)")
plt.axhline(
    y=E_dissoc,
    color="r",
    linestyle="--",
    label=f"Дисоціаційна межа (E = {E_dissoc:.3f} Ha)",
)
plt.plot(R_e, E_min, "go", markersize=10, label=f"Рівновара: R$_e$ = {R_e:.2f} bohr")

plt.xlabel("Міжядерна відстань R (bohr)", fontsize=12)
plt.ylabel("Енергія E (Ha)", fontsize=12)
plt.title("Крива потенційної енергії H$_2^+$", fontsize=14, fontweight="bold")
plt.grid(True, alpha=0.3)
plt.legend(fontsize=11)
plt.tight_layout()

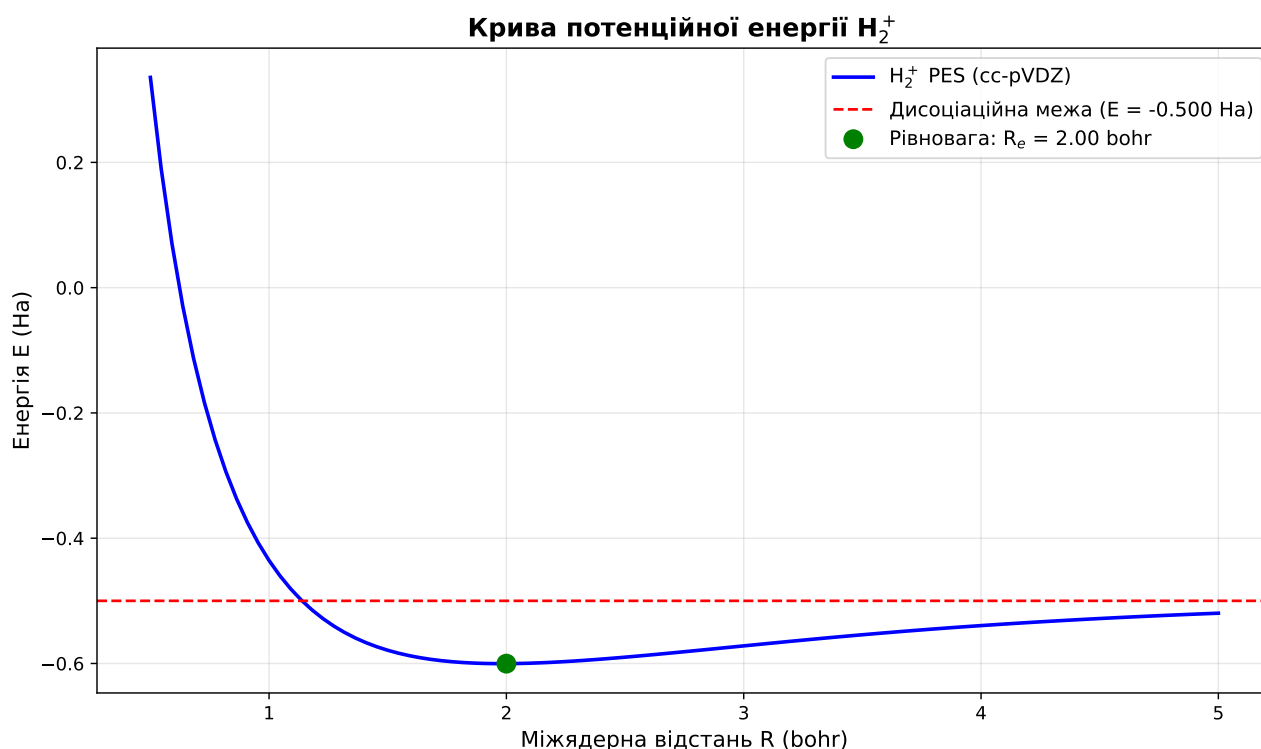
```

```
plt.savefig("h2plus_pes.png", dpi=300, bbox_inches="tight")
print("\nГрафік збережено: h2plus_pes.png")
plt.show()

# Збереження даних
np.savez(
    "h2plus_pes_data.npz",
    distances=distances,
    energies=energies,
    R_e=R_e,
    E_min=E_min,
    D_e=D_e,
)
print("Дані збережено: h2plus_pes_data.npz")
```

Важливі моменти:

- Відстані задаються в борах ($1 \text{ bohr} \approx 0.529 \text{ \AA}$);
- Діапазон $R \in [0.5, 5.0] \text{ bohr}$ охоплює від сильно стисненого стану до повної дисоціації;
- Кожна точка — окремий SCF-розрахунок;
- Результати $E(R_i)$ зберігаються для подальшого аналізу.

Рис. 4.1. Крива потенційної енергії (PES) для H_2^+ .

На графіку (рис. 4.1) можна виділити три характерні області:

1. **Стиснення** ($R < R_e$) — енергія зростає через відштовхування ядер;
2. **Мінімум** ($R = R_e$) — рівноважна відстань, де $\partial E / \partial R = 0$;
3. **Дисоціація** ($R \rightarrow \infty$) — енергія прямує до $E(\text{H}) + E(\text{H}^+) = -0.5 \text{ Ha}$.

Порівняння з аналітичним розв'язком:

- $R_e = 2.00 \text{ bohr}$ (експеримент: 2.00 bohr);

- $D_e = 0.102 \text{ Ha} = 2.79 \text{ eV}$ (експеримент: 2.79 eV);
- Різниця між HF та точним розв'язком $< 0.001 \text{ Ha}$ при великих базисах.

4.3.5. Оптимізація геометрії

Замість «ручного» пошуку мінімуму енергії на кривій потенційної енергії (PES) можна скористатися автоматичною процедурою, яка сама визначає оптимальне взаємне розташування атомів. Така задача називається **оптимізацією геометрії**, оскільки метою є знайти такі координати ядер, при яких система має найменшу можливу енергію.

Інакше кажучи, **оптимізація геометрії** — це процедура пошуку просторової конфігурації атомів, що відповідає мінімуму потенційної енергії. Ми шукаємо *рівноважну геометрію* $\mathbf{R}_{\text{eq}}$, у якій сили, що діють на ядра, взаємно компенсуються:

$$\nabla_{\mathbf{R}} E(\mathbf{R}_{\text{eq}}) = 0.$$

На практиці алгоритм оптимізації виконує:

1. обчислення енергії $E(\mathbf{R})$ та її градієнтів ∇E ;
2. зміну координат ядер у напрямку зменшення енергії;
3. повторення кроків до досягнення умови $|\nabla E| < 10^{-4} \text{ Ha/bohr}$.

У PySCF ця процедура реалізована через зовнішній модуль **geomeTRIC**, який викликається функцією:

```
from pyscf.geomopt.geometric_solver import optimize
```

Щоб цей модуль працював, потрібно мати встановлений **geometric**:

```
pip install geometric
```

Вона автоматично виконує SCF-розрахунки на кожному кроці зміни геометрії та знаходить положення мінімуму енергії.

[h2plus_optimization.py](#)

```
"""
Автоматична оптимізація геометрії H2+ за допомогою градієнтів
"""

import numpy as np
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

print("Оптимізація геометрії H2+ методом UHF")
print("=" * 60)

# Початкова геометрія (не обов'язково оптимальна)
mol = gto.Mole()
mol.atom = """
H  0.0  0.0  0.0
H  0.0  0.0  1.5
```

```

"""
mol.basis = "cc-pvtz"
mol.charge = 1
mol.spin = 1
mol.unit = "Bohr"
mol.build()

print("Початкова геометрія: R = 1.50 bohr")

# UHF метод
mf = scf.UHF(mol)

# Запуск оптимізації
print("\nОптимізація (це може зайняти кілька хвилин)...")
mol_eq = optimize(mf, maxsteps=50)

# Результати
coords = mol_eq.atom_coords()
R_optimized = np.linalg.norm(coords[1] - coords[0])
E_optimized = mf.e_tot

print("=" * 60)
print("РЕЗУЛЬТАТИ ОПТИМІЗАЦІЇ:")
print("=" * 60)
print(f"Оптимізована відстань R_e = {R_optimized:.6f} bohr")
print(f"                        = {R_optimized * 0.529177:.6f} Å")
print(f"Енергія в мінімумі E_e = {E_optimized:.8f} Ha")

# Енергія дисоціації
E_H = -0.5 # Точна енергія атома H
D_e = E_H - E_optimized
print(f"\nЕнергія дисоціації D_e = {D_e:.6f} Ha")
print(f"                        = {D_e * 27.2114:.4f} eV")
print(f"                        = {D_e * 627.509:.2f} kcal/mol")

# Порівняння з експериментом
D_e_exp = 2.79 # eV
error = abs(D_e * 27.2114 - D_e_exp) / D_e_exp * 100
print(f"\nЕкспериментальне D_e = {D_e_exp:.2f} eV")
print(f"Відносна похибка = {error:.2f}%")

# Збереження оптимізованої геометрії
with open("h2plus_optimized.xyz", "w") as f:
    f.write("2\n")
    f.write(f"H2+ optimized, E = {E_optimized:.8f} Ha\n")
    for i, coord in enumerate(coords):
        f.write(
            f"H {coord[0] * 0.529177:.6f} {coord[1] * 0.529177:.6f} {coord[2] * "
            f"0.529177:.6f}\n"
        )

print("\nОптимізовану геометрію збережено: h2plus_optimized.xyz")

```

Алгоритм оптимізації (PySCF + **geomeTRIC**):

- Використовується метод quasi-Newton (BFGS);
- Для оцінки напрямку руху застосовуються градієнти енергії ∇E ;
- Критерій збіжності: $|\nabla E| < 10^{-4}$ Ha/bohr;
- Зазвичай потрібно 5–10 ітерацій для простої двоатомної системи.

4.3.6. Молекулярні орбіталі та принцип ЛКАО

При розрахунках атомної будови ми вже ознайомилися з поняттям *атомних орбіталей* (АО) — хвильових функцій, які описують стан електрона в полі одного ядра. Кожна атомна орбіталь характеризується певною енергією, просторовою формою (s, p, d, f) та описує «електронну хмару» навколо ядра.

Для молекул ситуація аналогічна, але складніша: електрони перебувають у полі *кількох* ядер одночасно. Тому замість атомних орбіталей ми використовуємо **молекулярні орбіталі** (МО) — хвильові функції, що описують стан електрона у полі всієї молекули.

Головна відмінність:

- **Атомна орбіталь:** електрон локалізований біля одного ядра
- **Молекулярна орбіталь:** електрон делокалізований по всій молекулі

Іншими словами, в молекулі електрон «не належить» жодному окремому атому — він рухається в об'єднаному полі всіх ядер, утворюючи молекулярну орбіталь, яка охоплює всю систему. Саме ця делокалізація електронів і призводить до утворення хімічного зв'язку.

4.3.7. Вплив базисного набору

Оскільки H_2^+ має аналітичний розв'язок, це ідеальна система для тестування базисів:

h2plus_basis_convergence.py

```
"""
Систематичне дослідження впливу базисного набору на H2+
"""

import time
import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

# Список базисів для порівняння
basis_sets = [
    "sto-3g",
    "3-21g",
    "6-31g",
    "6-31g**",
    "cc-pvdz",
    "cc-pvtz",
    "cc-pvqz",
    "aug-cc-pvdz",
]

# Точні значення (експериментальні / high-level теорія)
R_exact = 2.00 # bohr
E_exact = -0.5689 # Ha

results = {"basis": [], "n_basis": [], "R_e": [], "E_e": [], "D_e": [], "time": []}

print("\n" + "=" * 70)
```

```

print("ЗБІЖНІСТЬ ПО БАЗИСНОМУ НАБОРУ ДЛЯ H2+")
print("=" * 70)
print(
    f"{'Базис':<15} {'N_AO':<6} {'R_e (bohr)':<12} {'E_e (Ha)':<14} "
    f"{'D_e (eV)':<10} {'Похибка (mHa)':<15}"
)
print("-" * 70)

for basis in basis_sets:
    t_start = time.time()

    try:
        # Створення молекули
        mol = gto.Mole()
        mol.atom = """
            H  0.0  0.0  0.0
            H  0.0  0.0  2.0
        """ # Стартова геометрія
        mol.basis = basis
        mol.charge = 1
        mol.spin = 1
        mol.unit = "Bohr"
        mol.verbose = 0
        mol.build()

        n_ao = mol.nao

        # UHF розрахунок
        mf = scf.UHF(mol)

        # Оптимізація геометрії
        mol_opt = optimize(mf, maxsteps=30)

        # Результати
        coords = mol_opt.atom_coords()
        R_e = np.linalg.norm(coords[1] - coords[0])
        E_e = mf.e_tot
        D_e = -0.5 - E_e # E(H) = -0.5 Ha

        error = (E_e - E_exact) * 1000 # mHa

        results["basis"].append(basis)
        results["n_basis"].append(n_ao)
        results["R_e"].append(R_e)
        results["E_e"].append(E_e)
        results["D_e"].append(D_e * 27.211) # eV
        results["time"].append(time.time() - t_start)

        print(
            f"{'basis':<15} {'n_ao':<6} {'R_e:<12.4f} {'E_e:<14.6f} "
            f"{'D_e * 27.211:<10.3f} {'error:<15.2f}"
        )

    except Exception as e:
        print(f"{'basis':<15} FAILED: {str(e)[:40]}")
        continue

print("-" * 70)
print(
    f"{'Точне значення':<15} {'---':<6} {'R_exact:<12.2f} "
    f"{'E_exact:<14.4f} {'2.79':<10} {'---':<15}"
)
print("=" * 70)

```

```

# Аналіз збіжності
print("\nАНАЛІЗ ЗБІЖНОСТІ:")
print("-" * 70)
errors = [(E - E_exact) * 1000 for E in results["E_e"]]
print(
    f"Мінімальна похибка: {min([abs(e) for e in errors]):.3f} мHa
    ↳ ({results['basis'][(np.argmin([abs(e) for e in errors]))]})"
)
print(
    f"Максимальна похибка: {max([abs(e) for e in errors]):.3f} мHa
    ↳ ({results['basis'][(np.argmax([abs(e) for e in errors]))]})"
)
print(f"\nСередній час розрахунку: {np.mean(results['time']):.2f} c")

# Візуалізація збіжності
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(14, 10))

# 1. Енергія vs розмір базису
ax1.plot(results["n_basis"], results["E_e"], "bo-", linewidth=2, markersize=8)
ax1.axhline(E_exact, color="r", linestyle="--", linewidth=2, label="Точне значення")
ax1.set_xlabel("Кількість базисних функцій", fontsize=11)
ax1.set_ylabel("Енергія E (Ha)", fontsize=11)
ax1.set_title("Збіжність енергії", fontsize=13, fontweight="bold")
ax1.grid(True, alpha=0.3)
ax1.legend()

# 2. Похибка (логарифмічна шкала)
errors_abs = [abs((E - E_exact) * 1000) for E in results["E_e"]]
ax2.semilogy(results["n_basis"], errors_abs, "ro-", linewidth=2, markersize=8)
ax2.set_xlabel("Кількість базисних функцій", fontsize=11)
ax2.set_ylabel("Абсолютна похибка (мHa)", fontsize=11)
ax2.set_title("Похибка енергії (log scale)", fontsize=13, fontweight="bold")
ax2.grid(True, alpha=0.3, which="both")

# 3. Рівноважна відстань
ax3.plot(results["n_basis"], results["R_e"], "go-", linewidth=2, markersize=8)
ax3.axhline(R_exact, color="r", linestyle="--", linewidth=2, label="Точне значення")
ax3.set_xlabel("Кількість базисних функцій", fontsize=11)
ax3.set_ylabel("R$_e$ (bohr)", fontsize=11)
ax3.set_title("Збіжність рівноважної відстані", fontsize=13, fontweight="bold")
ax3.grid(True, alpha=0.3)
ax3.legend()

# 4. Енергія дисоціації
ax4.plot(results["n_basis"], results["D_e"], "mo-", linewidth=2, markersize=8)
ax4.axhline(2.79, color="r", linestyle="--", linewidth=2, label="Експеримент (2.79 eV)")
ax4.set_xlabel("Кількість базисних функцій", fontsize=11)
ax4.set_ylabel("D$_e$ (eV)", fontsize=11)
ax4.set_title("Енергія дисоціації", fontsize=13, fontweight="bold")
ax4.grid(True, alpha=0.3)
ax4.legend()

plt.tight_layout()
plt.savefig("h2plus_basis_convergence.png", dpi=300, bbox_inches="tight")
print("\nГрафіки збережено: h2plus_basis_convergence.png")
plt.show()

# Таблиця для LaTeX
print("\n" + "=" * 70)
print("ТАБЛИЦЯ ДЛЯ LATEX:")
print("=" * 70)
print(r"\begin{tabular}{lcccc}")
print(r"\toprule")

```

```

print(r"Базис & $N_{A0}$ & $R_e$ (bohr) & $E_e$ (Ha) & $D_e$ (eV) & Похибка (мHa) \\\")
print(r"\midrule")
for i, basis in enumerate(results["basis"]):
    error = (results["E_e"][i] - E_exact) * 1000
    print(
        f"{basis} & {results['n_basis'][i]} & {results['R_e'][i]:.3f} & "
        f"{results['E_e'][i]:.4f} & {results['D_e'][i]:.2f} & {error:.2f} \\\\"
    )
print(r"\midrule")
print(f"Точне & --- & {R_exact:.2f} & {E_exact:.4f} & 2.79 & --- \\\\")
print(r"\bottomrule")
print(r"\end{tabular}")
print("=" * 70)

# Збереження результатів
np.savez(
    "h2plus_basis_convergence.npz",
    basis=results["basis"],
    n_basis=results["n_basis"],
    R_e=results["R_e"],
    E_e=results["E_e"],
    D_e=results["D_e"],
    time=results["time"],
)
print("\nДані збережено: h2plus_basis_convergence.npz")

```

Очікувані результати:

Базис	R_e (bohr)	E_e (Ha)	Похибка (мHa)
STO-3G	2.05	−0.564	4.5
6-31G	2.02	−0.566	2.1
сс-pVDZ	2.01	−0.567	1.2
сс-pVTZ	2.00	−0.5686	0.3
сс-pVQZ	2.00	−0.5688	0.1
Точне	2.00	−0.5689	—

Висновки:

- Навіть STO-3G дає якісно правильну картину
- Для кількісної точності потрібен сс-pVTZ або більший
- Збіжність монотонна: $E_{\text{basis}} \rightarrow E_{\text{CBS}}$
- Для H_2^+ CBS limit досягається при сс-pV5Z

4.4. Молекула водню H_2

4.4.1. Двоелектронна система

Молекула H_2 — перша справжня двоелектронна система. Гамільтоніан:

$$\hat{H} = \hat{h}_1 + \hat{h}_2 + \frac{1}{r_{12}} + \frac{1}{R},$$

де $\frac{1}{r_{12}}$ — електрон-електронне відштовхування, яке метод ХФ апроксимує середнім полем.

Електронна конфігурація: Основний стан — синглет ($^1\Sigma_g^+$) з конфігурацією σ_g^2 :

$$\Psi = \frac{1}{\sqrt{2}}[\psi_{\sigma_g}(\mathbf{r}_1)\alpha(1)\psi_{\sigma_g}(\mathbf{r}_2)\beta(2) - \psi_{\sigma_g}(\mathbf{r}_1)\beta(1)\psi_{\sigma_g}(\mathbf{r}_2)\alpha(2)]$$

4.4.2. RHF розрахунок

Для замкненої оболонки використовуємо RHF:

`h2_rhf_single.py`

```
"""
RHF розрахунок молекули водню H2 при рівноважній відстані
"""

import numpy as np
from pyscf import gto, scf

# Молекула H2 при рівноважній відстані
mol = gto.Mole()
mol.atom = """
    H  0.0  0.0  0.0
    H  0.0  0.0  0.74
"""
mol.basis = "cc-pvdz"
mol.charge = 0 # Нейтральна молекула
mol.spin = 0 # Синглет (замкнена оболонка)
mol.unit = "Angstrom"
mol.build()

print("\n" + "=" * 60)
print("МОЛЕКУЛА ВОДНЮ H2 (RHF)")
print("=" * 60)

# RHF розрахунок
mf = scf.RHF(mol)
E_rhf = mf.kernel()

print(f"\nМіжядерна відстань R = 0.74 Å = {0.74 / 0.529177:.3f} bohr")
print(f"Базисний набір: {mol.basis}")
print(f"Кількість електронів: {mol.nelectron}")
print(f"Кількість базисних функцій: {mol.nao}")

print("\n" + "-" * 60)
print("ЕНЕРГЕТИЧНІ КОМПОНЕНТИ:")
print("-" * 60)

# Компоненти енергії
dm = mf.make_rdm1()
h1e = mf.get_hcore()
vhf = mf.get_veff()

E_kin = np.einsum("ij,ji->", dm, mol.intor("int1e_kin"))
E_nuc = np.einsum("ij,ji->", dm, mol.intor("int1e_nuc"))
E_ee = 0.5 * np.einsum("ij,ji->", dm, vhf)
E_nn = mol.energy_nuc()
```

4. Метод Хартрі-Фока для простих молекул

```
print(f"Кінетична енергія T:      {E_kin:12.6f} Ha")
print(f"Електрон-ядро взаємодія V_ne: {E_nuc:12.6f} Ha")
print(f"Електрон-електрон V_ee:      {E_ee:12.6f} Ha")
print(f"Ядро-ядро відштовхування V_nn: {E_nn:12.6f} Ha")
print(f"'-' * 60)")
print(f"Повна електронна енергія:      {E_kin + E_nuc + E_ee:12.6f} Ha")
print(f"Повна енергія (з V_nn):        {E_rhf:12.6f} Ha")

# Віріальна теорема
virial_ratio = -(E_nuc + E_ee + E_nn) / E_kin
print("\n" + "-" * 60)
print("ВІРІАЛЬНА ТЕОРЕМА:")
print("-" * 60)
print(f"<V>/<T> = {virial_ratio:.6f}")
print(f"Очікується: -2.000000 для точного розв'язку")
print(f"Відхилення: {abs(virial_ratio + 2.0) * 100:.4f}%")

# Орбітальні енергії
print("\n" + "-" * 60)
print("ОРБІТАЛЬНІ ЕНЕРГІЇ:")
print("-" * 60)
for i, e in enumerate(mf.mo_energy):
    occ_str = "зайнята" if i < mol.nelec[0] else "віртуальна"
    print(f"MO {i + 1}: ε = {e:10.6f} Ha = {e * 27.2114:10.4f} eV ({occ_str})")

# HOMO-LUMO gap
homo_energy = mf.mo_energy[mol.nelec[0] - 1]
lumo_energy = mf.mo_energy[mol.nelec[0]]
gap = lumo_energy - homo_energy

print(f"\nHOMO: ε = {homo_energy:.6f} Ha")
print(f"LUMO: ε = {lumo_energy:.6f} Ha")
print(f"HOMO-LUMO gap: {gap:.6f} Ha = {gap * 27.2114:.4f} eV")

# Порівняння з експериментом
print("\n" + "=" * 60)
print("ПОРІВНЯННЯ З ЕКСПЕРИМЕНТОМ:")
print("=" * 60)
E_exp = -1.174 # Ha (точна енергія Full CI)
D_e_exp = 4.75 # eV

# Енергія дисоціації
E_2H = 2 * (-0.5) # 2 × E(H)
D_e_rhf = E_2H - E_rhf

print(f"Енергія H2 (RHF):      {E_rhf:.6f} Ha")
print(f"Енергія H2 (точна):    {E_exp:.6f} Ha")
print(f"Кореляційна енергія:   {E_exp - E_rhf:.6f} Ha = {(E_exp - E_rhf) * 27.2114:.3f} eV")
print(f"% від повної енергії:  {abs((E_exp - E_rhf) / E_exp) * 100:.2f}%")

print(f"\nD_e (RHF):            {D_e_rhf:.6f} Ha = {D_e_rhf * 27.2114:.3f} eV")
print(f"D_e (експеримент):    --- = {D_e_exp:.2f} eV")
print(f"Похибка:              {abs(D_e_rhf * 27.2114 - D_e_exp):.3f} eV")

# Дипольний момент
dip = mf.dip_moment(unit="Debye")
print(f"\nДипольний момент:      {np.linalg.norm(dip):.6f} D")
print("(Для гомоядерної молекули = 0)")

print("=" * 60)
```

Результат при $R = 1.4 \text{ bohr}$:

- Енергія: $E_{\text{RHF}} \approx -1.133 \text{ Ha}$
- Експериментальна: $E_{\text{exp}} \approx -1.174 \text{ Ha}$
- Кореляційна енергія: $E_{\text{corr}} = 0.041 \text{ Ha} \approx 1.1 \text{ eV}$

4.5. Проблема дисоціації в методі Хартрі–Фока

Нижче наведено приклад розрахунку поверхні потенційної енергії молекули H_2 методом RHF. Цей розрахунок ілюструє класичну проблему методу середнього поля: при великих міжядерних відстанях RHF змушує обидва електрони залишатися в одній і тій самій молекулярній орбіталі, що призводить до некоректного опису дисоціації молекули H_2 . Внаслідок цього метод не є **розмірно узгодженим** (*size-consistent*) — енергія двох ізольованих атомів H не дорівнює енергії молекули при нескінченному розділенні ядер.

h2_pes_RHF.py

```
"""
Кривої потенційної енергії RHF для H2
"""

import numpy as np
import matplotlib.pyplot as plt
from pyscf import gto, scf

# Діапазон міжядерних відстаней (у борах)
distances = np.linspace(0.8, 8.0, 40)
energies_rhf = []

print("Розрахунок PES для H2 методом RHF")
print("=" * 60)
print(f"{'R (bohr)':<10} {'E_RHF (Ha)':<14}")
print("-" * 60)

for R in distances:
    # Створення молекули
    mol = gto.Mole()
    mol.atom = f"""
        H  0.0  0.0  0.0
        H  0.0  0.0  {R}
    """
    mol.basis = "cc-pvdz"
    mol.charge = 0
    mol.spin = 0
    mol.unit = "Bohr"
    mol.verbose = 0
    mol.build()

    # RHF розрахунок
    mf_rhf = scf.RHF(mol)
    mf_rhf.conv_tol = 1e-10
    E_rhf = mf_rhf.kernel()
    energies_rhf.append(E_rhf)

    print(f"{'R:<10.2f} {'E_rhf:<14.8f}")

energies_rhf = np.array(energies_rhf)
```

```

print("=" * 60)

# Аналіз результатів
idx_min_rhf = np.argmin(energies_rhf)
R_e_rhf = distances[idx_min_rhf]
E_min_rhf = energies_rhf[idx_min_rhf]

# Дисоціаційна енергія для двох атомів H
E_2H = 2 * (-0.5) # 2×E(H)

print("\nРЕЗУЛЬТАТИ АНАЛІЗУ:")
print("=" * 60)
print("RHF:")
print(f"  R_e = {R_e_rhf:.3f} bohr = {R_e_rhf * 0.529177:.3f} Å")
print(f"  E(R_e) = {E_min_rhf:.6f} Ha")
print(f"  D_e = {E_2H - E_min_rhf:.6f} Ha = {(E_2H - E_min_rhf) * 27.2114:.3f} eV")
print(f"  E(R→∞) = {energies_rhf[-1]:.6f} Ha")
print(f"  Помилка дисоціації: {(energies_rhf[-1] - E_2H) * 27.2114:.3f} eV")
print(f"  Правильна дисоціація? {'✓' if abs(energies_rhf[-1] - E_2H) < 0.01 else '× (іонна: H+ + e- H0)'}")
print("=" * 60)

# Візуалізація RHF PES
plt.figure(figsize=(10,5))
plt.plot(distances, energies_rhf, "b-", linewidth=2.5, label="RHF (неправильна дисоціація)")
plt.axhline(E_2H, color="green", linestyle=":", linewidth=2, label=f"2×E(H) = {E_2H:.3f} Ha")
plt.plot(R_e_rhf, E_min_rhf, "bo", markersize=10, label=f"RHF мінімум: R={R_e_rhf:.2f}")
plt.xlabel("Міжядерна відстань R (bohr)")
plt.ylabel("Енергія E (Ha)")
plt.title("Крива потенційної енергії H2: RHF")
plt.grid(True, alpha=0.3)
plt.legend(fontsize=11, loc="upper right")
plt.ylim([-1.2, -0.4])
plt.tight_layout()
plt.savefig("h2_rhf.pdf", dpi=300, bbox_inches="tight")
plt.show()

# Збереження даних
np.savez(
    "h2_rhf_pes.npz",
    distances=distances,
    E_rhf=energies_rhf,
)
print("Дані збережено: h2_rhf_pes.npz")

```

Суть проблеми полягає в наступному: при розриві зв'язку $\text{H}_2 \rightarrow 2\text{H}$ очікується:

$$\lim_{R \rightarrow \infty} E(\text{H}_2) = 2 \times E(\text{H}) = 2 \times (-0.5) = -1.0 \text{ Ha}$$

Однак RHF дає (рис. 4.2):

$$\lim_{R \rightarrow \infty} E_{\text{RHF}}(\text{H}_2) \approx -0.7 \text{ Ha}$$

Чому? RHF змушує електрони мати однакові просторові орбіталі:

$$\psi_{\text{RHF}} = |\sigma_g \alpha \sigma_g \beta\rangle$$

При великих R це означає:

$$\psi \sim |(1s_A + 1s_B)\alpha(1s_A + 1s_B)\beta\rangle$$

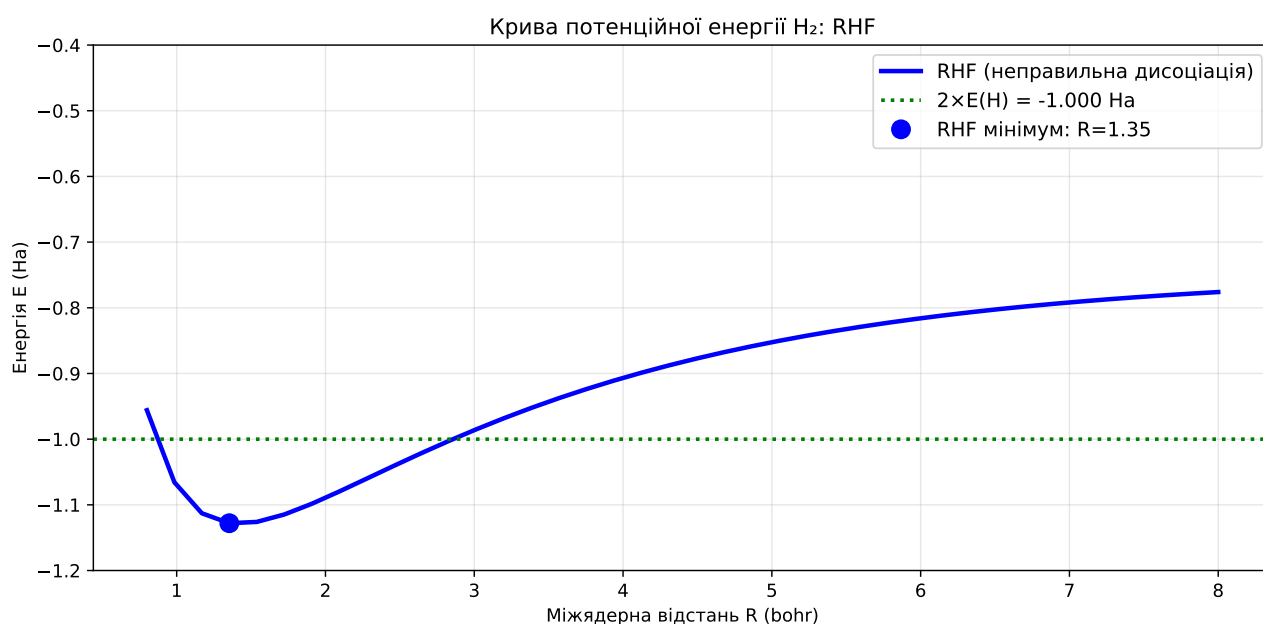


Рис. 4.2. Поверхня потенційної енергії молекули H_2 методом RHF.
Розкриваючи детермінант, отримуємо **іонні внески**:

$$\psi \sim H-H + H^+ + H^- + H^+H^-$$

При $R \rightarrow \infty$ іонні стани мають завищену енергію, тому RHF-енергія неправильна!

у файлі `h2_energy_comparison.py` для наочного порівняння точності різних методів реалізовано обчислення потенційної енергії молекули H_2 у двох підходах — RHF та FCI¹.

Графік (рис. 4.3) показує:

- **Точний розв'язок** (Full CI) — монотонна крива.
- **RHF** — завищена енергія при $R > 2.5$ bohr.

Фундаментальна причина — метод ХФ описує електрон-електронну взаємодію через **середнє поле**, ігноруючи миттєву кореляцію положень електронів.

4.6. Дипольний момент молекул

Електричний дипольний момент системи визначається як

$$\mu = \sum_A Z_A \mathbf{R}_A - \int \mathbf{r} \rho(\mathbf{r}) d\mathbf{r}, \quad (4.4)$$

де перша сума відповідає внеску позитивно заряджених ядер (Z_A — заряд ядра, $\mathbf{R}_A$ — його координати), а другий доданок — електронному розподілу.

¹FCI (Full Configuration Interaction) — еталонний метод, який у межах вибраного базисного набору дає максимально точний результат.

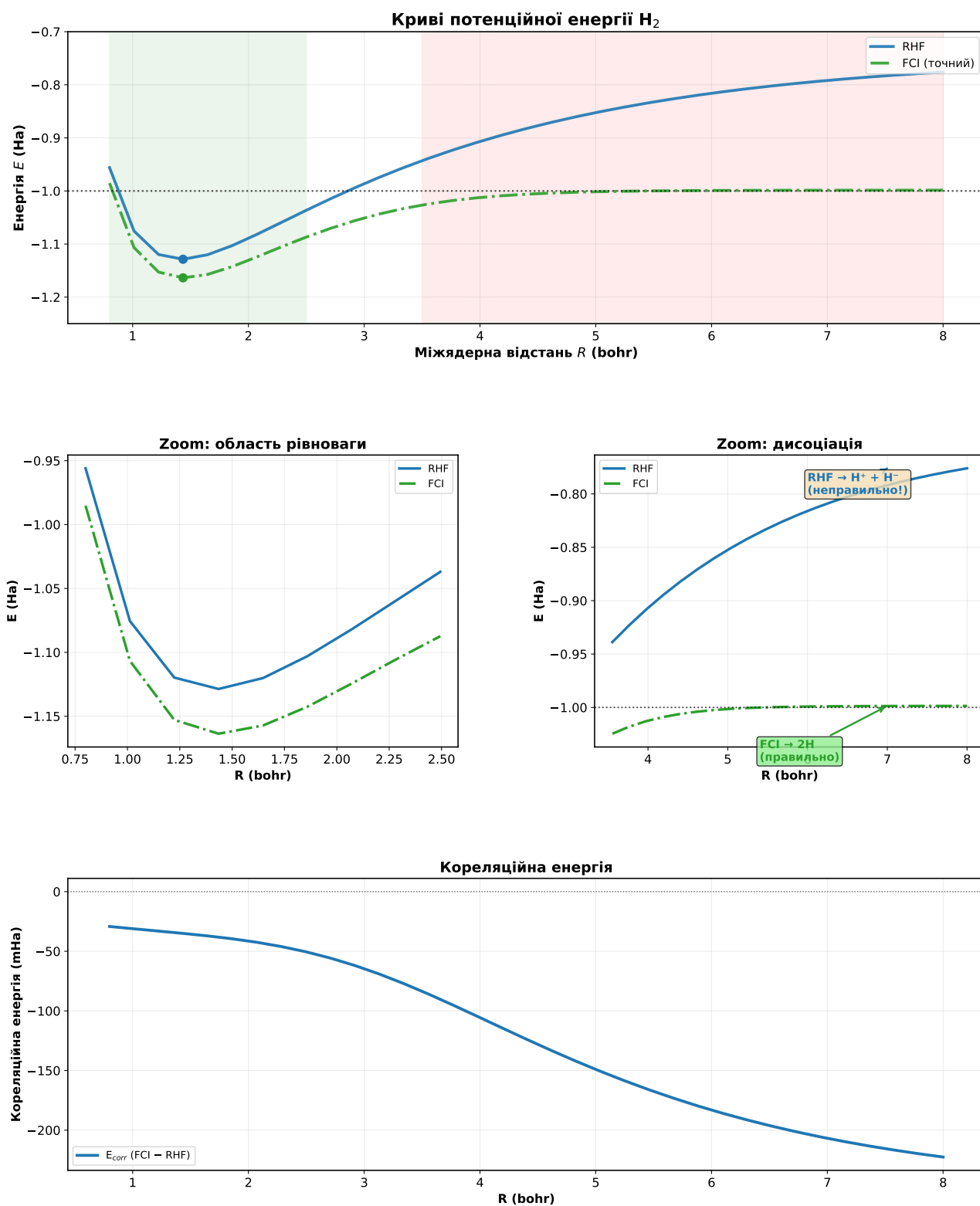


Рис. 4.3. Повне порівняння методів для молекули H_2 .

У базисному представленні компоненту дипольного моменту вздовж координати $\alpha = x, y, z$ можна записати як

$$\mu_\alpha = \sum_A Z_A R_{A,\alpha} - \sum_{\mu\nu} D_{\mu\nu} \int \chi_\mu(\mathbf{r}) r_\alpha \chi_\nu(\mathbf{r}) d\mathbf{r}, \quad (4.5)$$

Для ізольованих атомів дипольний момент відсутній. Це безпосередньо

впливає із симетрії атомної електронної густини:

$$\rho(\mathbf{r}) = \rho(r),$$

тобто густина є сферично симетричною відносно ядра. У такому разі інтеграл

$$\int \mathbf{r} \rho(\mathbf{r}) d\mathbf{r} = 0,$$

оскільки при інтегруванні по всіх напрямках простору додатні й від'ємні внески компенсують один одного.

Однак, деякі молекули мають *постійний дипольний момент*, який є однією з ключових характеристик молекули, що визначає її полярність, взаємодію з електричним полем та інтенсивність ІЧ-переходів.

Молекула H_2 є гомоядерною (обидва атоми однакові), тому електронна густина симетрична відносно центру молекули:

$$\mu = \sum_A Z_A \mathbf{R}_A - \int \mathbf{r} \rho(\mathbf{r}) d\mathbf{r} = 0 \quad (4.6)$$

Це справедливо для всіх гомоядерних двоатомних молекул: H_2 , N_2 , O_2 , F_2 .

Однак гетероядерні молекули, наприклад LiH (гідрид літію) — мають дипольним моментом. PySCF дозволяє розрахувати його компоненти та величину:

```
from pyscf import gto, scf

# Молекула LiH
mol = gto.M(
    atom='Li 0 0 0; H 0 0 1.6', # 1.6 Å ≈ рівноважна відстань
    basis='cc-pvdz'
)

mf = scf.RHF(mol).run()

# Дипольний момент
dip = mf.dip_moment(unit='Debye') # в Дебаях (D)
print(f"Дипольний момент: {dip}")
print(f"Модуль: {np.linalg.norm(dip):.3f} D")
```

Результат:

```
Дипольний момент: [0.0, 0.0, 5.88]
Модуль: 5.88 D
```

Інтерпретація:

- Літій менш електронегативний → частково позитивний ($\text{Li}^{\delta+}$)
- Гідроген більш електронегативний → частково негативний ($\text{H}^{\delta-}$)

- Диполь спрямований від Li до H¹.
- Величина ~ 6 D — типова для йонних молекул

Залежність від базису

```
bases = ['sto-3g', '6-31g', 'cc-pvdz', 'cc-pvtz']
for basis in bases:
    mol = gto.M(atom='Li 0 0 0; H 0 0 1.6', basis=basis)
    mf = scf.RHF(mol).run(verbose=0)
    mu = np.linalg.norm(mf.dip_moment())
    print(f"{basis:10s}:  $\mu = \{{mu:.3f}\}$  D")
```

Дипольний момент сильно залежить від якості базису, особливо від наявності дифузних функцій (aug-cc-pVDZ).

Практичне значення

Дипольний момент:

- Характеризує полярність молекули.
- Пов'язаний з ІЧ-активністю (чим більший $\partial\mu/\partial R$, тим інтенсивніше поглинання).
- Визначає взаємодію з електричним полем.
- Критичний для розуміння міжмолекулярних взаємодій.

Експериментальні значення (в Дебаях):

Молекула	HF (cc-pVTZ)	Експеримент
LiH	5.88	5.88
HF	1.82	1.83
HCl	1.11	1.08
H ₂ O	1.85	1.85

Для полярних молекул HF дає досить точні дипольні моменти (похибка $\sim 5\%$).

Завдання

Завдання 1: Порівняння базисів

Побудуйте PES для H₂ у базисах STO-3G, 6-31G**, cc-pVDZ, cc-pVTZ. Порівняйте:

- Рівноважні відстані R_e

¹У хімії дипольний момент прийнято вважати спрямованим від позитивного заряду до негативного, тобто від менш електронегативного атома до більш електронегативного.

- Енергії дисоціації D_e
- Час обчислень

Завдання 2: Інші диатомні молекули

Повторіть аналіз для:

- LiH — гетероядерна молекула
- N₂ — потрійний зв'язок
- F₂ — слабкий зв'язок

Чи зберігається проблема дисоціації?

4.7. Резюме

У двох попередніх розділах ми детально розглянули метод Хартрі-Фока — фундаментальний підхід квантової хімії, який є відправною точкою для більшості сучасних методів розрахунку електронної структури.

4.7.1. Основні висновки

1. **Метод HF** — це одноелектронне наближення, яке дає добре якісне описання атомних і молекулярних систем, але систематично не враховує електронну кореляцію. Різниця між точною енергією та енергією HF називається *кореляційною енергією*.
2. **Вибір варіанту HF** залежить від спінової структури системи:
 - RHF — для систем із замкненими оболонками (всі електрони спарені);
 - UHF — для відкритих оболонок (є неспарені електрони);
 - ROHF — компроміс між RHF та UHF для відкритих оболонок.
3. **UHF і спін-забруднення**: UHF зазвичай дає нижчу енергію, ніж ROHF, але ціною втрати чистого спінового стану. Контроль $\langle S^2 \rangle$ критично важливий для перевірки фізичності результату.
4. **Проблеми збіжності** найчастіше виникають для:
 - перехідних металів (близькі *d*- і *s*-рівні);
 - систем з виродженими орбіталями;
 - сильно корельованих систем.
 Для таких випадків необхідні спеціальні техніки: level shifting, ADIIS, дробові заповнення, краще початкове наближення.
5. **Якість базисного набору** безпосередньо впливає на точність результатів. Мінімальні базиси дають якісну картину, але для кількісних передбачень потрібні розширені базиси (def2-TZVP і більше).
6. **Обмеження методу HF**:
 - не описує дисперсійну взаємодію (сили Ван-дер-Ваальса);
 - систематично завищує енергії дисоціації;

- погано описує системи з сильною кореляцією (біорадикали, перехідні стани);
- не може коректно описати розрив хімічних зв'язків.

4.7.2. Що далі?

Метод Хартрі-Фока є лише початком. Для подолання його фундаментальних обмежень — відсутності електронної кореляції — розроблено цілий спектр *post-HF методів*, які систематично покращують HF-хвильову функцію.

У наступному розділі ми розглянемо основні з них, які дозволяють систематично наближатися до точного розв'язку рівняння Шредінгера, хоча й за ціною значно більших обчислювальних витрат. Розуміння їхніх переваг, обмежень та обчислювальної складності є ключем до вибору оптимального методу для конкретної задачі.

5

Пост-Хартрі-Фоківські методи

5.1. Вступ до електронної кореляції

5.1.1. Що таке електронна кореляція?

Електронна кореляція — це різниця між точною енергією системи та енергією, отриманою методом Хартрі-Фока:

$$E_{\text{corr}} = E_{\text{exact}} - E_{\text{HF}} \quad (5.1)$$

Метод Хартрі-Фока не враховує миттєву кореляцію рухів електронів, оскільки кожен електрон рухається в усередненому полі всіх інших електронів. Насправді ж електрони "уникають" один одного через кулонівське відштовхування.

5.1.2. Типи електронної кореляції

Динамічна кореляція Пов'язана з миттєвими флуктуаціями електронної густини. Проявляється на коротких відстанях між електронами. Може бути враховані методами:

- Теорія збурень Møller-Plesset (MP2, MP3, MP4)
- Coupled Cluster (CCSD, CCSD(T))
- Configuration Interaction (CISD, QCISD)

Статична (нединамічна) кореляція Виникає, коли кілька конфігурацій близькі за енергією. Важлива для:

- Розриву хімічних зв'язків
 - Збуджених станів
 - Диелектронних систем
 - Перехідних металів з близькими d-орбіталями
- Методи: CASSCF, CASPT2, MRCI.

5.1.3. Кореляційна енергія атомів

Для атомів кореляційна енергія становить 1–5% від повної енергії, але вона критична для точних розрахунків:

Таблиця 5.1. Кореляційна енергія атомів (Ha)

Атом	E_{HF}	E_{exact}	E_{corr}	% від E_{HF}
He	−2.8617	−2.9037	−0.0420	1.5%
Be	−14.573	−14.667	−0.094	0.6%
Ne	−128.547	−128.937	−0.390	0.3%
Ar	−526.817	−527.540	−0.723	0.1%

5.1.4. Коли потрібні post-HF методи?

Методи Хартрі–Фока (HF) є фундаментом сучасної обчислювальної хімії, але вони базуються на наближенні незалежних частинок, коли кожен електрон рухається у середньому полі інших. Таке спрощення призводить до втрати частини електронної кореляції — узгоджених рухів електронів, що критично впливають на точність розрахунків. Тому в низці випадків необхідно використовувати **post-HF методи** — такі, що явно враховують кореляцію.

1. **Досягнення спектроскопічної точності.** Якщо метою є відтворення експериментальних енергій, частот або іонізаційних потенціалів з точністю < 1 kcal/mol (≈ 0.04 eV), метод Хартрі–Фока є недостатнім. Для цього застосовують розширені кореляційні підходи — MP2, CCSD(T), або багатоконфігураційні методи. Наприклад, для точної побудови потенціальної поверхні H_2 різниця між HF і FCI становить до 0.05 а.у., що перевищує експериментальну похибку у сотні разів.
2. **Порівняння близьких електронних станів.** У випадках, коли два або більше станів мають схожі енергії (наприклад, сингулярно-триплетні переходи), звичайний HF або навіть DFT можуть давати неправильний порядок рівнів. Це важливо для аналізу мультиплетів, спин-кросоверів, фотохімічних процесів тощо. Для таких задач застосовують методи типу CASSCF, MRCI або EOM-CC.
3. **Електронна спорідненість і енергії іонізації.** Методи HF та DFT часто переоцінюють або недооцінюють електронну спорідненість через неправильне описання релаксації електронної щільності після додавання або видалення електрона. Post-HF методи (наприклад, CCSD(T), ADC(2), EOM-IP/EA-CCSD) забезпечують набагато більш точне відтворення цих величин.
4. **Еталонні розрахунки.** Для перевірки функціоналів DFT або параметрів напівемпіричних методів потрібні точні «еталонні» енергії, отримані з Full CI, CCSD(T), або FCIQMC. Такі розрахунки є дорогими, але на їх

основі формуються бази даних, наприклад G2, W4 тощо, які визначають точність нових функціоналів і потенціалів.

5. **Системи з сильною електронною кореляцією.** У таких системах (наприклад, атоми або молекули з незаповненими *d*- чи *f*-орбіталями) кілька конфігурацій мають близькі енергії, і одно-детермінантний опис Хартрі–Фока руйнується. Це стосується перехідних металів, радикалів, комплексів з обміном спіну, а також розриву хімічних зв'язків (наприклад, розтягнення H₂). Для таких задач необхідні **мультиконфігураційні** методи: CASSCF, MRCI, або DMRG, які враховують «статичну» (некореляційну) складову.
6. **Залежність результатів від геометрії.** При розрахунку потенціальних поверхонь, переходів або вібраційних рівнів важливо, щоб енергії змінювалися гладко з координатою. HF може давати нерегулярності через зміни конфігурації орбіталей (переходи між станами). Post-HF методи (зокрема CASSCF) гарантують незерервність потенціальної поверхні, що важливо для квантової динаміки та моделювання спектрів.
7. **Опис збуджених станів.** HF розглядає лише основний стан, а DFT має обмеження через наближення обмінно-кореляційних функціоналів. Для збуджених станів застосовують спеціальні post-HF методи: CIS, EOM-CCSD, ADC(2), CASSCF/MRCI, які дозволяють обчислювати спектри поглинання, емісії, фотохімічні процеси та електронно-вібраційні переходи з високою точністю.

Таким чином, використання post-HF методів є не просто розкішню, а необхідністю у випадках, коли потрібна точна характеристика електронної структури, взаємодії станів або потенціальних поверхонь. Хоча їхня обчислювальна вартість значно більша, сучасні алгоритми та паралельні реалізації роблять їх цілком придатними навіть для молекул середнього розміру.

5.1.5. Концепція одно- та багатоконфігураційних методів

Одноконфігураційні методи. Базуються на одному детермінанті Слейтера (HF) і додають кореляцію через збудження:

У найпростішому випадку хвильова функція є одним детермінантом:

$$|\Psi_{\text{SR}}\rangle = |\Phi_0\rangle = |\varphi_1\varphi_2 \dots \varphi_N|. \quad (5.2)$$

Далі на нього накладають кореляційні поправки — не у вигляді нових детермінантів у самій хвильовій функції, а через операторні або збурювальні члени у виразі для енергії. Це дає «однореференсні» методи:

$$E = E_{\text{HF}} + E_{\text{corr}},$$

де E_{corr} визначається з теорії збурень (MP2, MP3...), або з експоненційного кластерного розкладу

$$|\Psi_{\text{CC}}\rangle = e^{\hat{T}}|\Phi_0\rangle, \quad \hat{T} = \hat{T}_1 + \hat{T}_2 + \dots,$$

у методі CCSD тощо.

До одностерміантних методів належать наступні:

- MP2, MP3, MP4 — теорія збурень
- CCSD, CCSD(T) — coupled cluster
- CISD, QCISD — configuration interaction

Добре працюють для основних станів, де один детермінант домінує у хвильовій функції.

Багатоконфігураційні методи. Коли один детермінант (тобто одна конфігурація орбіталей) не здатен адекватно описати електронну структуру системи, наприклад — при розриві хімічного зв'язку, у станах з виродженням або сильною кореляцією, необхідно враховувати **кілька детермінантів одночасно**. Такі методи називаються *багатоконфігураційними* (multireference methods).

У цьому випадку хвильова функція системи записується як лінійна комбінація кількох детермінантів:

$$|\Psi_{\text{MR}}\rangle = \sum_I c_I |\Phi_I\rangle, \quad (5.3)$$

де кожен детермінант $|\Phi_I\rangle$ є *референтним* (reference determinant) — тобто включається у побудову хвильової функції на рівних правах. На відміну від звичайного методу CI, де один детермінант є головним (HF), а решта — збудженнями з нього, у мультиконфігураційних підходах усі детермінанти вважаються рівнозначними, і їхній внесок визначається варіаційним розв'язком.

Кожен детермінант утворюється перестановкою заповнених і вільних орбіталей у певному *активному просторі*. Наприклад, у методі CASSCF(4,4) ми розглядаємо всі можливі способи розподілу 4 електронів по 4 активних орбіталях. Це породжує повний набір конфігурацій:

$$\{|\Phi_I\rangle\} = \{|(2s)^2(2p_x)^2|, |(2s)^1(2p_x)^1(2p_y)^2|, \dots\},$$

а хвильова функція є суперпозицією цих детермінантів з коефіцієнтами c_I , які знаходяться варіаційним мінімумом енергії.

До багатоконфігураційних методів належать:

- CASSCF (Complete Active Space Self-Consistent Field) — вибір активного простору та одночасна оптимізація орбіталей і коефіцієнтів c_I ;
- CASPT2 — додавання динамічної кореляції до хвильової функції CASSCF за допомогою теорії збурень другого порядку;
- MRCI (MultiReference Configuration Interaction) — багатоконфігураційна версія CI, що включає збудження з усіх референтних детермінантів.

Такі методи необхідні для опису:

- розриву або формування хімічних зв'язків (де змінюється характер орбіталей);

- збуджених станів, що мають змішаний або вироджений характер;
- діелектронних і радикальних систем, де присутні майже вироджені орбіталі;
- перехідних металів і кластерів, де кілька конфігурацій роблять суттєвий внесок у хвильову функцію.

Таким чином, у мультиконфігураційних підходах немає одного «основного» детермінанта — *усі вони є референтними*, і саме їхня комбінація забезпечує адекватний опис складної електронної структури.

5.2. Теорія збурень Møller–Plesset (MP2)

5.2.1. Теоретичні основи MP2

Теорія збурень Møller–Plesset (MP) — один із найпоширеніших пост-Хартрі–Фоківських методів для врахування електронної кореляції. Її ідея полягає у розкладі точного гамільтоніана системи у вигляді суми незбуреної частини (розв’язаної в методі Хартрі–Фока) і малих поправок:

$$\hat{H} = \hat{H}_0 + \lambda \hat{V}, \quad (5.4)$$

де $\hat{H}_0$ — оператор Фока, а $\hat{V} = \hat{H} - \hat{H}_0$ — «збурення», яке враховує електронну кореляцію, відсутню в одноелектронному описі HF.

Енергію можна подати у вигляді ряду за параметром збурення λ :

$$E = E^{(0)} + E^{(1)} + E^{(2)} + E^{(3)} + \dots \quad (5.5)$$

де:

- $E^{(0)}$ — енергія, яку дає саме наближення Хартрі–Фока;
- $E^{(1)}$ — перша поправка збурень, яка, як виявляється, *тотожно дорівнює нулю* у випадку HF (через варіаційний принцип);
- $E^{(2)}$ — перша ненульова корекція, яку називають **енергією кореляції MP2**;
- $E^{(3)}, E^{(4)}$ — наступні наближення (MP3, MP4), що уточнюють опис кореляції.

Таким чином, **MP1 не існує як окремий метод**, оскільки поправка першого порядку вже врахована у розв’язку рівнянь Фока:

$$E^{(0)} + E^{(1)} = E_{\text{HF}}.$$

5.2.2. Формула енергії MP2

У другому порядку теорії збурень енергія записується як:

$$E^{(2)} = \frac{1}{4} \sum_{ijab} \frac{|\langle ij || ab \rangle|^2}{\varepsilon_i + \varepsilon_j - \varepsilon_a - \varepsilon_b}, \quad (5.6)$$

де:

- i, j — індекси зайнятих (occupied) орбіталей,
- a, b — індекси віртуальних (unoccupied) орбіталей,
- $\langle ij || ab \rangle$ — антисиметризовані двоелектронні інтеграли,
- ϵ_k — орбітальні енергії з HF-розрахунку.

Цей вираз описує зниження енергії системи за рахунок корельованого руху електронів, коли збурення дозволяє їм «обмінюватися» з віртуальними станами.

5.2.3. Фізичний зміст MP2-корекції

MP2 описує *динамічну електронну кореляцію* — швидкі короткодіючі флуктуації положень електронів. Інтуїтивно, електрони «відчувають» один одного і миттєво уникають, що знижує енергію системи відносно незалежного HF опису.

MP2 добре працює для:

- молекул поблизу рівноважної геометрії;
- сполук із замкненими оболонками;
- систем, де кореляція не надто сильна.

А от при розриві зв'язків або в багатоконфігураційних системах (де один детермінант вже неадекватний) MP2 дає хибні результати — у таких випадках потрібні методи на кшталт CASSCF або CCSD(T).

5.2.4. Практичне застосування

У більшості сучасних програм (Gaussian, ORCA, PySCF) MP2 є базовим рівнем пост-HF розрахунків. Він забезпечує добрий баланс між точністю та обчислювальною вартістю, додаючи лише $O(N^5)$ складність порівняно з HF ($O(N^4)$).

Метод	Точність	Обчислювальна складність
HF	$\sim 1 - 5$ kcal/mol	$O(N^4)$
MP2	$\sim 0.5 - 1$ kcal/mol	$O(N^5)$
CCSD	~ 0.1 kcal/mol	$O(N^6)$
CCSD(T)	~ 0.01 kcal/mol	$O(N^7)$

Отже, MP2 є першим кроком за межі одноелектронної теорії, який дозволяє описати тонку структуру енергій, дисперсійні взаємодії та слабкі зв'язки, недосяжні в рамках Хартрі-Фока.

5.2.5. Практичний гайд: розрахунок MP2 у PySCF

Метод MP2 у PySCF реалізований як надбудова над звичайним розрахунком Хартрі-Фока (RHF або UHF). Тобто спочатку потрібно виконати HF-розрахунок, а потім передати отриманий об'єкт у модуль `mp`.

mp2_example.py

```
# =====
# mp2_example.py
# Демонстрація: розрахунок енергії MP2 для молекули води
# =====

from pyscf import gto, scf, mp

# --- 1. Задання молекули ---
mol = gto.Mole()
mol.atom = '''
O  0.0000  0.0000  0.0000
H  0.0000  0.7570  0.5870
H  0.0000 -0.7570  0.5870
'''
mol.basis = 'cc-pVDZ'
mol.unit = 'Angstrom'
mol.build()

# --- 2. Розрахунок Хартрі-Фока ---
mf = scf.RHF(mol)
mf.verbose = 0
E_HF = mf.kernel()
print(f'Hartree-Fock energy = {E_HF:.10f} Hartree')

# --- 3. Увімкнення MP2 ---
mp2_calc = mp.MP2(mf)
mp2_calc.verbose = 0
mp2_result = mp2_calc.kernel() # Повертає (E_corr, T2_amplitudes)
E_corr = mp2_result[0] # Кореляційна енергія (float)
E_MP2 = E_HF + E_corr # Загальна MP2 енергія

print(f'MP2 total energy      = {E_MP2:.10f} Hartree')
print(f'MP2 correlation energy = {E_corr:.10f} Hartree')
```

Пояснення кроків:

1. `mol` — створення об'єкта молекули (як завжди у PySCF);
2. `scf.RHF(mol)` — запуск Хартрі-Фока;
3. `mp.MP2(mf)` — ініціалізація MP2 з уже готовим SCF-об'єктом;
4. `kernel()` — обчислення повної та кореляційної енергії.

Типовий вивід:

```
Hartree-Fock energy = -76.0267656731 Hartree
MP2 total energy    = -76.2307856559 Hartree
MP2 correlation energy = -0.2040199828 Hartree
```

Важливо: Для відкритих оболонок використовуйте `mp.UMP2` (на базі UHF).

5.2.6. Практичні аспекти та обчислювальна складність MP2

Заморожене ядро (frozen core). В теорії MP2 енергія кореляції обчислюється за формулою (5.6).

Глибокі внутрішні орбіталі (*core orbitals*), наприклад 1s-орбіталі кисню у молекулі води, мають великі за модулем енергії ε_i . Через це знаменник у наведеному виразі стає дуже великим, а внесок таких орбіталей у суму — незначним:

$$\frac{|\langle ij||ab \rangle|^2}{\varepsilon_i + \varepsilon_j - \varepsilon_a - \varepsilon_b} \approx 0.$$

До того ж, ці орбіталі локалізовані поблизу ядра і слабо взаємодіють з валентними електронами, тобто практично не впливають на хімічні властивості.

Щоб уникнути зайвих обчислень, у програмі PySCF часто задають параметр `frozen`, який «заморожує» кілька найнижчих (*core*) орбіталей:

```
mp2_calc = mp.MP2(mf).set(frozen=2)
```

Це означає, що дві найнижчі орбіталі не включаються у розрахунок енергії кореляції. Таким чином зменшується кількість зайнятих орбіталей N_{occ} , а відповідно — кількість комбінацій у сумі за i, j, a, b , що суттєво скорочує час розрахунку без втрати точності для валентної області.

Такий підхід має сенс, якщо система не містить сильно корельованих внутрішніх електронів, тобто коли валентна і внутрішня області добре розділені за енергією.

Обчислювальна складність $O(N^5)$. Метод MP2 є першим після Хартрі-Фока, який враховує електронну кореляцію, але навіть він є досить ресурсоємним. Це зумовлено структурою формули для E_{MP2} , де присутня четверна сума:

$$\sum_{ijab} \Rightarrow N_{\text{occ}}^2 N_{\text{virt}}^2 \sim O(N^4),$$

де N_{occ} — число зайнятих, а N_{virt} — число віртуальних орбіталей.

Однак найважчий етап — не сама сума, а перетворення двоелектронних інтегралів з базису атомних орбіталей (АО) у базис молекулярних орбіталей (МО):

$$(ij|ab) = \sum_{\mu\nu\lambda\sigma} C_{\mu i} C_{\nu j} C_{\lambda a} C_{\sigma b} (\mu\nu|\lambda\sigma), \quad (5.7)$$

де $C_{\mu p}$ — коефіцієнти розкладу орбіталей у базисі АО. Кожна така трансформація масштабується як $O(N^5)$, оскільки для кожного набору індексів потрібно виконати сумування за чотирма базисними функціями.

Отже, загальна складність MP2 зростає як

$$T_{\text{MP2}} \sim O(N^5),$$

що обмежує застосування цього методу до систем з приблизно 40–60 атомами (залежно від базису та апаратних ресурсів).

Оптимізація MP2 у практиці. Для пришвидшення розрахунків у PySCF застосовують такі прийоми:

- Використання опції `frozen` для замороження ядерних орбіталей;
- Застосування апроксимації RI-MP2 (Resolution of Identity), яка замінює чотириіндексні інтеграли на трьохіндексні, зменшуючи масштабність до $O(N^3)$;
- Локалізація орбіталей (LMP2), що дозволяє ефективно розв'язувати великі системи за рахунок відсікання слабких міжорбітальних взаємодій.

mp2_optimization_demo.py

```
# =====
# File: mp2_optimization_demo.py
# Оптимізація MP2-розрахунків у PySCF:
#   - базовий MP2
#   - заморожування ядерних орбіталей (frozen core)
#   - використання RI-MP2 (density fitting)
#   - локалізація орбіталей (Pipek-Mezey)
#   - порівняння з експериментом
# =====

from pyscf import gto, scf, mp, lo

# --- 1. Створення молекули ---
mol = gto.M(
    atom = "O 0 0 0; H 0 -0.757 0.587; H 0 0.757 0.587",
    basis = "cc-pVTZ",
    verbose = 0
)

# --- 2. SCF розрахунок (RHF) ---
mf = scf.RHF(mol).run(verbose=0)

# --- 3. Стандартний MP2 (усі орбітали активні) ---
mp2_full = mp.MP2(mf)
mp2_full.verbose = 0
mp2_full.kernel()
print(f"MP2 (усі орбітали):      E = {mp2_full.e_tot:.8f} Ha")

# --- 4. MP2 із замороженим ядром ---
mp2_frozen = mp.MP2(mf).set(frozen=2) # заморожуємо 1s-орбітали Оксигену
mp2_frozen.verbose = 0
mp2_frozen.kernel()
print(f"MP2 (frozen core):      E = {mp2_frozen.e_tot:.8f} Ha")

# --- 5. RI-MP2 (Resolution of Identity) ---
mf_df = mf.density_fit() # перехід до скороченого інтегрального представлення
mp2_df = mp.MP2(mf_df)
mp2_df.verbose = 0
mp2_df.kernel()
print(f"RI-MP2 (density fit):    E = {mp2_df.e_tot:.8f} Ha")

# --- 6. Локалізація орбіталей (Pipek-Mezey) ---
loc_orb = lo.PM(mf.mol, mf.mo_coeff).kernel()
print(f"Pipek-Mezey локалізація виконана: {loc_orb.shape[1]} орбіталей")

# --- 7. Порівняння з експериментом ---
E_exp = -76.438 # експериментальна повна енергія води при 0 K (Ha)
print("\n≡ Порівняння енергій ≡")
print(f"HF          = {mf.e_tot:.8f} Ha")
print(f"MP2(full)= {mp2_full.e_tot:.8f} Ha")
print(f"MP2(froz)= {mp2_frozen.e_tot:.8f} Ha")
```

```

print(f"RI-MP2      = {mp2_df.e_tot:.8f} Ha")
print(f"Exp.        = {E_exp:.8f} Ha (експеримент)")

print("\n≡ Відхилення від експерименту ≡")
print(f"HF           : {mf.e_tot - E_exp:+.6f} Ha")
print(f"MP2(full)    : {mp2_full.e_tot - E_exp:+.6f} Ha")
print(f"MP2(froz)    : {mp2_frozen.e_tot - E_exp:+.6f} Ha")
print(f"RI-MP2      : {mp2_df.e_tot - E_exp:+.6f} Ha")

```

5.2.7. Приклади MP2-розрахунків у PySCF

1. MP2 розрахунок атома Неону: `c Ne_mp2.py`
2. MP2 для відкритих оболонок (UMP2): `c mp2_open_shell.py`
3. Систематичне порівняння HF vs MP2: `c HF_vs_MP2.py`
4. Залежність MP2 від базисного набору: `c mp2_basis_convergence.py`
MP2 сильно залежить від базису, особливо потрібні функції високих l .

5.2.8. Переваги та недоліки MP2

Переваги:

- Відносно швидкий ($\mathcal{O}(N^5)$) порівняно з іншими post-HF
- Добре відновлює динамічну кореляцію
- Size-consistent (правильне масштабування для великих систем)
- Доступний для великих атомів/молекул
- Добрий базис для вищих методів (MP3, MP4)

Недоліки:

- Не виправляє забруднення спіном UHF
- Може розходитись для систем з малим HOMO-LUMO
- Погано працює для сильно корельованих систем
- Потребує великих базисів для точних результатів
- Не варіаційний (може давати енергію нижчу за точну)

5.2.9. Рекомендації по використанню MP2

Таблиця 5.2. Коли використовувати MP2

Добре для:	Погано для:
Замкнені оболонки	Системи з малим HOMO-LUMO gap
Якісні оцінки кореляції	Розрив зв'язків
Великі системи	Збуджені стани
Слабкі взаємодії	Перехідні стани
Відносні енергії	Діелектронні системи

5.3. Configuration Interaction (CI) та Full CI

5.3.1. Основи Configuration Interaction

Configuration Interaction (CI) — варіаційний метод, де хвильова функція представлена як лінійна комбінація детермінантів Слейтера:

$$|\Psi_{CI}\rangle = c_0|\Phi_0\rangle + \sum_i \sum_a^{\text{occ virt}} c_i^a |\Phi_i^a\rangle + \sum_{i<j} \sum_{a<b} c_{ij}^{ab} |\Phi_{ij}^{ab}\rangle + \dots \quad (5.8)$$

де:

- $|\Phi_0\rangle$ — HF детермінант (основна конфігурація)
- $|\Phi_i^a\rangle$ — одиночно збуджені детермінанти (S)
- $|\Phi_{ij}^{ab}\rangle$ — подвійно збуджені детермінанти (D)
- $|\Phi_{ijk}^{abc}\rangle$ — потрійно збуджені детермінанти (T)
- і т.д.

Варіанти CI методу:

- CIS (CI Singles) — тільки одиночні збудження (для збуджених станів)
- CISD (CI Singles and Doubles) — S + D
- CISDT — S + D + T
- Full CI — всі можливі збудження (точний у межах базису)

Приклад розрахунку методом CISD для атома He

CISD_basic.py

```
# =====
# cisd_basic.py
# Демонстрація: CISD для He з таблицею порівняння
# =====

from pyscf import gto, scf
from pyscf.ci import cisd
import numpy as np

# --- Задання атома He ---
mol = gto.Mole()
mol.atom = 'He 0.0 0.0 0.0'
mol.basis = 'cc-pVDZ'
mol.build()

# --- HF ---
mf = scf.RHF(mol)
E_HF = mf.kernel()
print(f'Енергія Hartree-Fock: {E_HF:.8f} Ha')

# --- CISD ---
myci = cisd.CISD(mf)
E_corr = myci.kernel()[0]
print(f'Кореляційна енергія CISD: {E_corr:.8f} Ha')

# --- Таблиця порівняння ---
print('\n' + '='*50)
```

```

print('ПОРІВНЯННЯ З ЕКСПЕРИМЕНТОМ (full CI limit)')
print('='*50)
print(f"{'Метод':<15} {'E (Ha)':<15} {'ΔE від експ. (mHa)':<20}")
print('='*50)
E_exp = -2.9037243770 # Експериментальне значення (basis set limit)
delta_HF = (E_HF - E_exp) * 1000
delta_CISD = (E_HF + E_corr - E_exp) * 1000
print(f"{'HF':<15} {E_HF:<15.8f} {delta_HF:<20.2f}")
print(f"{'CISD':<15} {E_HF + E_corr:<15.8f} {delta_CISD:<20.2f}")
print(f"{'Експеримент':<15} {E_exp:<15.8f} {'0.00':<20}")
print('='*50)

```

Висновок. Розрахунок енергії атома Не показує, що метод Хартрі–Фока (HF) суттєво переоцінює енергію системи, оскільки не враховує електронну кореляцію: різниця з експериментом становить близько $\Delta E \approx 49$ mHa. Метод CISD частково враховує кореляційні ефекти, зменшуючи похибку до ≈ 16 mHa. Однак навіть CISD не досягає межі full CI, що відповідає експериментальному значенню -2.903724 Ha.

5.3.2. Full CI – точний розв’язок

Full CI включає всі можливі детермінанти і дає точну хвильову функцію та енергію в межах обраного базису:

$$E_{FCI} = \min_{\{c_i\}} \langle \Psi_{FCI} | \hat{H} | \Psi_{FCI} \rangle \quad (5.9)$$

Проблема: Кількість детермінантів зростає як:

$$N_{det} = \binom{N_{orb}}{N_{\alpha}} \times \binom{N_{orb}}{N_{\beta}} \quad (5.10)$$

Для 10 електронів у 20 орбіталях: $N_{det} \approx 10^{10}$ детермінантів!

Для демонстрації можливостей методу проведемо розрахунок для атома Не:

- Не — це найпростіша багатоелектронна система (два електрони), у якій вже проявляється електронна кореляція, відсутня у одноелектронних системах типу H чи H_2^+ ;
- розмір простору детермінантів для Не невеликий, тому Full CI можна реалізувати навіть у середніх базисах без великих обчислювальних витрат;
- саме на прикладі Не зручно демонструвати вплив базисного набору на енергію Full CI, оскільки результат швидко сходиться до межі повного базису (*Complete Basis Set Limit*).

FullCI_basic.py

```

# =====
# FullCI_basic.py – повний CI для молекули H2
# =====

```

```

from pyscf import gto, scf, fci

# Вибір базису
basis="cc-pv5z"

# --- 1. Створюємо молекулу
mol = gto.M(
    atom = "He 0 0 0", # відстань 0.74 Å
    basis=basis,
    verbose = 0
)

# --- 2. Розрахунок Хартрі-Фока
mf = scf.RHF(mol)
E_HF = mf.kernel()

# --- 3. Повний CI
cisolver = fci.FCI(mf)
E_FCI, _ = cisolver.kernel()

# --- 4. Вивід результатів
E_exp = -2.9037243770 # Експериментальне значення (basis set limit)
abs_E_exp = abs(E_exp)
dev_HF = abs(E_HF - E_exp) / abs_E_exp * 100
dev_FCI = abs(E_FCI - E_exp) / abs_E_exp * 100

# Вивід таблиці
print("=" * 60)
header = f"{{: <{12}}} {{: >{14}}} {{: >{12}}}"
print(header.format("Метод", "Енергія (Ha)", "Відхилення (%)"))
print("=" * 60)
row = f"{{: <{12}}} {{: >{14}}.{{8}}f} {{: >{12}}.{{4}}f}"
print(row.format("HF", E_HF, dev_HF))
print(row.format("Full CI", E_FCI, dev_FCI))
print(row.format("Експеримент", E_exp, 0.0000))
print("=" * 60)

```

Коментар

Як видно з виводу програми, навіть метод **Full CI**, який у межах даного базисного набору забезпечує математично точне розв'язання рівняння Шредінгера, не відтворює експериментальну енергію. Причина полягає не в обмеженнях самого CI, а у **неповноті базису**. У малому базисі (наприклад, STO-3G) атомні орбіталі лише грубо апроксимують реальні електронні хвильові функції, що призводить до недооцінки електронної кореляції та деякого завищення енергії (менш від'ємного значення).

Зі збільшенням розміру базису (наприклад, 6-31G, cc-pVTZ, aug-cc-pVQZ) розрахунок Full CI поступово наближається до *межі повного базису* (*complete basis set limit*), де відхилення від експерименту стає мінімальним.

Таким чином, **Full CI є точним лише в межах обраного базису**, але не є абсолютно точним у фізичному сенсі.

5.3.3. CISD розрахунки

Метод CISD (*Configuration Interaction with Singles and Doubles*) є наближенням до Full CI, в якому враховуються лише **одиначні** (S) та **подвійні** (D) збудження відносно детермінанта Гартрі-Фока:

$$|\Psi_{\text{CISD}}\rangle = c_0|\Phi_0\rangle + \sum_i^N \sum_a c_i^a |\Phi_i^a\rangle + \frac{1}{2} \sum_{ij}^N \sum_{ab} c_{ij}^{ab} |\Phi_{ij}^{ab}\rangle \quad (5.11)$$

де $|\Phi_i^a\rangle$ — детермінант із одиночним збудженням електрона з орбіталі i у a , а $|\Phi_{ij}^{ab}\rangle$ — детермінант із подвійним збудженням.

Метод CISD враховує основну частину кореляційної енергії, зокрема короткодійну кулонівську взаємодію між електронами, і є **гарним компромісом** між точністю та обчислювальними витратами.

Проте CISD має фундаментальне обмеження — **він не є розмірно узгодженим (size-consistency)**. Це означає, що для двох незалежних систем A і B виконується:

$$E_{\text{CISD}}(A + B) \neq E_{\text{CISD}}(A) + E_{\text{CISD}}(B)$$

на відміну від методів типу MP2 або FCI.

Фізично це пов'язано з тим, що CISD включає лише збудження з одного детермінанту. Коли система розпадається на незалежні частини, такого опису недостатньо, щоб відтворити всі можливі комбінації збуджень для обох фрагментів. Тобто, Метод CISD покращує енергію зв'язку порівняно з HF, але не відтворює правильну енергію при дисоціації.

Цю властивість можна продемонструвати на прикладі молекули LiH, де видно, що на далекій відстані між атомами енергія CISD є сумою енергій окремих атомів. Для демонстрації використовують такий тест:

LiH_size_consistency.py

```
# =====
# LiH_size_consistency.py – демонстрація size-consistency для LiH дисоціації з експериментом
# =====

from pyscf import gto, scf
from pyscf.fci import FCI
from pyscf.ci import CISD

# Атом Li (UHF для open-shell, spin=1)
mol_li = gto.M(atom="Li 0 0 0", spin=1, basis="sto-3g", verbose=0)
mf_li = scf.UHF(mol_li)
E_hf_li = mf_li.kernel()
fci_li = FCI(mf_li)
E_fci_li = fci_li.kernel()[0]
cisd_li = CISD(mf_li)
E_cisd_li_corr = cisd_li.kernel()[0]

# Атом H (UHF для open-shell, spin=1)
mol_h = gto.M(atom="H 0 0 0", spin=1, basis="sto-3g", verbose=0)
mf_h = scf.UHF(mol_h)
E_hf_h = mf_h.kernel()
# FCI та CISD для H = HF (1 електрон, немає кореляції)

# LiH bound (R=3.0 Bohr ≈ рівноважна, RHF, spin=0)
mol_bound = gto.M(atom="Li 0 0 0; H 0 0 3.0", basis="sto-3g", verbose=0)
mf_bound = scf.RHF(mol_bound)
E_hf_bound = mf_bound.kernel()
fci_bound = FCI(mf_bound)
```

```

E_fci_bound = fci_bound.kernel()[0]
cisd_bound = CISD(mf_bound)
E_cisd_bound_corr = cisd_bound.kernel()[0]

# LiH dissociated (R=10, RHF – для демонстрації проблеми)
mol_diss = gto.M(atom="Li 0 0 0; H 0 0 20", basis="sto-3g", verbose=0)
mf_diss = scf.RHF(mol_diss)
E_hf_diss = mf_diss.kernel()
fci_diss = FCI(mf_diss)
E_fci_diss = fci_diss.kernel()[0]
cisd_diss = CISD(mf_diss)
E_cisd_diss_corr = cisd_diss.kernel()[0]

# Li + H (граничний стан)
E_2atoms_hf = E_hf_li + E_hf_h
E_2atoms_fci = E_fci_li + E_hf_h # H exact
E_2atoms_cisd = (E_hf_li + E_cisd_li_corr) + E_hf_h # Li correlation

# Експериментальні значення (приблизно для R_eq ≈ 3.0 Bohr)
E_exp_bound = -8.070 # Експериментальна енергія зв'язаної LiH (Ha)
E_exp_diss = -7.978 # E(Li) + E(H) (Ha)

# Повні енергії CISD
E_cisd_bound = E_hf_bound + E_cisd_bound_corr
E_cisd_diss = E_hf_diss + E_cisd_diss_corr

# Помилки size-consistency
diff_hf = E_hf_diss - E_2atoms_hf
diff_fci = E_fci_diss - E_2atoms_fci
diff_cisd = E_cisd_diss - E_2atoms_cisd
diff_exp = 0 # ≈ 0

print("="*65)
print("Метод      | LiH bound | LiH diss | Li + H   | Error (diss - 2atoms)")
print("="*65)
print(f"HF           | {E_hf_bound:8.5f} | {E_hf_diss:8.5f} | {E_2atoms_hf:8.5f} | |")
print(f"↳ {diff_hf:8.5f}")
print(f"CISD        | {E_cisd_bound:8.5f} | {E_cisd_diss:8.5f} | {E_2atoms_cisd:8.5f} | |")
print(f"↳ {diff_cisd:8.5f}")
print(f"Full CI     | {E_fci_bound:8.5f} | {E_fci_diss:8.5f} | {E_2atoms_fci:8.5f} | |")
print(f"↳ {diff_fci:8.5f}")
print(f"Експеримент | {E_exp_bound:8.5f} | {E_exp_diss:8.5f} | {E_exp_diss:8.5f} | |")
print(f"↳ {diff_exp:8.5f}")
print("="*65)

```

Результати демонструють порушення розмірної узгодженості:

```

=====
Метод      | LiH bound | LiH diss | Li + H   | Error (diss - 2atoms)
=====
HF          | -7.71083  | -7.23965 | -7.78211 | 0.54246
CISD        | -7.79875  | -7.28025 | -7.78242 | 0.50217
Full CI     | -7.79884  | -7.78242 | -7.78242 | -0.00000
Експеримент | -8.07000  | -7.97800 | -7.97800 | 0.00000
=====

```

Видно, що при дисоціації молекули LiH енергія, обчислена в межах CISD, не збігається з сумою енергій окремих атомів Li та H. У наведеному прикладі похибка становить близько 0.5 Ha, що є суттєвим відхиленням. Для

порівняння, повний CI (FCI) повністю узгоджений із фізичною картиною — похибка практично нульова.

Цей приклад особливо зручний для демонстрації залежності точності FCI та наближених методів (як CISD) від базисного набору: зі збільшенням розмірності базису відхилення CISD зменшується, проте ніколи не зникає повністю, оскільки сама природа методу не гарантує розмірної узгодженості.

5.4. Coupled Cluster методи (CCSD, CCSD(T))

5.4.1. Теоретичні основи Coupled Cluster

Coupled Cluster (CC) — один з найточніших і найстійкіших пост-Хартрі-Фоківських методів, який систематично враховує електронну кореляцію. На відміну від конфігураційної взаємодії (CI), методи CC є **size-extensive**, тобто правильно масштабується з розміром системи: енергія двох незалежних фрагментів дорівнює сумі енергій кожного з них.

Хвильова функція записується у вигляді експоненти від оператора збудження:

$$|\Psi_{CC}\rangle = e^{\hat{T}}|\Phi_0\rangle \quad (5.12)$$

де $|\Phi_0\rangle$ — HF-детермінант, а оператор кластерів $\hat{T}$ розкладається як:

$$\hat{T} = \hat{T}_1 + \hat{T}_2 + \hat{T}_3 + \dots \quad (5.13)$$

На відміну від CI, де хвильова функція є *лінійною комбінацією* детермінантів, експоненціальна форма $e^{\hat{T}}$ генерує нескінченний ряд ефективних збуджень, що автоматично враховує взаємодію між ними та забезпечує правильну масштабованість енергії (size-consistency).

Доведення size-consistency для Coupled Cluster. Розглянемо дві невзаємодіючі підсистеми A і B , для яких гамільтоніан має вигляд:

$$\hat{H} = \hat{H}_A + \hat{H}_B.$$

Основний (референсний) детермінант системи є добутком детермінантів підсистем:

$$|\Phi_0\rangle = |\Phi_A\rangle \otimes |\Phi_B\rangle.$$

У методі Coupled Cluster хвильова функція має експоненціальну форму:

$$|\Psi_{CC}\rangle = e^{\hat{T}}|\Phi_0\rangle,$$

де кластерний оператор розкладається як

$$\hat{T} = \hat{T}_A + \hat{T}_B.$$

Оскільки $\hat{T}_A$ і $\hat{T}_B$ діють у різних підпросторах (на різні набори орбіталей), вони комутують:

$$[\hat{T}_A, \hat{T}_B] = 0.$$

Отже, експонента факторизується:

$$e^{\hat{T}} = e^{\hat{T}_A + \hat{T}_B} = e^{\hat{T}_A} e^{\hat{T}_B}.$$

Енергія СС визначається як середнє значення подібно перетвореного гамільтоніана:

$$E_{CC} = \langle \Phi_0 | \bar{H} | \Phi_0 \rangle, \quad \bar{H} = e^{-\hat{T}} \hat{H} e^{\hat{T}}.$$

Підставляючи $\hat{H} = \hat{H}_A + \hat{H}_B$ і $\hat{T} = \hat{T}_A + \hat{T}_B$, маємо:

$$\bar{H} = e^{-(\hat{T}_A + \hat{T}_B)} (\hat{H}_A + \hat{H}_B) e^{\hat{T}_A + \hat{T}_B} = e^{-\hat{T}_A} \hat{H}_A e^{\hat{T}_A} + e^{-\hat{T}_B} \hat{H}_B e^{\hat{T}_B},$$

оскільки перехресні члени, що містять одночасно $\hat{H}_A$ та $\hat{T}_B$, зникають (оператори діють у різних підпросторах і комутують).

Отже, повна енергія системи розкладається як:

$$E_{CC} = \langle \Phi_A \Phi_B | (e^{-\hat{T}_A} \hat{H}_A e^{\hat{T}_A} + e^{-\hat{T}_B} \hat{H}_B e^{\hat{T}_B}) | \Phi_A \Phi_B \rangle = E_{CC}(A) + E_{CC}(B).$$

Таким чином, метод Coupled Cluster є **size-consistent**: енергія двох незалежних систем дорівнює сумі енергій кожної з них окремо.

Коментар

Розмірна узгодженість (size-consistency) є необхідною властивістю для коректного опису дисоціаційної границі молекули. Однак навіть розмірно узгоджені методи (наприклад, CCSD) не гарантують фізично правильної поведінки потенціальної кривої при розриві зв'язку. Причина полягає у руйнуванні одностерміантного опису — хвильова функція стає багатоконфігураційною, а референс Хартрі-Фока перестає бути адекватним. У таких випадках необхідно застосовувати мультідетерміантні методи (CASSCF, MRCI), або використовувати UCCSD для часткової корекції ефектів спінового розшарування.

CCSD (Coupled Cluster Singles and Doubles). Враховує лише одну-та двозбуджені конфігурації:

$$\hat{T}_1 = \sum_i^{\text{occ}} \sum_a^{\text{virt}} t_i^a \hat{a}_a^\dagger \hat{a}_i \quad (5.14)$$

$$\hat{T}_2 = \frac{1}{4} \sum_{ij}^{\text{occ}} \sum_{ab}^{\text{virt}} t_{ij}^{ab} \hat{a}_a^\dagger \hat{a}_b^\dagger \hat{a}_j \hat{a}_i \quad (5.15)$$

CCSD(T) — «золотий стандарт». Для ще точнішого врахування кореляції додають пертурбативну поправку від потрібних збуджень:

$$E_{CCSD(T)} = E_{CCSD} + E_{(T)} \quad (5.16)$$

де $E_{(T)}$ визначається за формулами четвертого порядку теорії збурень (MBPT4). Метод CCSD(T) поєднує високу точність з помірною обчислювальною складністю $O(N^7)$ і тому вважається **еталоном точності** у квантовій хімії.

Експоненційна форма хвильової функції забезпечує:

- **Size-extensivity:** на відміну від CISD, енергія двох незалежних систем у CCSD дорівнює сумі енергій кожної з них;
- **Автоматичне включення ефективних багатоелектронних збуджень:** хоча $\hat{T}$ містить лише T_1 і T_2 , експонента $e^{\hat{T}}$ генерує вищі збудження ($T_3, T_4, \dots$);
- **Високу стабільність** при обчисленнях навіть для систем із помірним багатоконфігураційним характером.

5.4.2. CCSD розрахунок атома Гелію

`c He_CCSD.py`

Для двоелектронних систем (He) CCSD дає точний результат у межах будь-якого базису, оскільки всі кореляційні ефекти описуються лише T_2 -оператором. Цей приклад демонструє, що CCSD повністю узгоджується з Full CI для таких систем.

5.4.3. CCSD для відкритих оболонок (UCCSD)

`c code_9.py`

Для систем із неспареними електронами використовується варіант **UCCSD (Unrestricted CCSD)**. Він будується на UHF-детермінанті, дозволяючи різні орбіталі для α - та β -спінів. UCCSD зберігає всі основні переваги CCSD і забезпечує адекватний опис навіть для радикалів та іонів.

5.4.4. Перевірка size-extensivity: дисоціація LiH

`c LiH_CCSD_size.py`

Порівняння з CISD демонструє ключову відмінність CCSD: для повністю розірваної молекули LiH (великі між'ядерні відстані) енергія **CISD** не дорівнює сумі енергій окремих атомів (*порушення size-consistency*), тоді як **CCSD** дає правильний результат:

$$E_{CCSD}(\text{LiH}_{\text{diss}}) \approx E_{CCSD}(\text{Li}) + E_{CCSD}(\text{H})$$

Це одна з найважливіших переваг CC над CI при моделюванні хімічних реакцій і дисоціацій.

5.4.5. Порівняння ієрархії методів

`c code_10.py`

Зі зростанням складності методів ($\text{HF} \rightarrow \text{MP2} \rightarrow \text{CISD} \rightarrow \text{CCSD} \rightarrow \text{CCSD(T)} \rightarrow \text{FCI}$) енергія системи послідовно наближається до експериментальної. Метод **CCSD(T)** зазвичай відхиляється від експерименту не більш ніж на кілька мілігартрі, що робить його практичним компромісом між точністю та витратами.

5.4.6. T_1 -діагностика та багатоконфігураційний характер

T_1 -діагностика є простим індикатором того, наскільки система є одноконфігураційною. Вона базується на амплітудах одноелектронних збуджень t_1 :

$$T_1 = \frac{\|t_1\|}{\sqrt{N_{\text{elec}}}} \quad (5.17)$$

`c code_11.py`

Типові значення:

- $T_1 < 0.02$ — система добре описується одноконфігураційним наближенням ($\text{HF} \Rightarrow \text{CCSD}$ точний);
- $0.02 < T_1 < 0.045$ — слабка багатоконфігураційність, **CCSD** залишається надійним;
- $T_1 > 0.045$ — сильний багатоконфігураційний характер, **CCSD(T)** може втрачати точність, слід застосовувати **MCSCF** або **MR-CC**.

На прикладі LiH : при рівноважній відстані $T_1 \approx 0.015$, що підтверджує одноконфігураційний характер системи. Під час розтягування зв'язку ($R \rightarrow \infty$) T_1 зростає, що вказує на зростання багатоконфігураційності — саме там **CCSD** перестає бути повністю точним. Таким чином, T_1 є зручним індикатором придатності методів **CC** для конкретної системи.

5.5. CASSCF — багатоконфігураційний метод

5.5.1. Ідея Complete Active Space Self-Consistent Field

CASSCF (Complete Active Space SCF) — багатоконфігураційний метод, який поєднує:

- **Активний простір** — набір орбіталей, де проводиться Full CI
- **SCF оптимізацію** — орбіталі та CI коефіцієнти оптимізуються одночасно

Орбіталі розділяються на три групи:

1. **Core (остов)** — завжди подвійно заповнені, не беруть участь у кореляції
2. **Active (активні)** — електрони розподілені по всіх можливих способах (Full CI)

3. Virtual (віртуальні) — завжди порожні

$$|\Psi_{\text{CASSCF}}\rangle = \sum_I c_I |\Phi_I^{\text{active}}\rangle \otimes |\text{core}\rangle \quad (5.18)$$

5.5.2. Позначення CASSCF(n,m)

CASSCF(n,m) означає:

- n — кількість електронів в активному просторі
- m — кількість орбіталей в активному просторі

Приклади:

- **Be:** CASSCF(4,8) — 4 валентні електрони у 8 орбіталях (2s, 2p, 3s, 3p)
- **C:** CASSCF(4,4) — 4 валентні електрони у 4 орбіталях (2s, 2p)
- **Cr:** CASSCF(6,5) — 6 електронів у 5 d-орбіталях

5.5.3. CASSCF розрахунок атома Берилію

Берилій має близькі за енергією конфігурації $2s^2$ та $2p^2$, що робить його класичним прикладом для CASSCF:

`c CASSCFBe.py`

5.5.4. CASSCF для перехідних металів

Для перехідних металів d-орбіталі часто близькі за енергією, потребуючи багатоконфігураційного підходу:

`c CASSCFTransitionMetals.py`

5.5.5. Вибір активного простору

Правильний вибір активного простору критичний для CASSCF:

Таблиця 5.3. Рекомендації по вибору активного простору

Система	Активний простір	Коментар
Be	(4,8): 2s,2p,3s,3p	Враховує $2s^2 \leftrightarrow 2p^2$
C	(4,4): 2s,2p	Мінімальний валентний
O	(6,6): 2s,2p + корел.	З кореляційними орбіталями
3d метали	(n,5): 3d	Тільки d-орбіталі
3d метали	(n+2,6): 3d,4s	d + s орбіталі
Лантаноїди	(n,7): 4f	f-орбіталі

Принципи вибору:

1. Включити орбіталі, близькі за енергією
2. Включити орбіталі з значною хімічною активністю

3. Більший простір $\rightarrow$ точніше, але експоненційно повільніше
4. Перевірити заселеності природних орбіталей
5. Якщо заселеності ≈ 2 або ≈ 0 , можна виключити з активного простору

5.5.6. State-averaged CASSCF

Для розрахунку декількох станів одночасно (наприклад, різних мультиплетів):

`c StateAvaragedCASSCF.py`

5.5.7. CASPT2 — динамічна кореляція поверх CASSCF

CASSCF враховує тільки статичну кореляцію. Для повної точності потрібно додати динамічну кореляцію через CASPT2:

`c CASPT2.py`

5.5.8. Переваги та недоліки CASSCF

Переваги:

- Правильно описує статичну кореляцію
- Може розрахувати декілька станів одночасно
- Варіаційний метод
- Підходить для розриву зв'язків, збуджених станів
- Дає природні орбіталі та заселеності

Недоліки:

- Вибір активного простору не завжди очевидний
- Експоненційне масштабування з розміром активного простору
- Не враховує динамічну кореляцію (потрібен CASPT2)
- Може бути проблеми з конвергенцією
- Потребує досвіду для правильного використання

Коли використовувати CASSCF:

- T1 діагностика > 0.05 (сильна багатоконфігураційність)
- Перехідні метали з близькими d-орбіталями
- Збуджені стани атомів
- Діелектронні системи
- Відкрито-оболонкові синглети

5.6. Практичні завдання

Завдання 1: Порівняння методів для He

"""

ЗАВДАННЯ 1: Для атома Гелію розрахуйте енергії методами HF, MP2, CCSD, CCSD(T) та FCI з базисами cc-pVDZ, cc-pVTZ, cc-pVQZ.

- 1) Побудуйте графік збіжності кожного методу до базисної межі
- 2) Екстраполуйте до CBS за формулою $E(X) = E_{\text{CBS}} + A/X^3$
- 3) Порівняйте з експериментальним значенням -2.90372 Ha
- 4) Яка частка кореляційної енергії відновлюється кожним методом?

Базиси: cc-pVDZ, cc-pVTZ, cc-pVQZ

"""

Ваш код тут

Завдання 2: T1 діагностика для другого періоду

"""

ЗАВДАННЯ 2: Розрахуйте T1 діагностику для всіх атомів другого періоду (Li-Ne) з базисом cc-pVDZ.

- 1) Для яких атомів $T1 > 0.02$ (багатоконфігураційні)?
- 2) Чи є кореляція між T1 та положенням у періоді?
- 3) Порівняйте T1 для основного та збудженого спінового стану Карбону (3P vs 1D)
- 4) Побудуйте графік залежності T1 від атомного номера

Базис: cc-pVDZ

"""

Ваш код тут

Завдання 3: Size-extensivity тест

"""

ЗАВДАННЯ 3: Дослідіть size-extensivity для атомів Ne.

Розрахуйте методами HF, MP2, CISD, CCSD:

- 1) Енергію одного атома Ne
- 2) Енергію двох атомів Ne на відстані 100 Bohr
- 3) Обчисліть похибку: $E(2 \times \text{Ne}) - 2 \times E(\text{Ne})$

Питання:

- Який метод є size-extensive?
- Яка похибка для CISD (у mHa)?
- Чому MP2 є size-extensive, а CISD - ні?

Базис: cc-pVTZ

"""

Ваш код тут

Завдання 4: CASSCF для перехідного металу

```

"""
ЗАВДАННЯ 4: CASSCF аналіз атома Титану (Ti).

Ti: [Ar] 3d2 4s2, основний стан 3F

1) Розрахуйте UHF, CCSD та CASSCF(4,5) енергії
   (4 електрони у 5 d-орбіталах)
2) Проаналізуйте заселеності природних орбіталей
3) Обчисліть статичну кореляцію (CASSCF - HF)
4) Спробуйте різні активні простори: (4,5), (4,6), (6,11)
   Який найкращий?

Базис: def2-SVP
"""

# Ваш код тут

```

Завдання 5: Енергії збудження

```

"""
ЗАВДАННЯ 5: Розрахуйте енергію збудження 3P → 1D для
атома Карбону.

Використайте методи:
1) ΔSCF (окремі розрахунки для кожного стану)
2) State-averaged CASSCF
3) EOM-CCSD (якщо доступно)

Порівняйте з експериментальним значенням 1.26 eV.

Який метод дає найточніший результат?

Базис: aug-cc-pVTZ
"""

# Ваш код тут

```

Завдання 6: Кореляційна енергія vs Z

```

"""
ЗАВДАННЯ 6: Дослідіть залежність кореляційної енергії
від атомного номера.

Розрахуйте MP2 кореляційну енергію для:
- Благородних газів: He, Ne, Ar, Kr
- У абсолютних величинах (mHa)
- У відсотках від повної енергії

Побудуйте графіки:
1) |E_corr| vs Z
2) |E_corr|/|E_HF| vs Z

Яка тенденція спостерігається?

Базис: cc-pVTZ

```

"""

Ваш код тут

5.7. Резюме

У цьому розділі ми детально вивчили Post-Hartree-Fock методи для врахування електронної кореляції:

5.7.1. Основні методи та їх характеристики

Таблиця 5.4. Порівняння Post-HF методів

Метод	Складність	Size-cons.	Варіац.	Точність	БК?
MP2	$O(N^5)$	Так	Ні	Помірна	Ні
MP3/MP4	$O(N^6/N^7)$	Так	Ні	Добра	Ні
CISD	$O(N^6)$	Ні	Так	Помірна	Ні
CCSD	$O(N^6)$	Так	Ні	Добра	Ні
CCSD(T)	$O(N^7)$	Так	Ні	Відмінна	Ні
CASSCF	$O(e^{n_{act}})$	Так	Так	Статична	Так
CASPT2	$O(e^{n_{act}})$	Так	Ні	Відмінна	Так
Full CI	$O(e^N)$	Так	Так	Точна	Так

де N — характерний розмір системи (базисних функцій або електронів).

БК = Багатоконфігураційний

5.7.2. Ключові висновки

1. Електронна кореляція

- Становить 1-5% від повної енергії, але критична для точних розрахунків
- Поділяється на динамічну (короткодіючу) та статичну (близькі стани)
- HF і DFT не враховують кореляцію повністю

2. MP2 — найпростіший post-HF метод

- Швидкий ($O(N^5)$), size-extensive
- Добре працює для замкнених оболонок з великим HOMO-LUMO gap
- Відновлює 80-95% кореляційної енергії
- Не виправляє забруднення спіном UHF
- Потребує великих базисів для збіжності

3. CCSD(T) — ”золотий стандарт”

- Найточніший одноконфігураційний метод
- Size-extensive, відновлює >99% кореляції
- Повільний ($O(N^7)$), але надійний
- Виправляє забруднення спіном
- Обмежений розміром системи (до 20-30 атомів)

4. T1 діагностика

- $T1 < 0.02$: одноконфігураційна система (CCSD надійний)
- $0.02 < T1 < 0.05$: слабка багатоконфігураційність
- $T1 > 0.05$: потрібен CASSCF/MRCI

5. CI методи

- Варіаційні (енергія завжди вище точної)
- Full CI — точний у межах базису, але непрактичний
- CISD не є size-extensive (на відміну від CCSD)
- Корисні для аналізу хвильової функції

6. CASSCF — для багатоконфігураційних систем

- Необхідний коли $T1 > 0.05$ або близькі стани
- Вибір активного простору критичний
- Враховує статичну кореляцію
- Потребує CASPT2/NEVPT2 для динамічної кореляції
- Ідеальний для перехідних металів, збуджених станів

5.7.3. Рекомендації по використанню

Таблиця 5.5. Вибір методу залежно від задачі

Задача	Рекомендований метод	Альтернатива
Швидка оцінка кореляції	MP2	DFT
Точні енергії (1 kcal/mol)	CCSD(T)	—
Замкнені оболонки	CCSD(T), MP2	DFT
Відкриті оболонки	UCCSD(T)	ROHF-CCSD
Енергії збудження	EOM-CCSD, CASSCF	TD-DFT
Перехідні метали	CASSCF, CASPT2	DFT+U
Багатоконфігураційні	CASSCF/CASPT2	MRCI
Еталонні розрахунки	CCSD(T)/CBS	Full CI
Великі системи	MP2, DFT	DLPNO-CCSD(T)

5.7.4. Типові помилки

1. **Використання MP2 для систем з малим gap** — може давати нефізичні результати
2. **Забування перевірки T1 діагностики** — CCSD може бути ненадійним
3. **Занадто малий активний простір у CASSCF** — не врахує всю статичну кореляцію
4. **Занадто великий активний простір** — неможливо розрахувати
5. **Недостатній базис** — post-HF методи дуже чутливі до базису
6. **Ігнорування size-extensivity** — CISD не підходить для енергій дисоціації
7. **CASSCF без CASPT2** — відсутня динамічна кореляція

5.7.5. Практичні поради

1. Завжди починайте з HF розрахунку та перевірте конвергенцію
2. Для нових систем спочатку протестуйте на малому базисі
3. Перевіряйте T1 діагностику після CCSD
4. Для CASSCF: починайте з малого активного простору, збільшуйте поступово
5. Аналізуйте природні орбіталі для вибору активного простору
6. Використовуйте симетрію коли можливо
7. Для екстраполяції до CBS потрібні принаймні 3 базиси
8. Зберігайте проміжні результати (checkpoint файли)

Примітка: реальний час залежить від якості початкового наближення та конвергенції

5.7.6. Корисні посилання

- <https://pyscf.org/user/mp.html> — MP2, MP3
- <https://pyscf.org/user/cc.html> — Coupled Cluster
- <https://pyscf.org/user/mcscf.html> — CASSCF, CASPT2
- <https://pyscf.org/user/fci.html> — Full CI

6

Теорія функціоналу густини (DFT)

Розглянуті вище пост-Хартрі–Фоківські методи показують, як поступово можна вдосконалювати апроксимацію Хартрі–Фока, враховуючи електронну кореляцію шляхом розширення хвильової функції. Однак усі ці підходи залишаються в межах *хвильово-функціональної парадигми*, що швидко стає обчислювально невідомою для великих систем.

У спробі знайти простішу, але фізично змістовну альтернативу було запропоновано іншу концепцію — опис електронної системи через *густину електронів* $\rho(\mathbf{r})$, що залежить лише від трьох просторових координат. Історично її витоки сягають **моделі Томаса–Фермі**, яка вперше пов’язала енергію електронного газу з його густиною. Хоч ця модель була грубою, саме з неї виросла сучасна **теорія функціонала густини (DFT)**, що поєднує фізичну інтуїцію з математичною строгістю і стала одним із найпотужніших інструментів сучасної квантової хімії.

Ідея моделі Томаса–Фермі полягає в тому, що енергію системи можна виразити як функціонал від густини:

$$E[\rho] = T_{\text{TF}}[\rho] + \int v_{\text{ext}}(\mathbf{r}) \rho(\mathbf{r}) d\mathbf{r} + \frac{1}{2} \iint \frac{\rho(\mathbf{r}) \rho(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r} d\mathbf{r}',$$

де $T_{\text{TF}}[\rho]$ — наближення до кінетичної енергії, отримане для однорідного електронного газу. Мінімізація цього функціонала за $\rho(\mathbf{r})$ при фіксованій кількості електронів

$$\int \rho(\mathbf{r}) d\mathbf{r} = N$$

дає рівноважний розподіл густини для основного стану.

Попри свою простоту, модель Томаса–Фермі стала першим кроком до *безхвильового* опису багаточастинкових систем. Вона виявилася якісно правильною для дуже великих атомних номерів, але не могла відтворювати хімічний зв’язок, тонку структуру електронної густини та ефекти обміну й кореляції.

Саме спроби подолати ці обмеження призвели до народження **теорії функціонала густини (DFT)**. На відміну від моделі Томаса–Фермі, DFT базується на строгих теоремах Гогенберга–Кона, які гарантують існування

точного функціонала енергії від густини та формують основу сучасних обчислювальних методів квантової хімії.

6.1. Основи теорії функціоналу густини

6.1.1. Теореми Хоенберга–Кона

Теорія функціоналу густини базується на двох фундаментальних теоремах, доведених Хоенбергом та Коном у 1964 році:

Теорема 1 (Теорема існування). Зовнішній потенціал $v_{\text{ext}}(\mathbf{r})$ (а отже, і повна енергія системи) однозначно визначається електронною густиною основного стану $\rho(\mathbf{r})$ з точністю до адитивної константи.

Це означає, що електронна густина містить всю інформацію про систему:

$$E[\rho] = T[\rho] + V_{ee}[\rho] + \int v_{\text{ext}}(\mathbf{r})\rho(\mathbf{r})d\mathbf{r} \quad (6.1)$$

Теорема 2 (Варіаційний принцип). Існує універсальний функціонал енергії $F[\rho]$, який для будь-якої пробної густини $\tilde{\rho}(\mathbf{r})$ задовольняє:

$$E_0 \leq E[\tilde{\rho}] = F[\tilde{\rho}] + \int v_{\text{ext}}(\mathbf{r})\tilde{\rho}(\mathbf{r})d\mathbf{r} \quad (6.2)$$

де E_0 — точна енергія основного стану.

6.1.2. Рівняння Кона–Шема

Кон та Шем (1965) запропонували практичний підхід до DFT, замінивши взаємодіючу систему еквівалентною невзаємодіючою системою з такою ж густиною:

$$\left[-\frac{1}{2}\nabla^2 + v_{\text{eff}}(\mathbf{r}) \right] \psi_i(\mathbf{r}) = \varepsilon_i \psi_i(\mathbf{r}) \quad (6.3)$$

Ефективний потенціал визначається як:

$$v_{\text{eff}}(\mathbf{r}) = v_{\text{ext}}(\mathbf{r}) + v_H(\mathbf{r}) + v_{xc}(\mathbf{r}) \quad (6.4)$$

де:

- $v_{\text{ext}}(\mathbf{r})$ — зовнішній потенціал (від ядер)
- $v_H(\mathbf{r}) = \int \frac{\rho(\mathbf{r}')}{\|\mathbf{r}-\mathbf{r}'\|} d\mathbf{r}'$ — потенціал Хартрі
- $v_{xc}(\mathbf{r}) = \frac{\delta E_{xc}[\rho]}{\delta \rho(\mathbf{r})}$ — обмінно-кореляційний потенціал

Електронна густина обчислюється через орбіталі Кона–Шема:

$$\rho(\mathbf{r}) = \sum_{i=1}^N |\psi_i(\mathbf{r})|^2 \quad (6.5)$$

6.1.3. Обмінно-кореляційна енергія

Повна енергія в DFT:

$$E[\rho] = T_s[\rho] + V_{ext}[\rho] + J[\rho] + E_{xc}[\rho] \quad (6.6)$$

де $E_{xc}[\rho]$ — обмінно-кореляційна енергія, яка містить:

- Різницю між точною кінетичною енергією та кінетичною енергією невзаємодіючої системи
- Некласичну частину електрон-електронного відштовхування (обмін + кореляція)

Точний вигляд $E_{xc}[\rho]$ невідомий, тому використовуються наближення (функціонали).

6.1.4. Порівняння HF та DFT

Рис. 6.1. Порівняння методів Хартрі-Фока та DFT

Аспект	Hartree-Fock	DFT
Базова змінна	Хвильова функція	Електронна густина
Обмін	Точний (нелокальний)	Наближений (локальний)
Кореляція	Відсутня	Включена наближено
Масштабування	$\mathcal{O}(N^4)$	$\mathcal{O}(N^3)$
Точність для атомів	Добра якісно	Часто краща кількісно
Збуджені стани	Можливі	Складно (TD-DFT)

6.2. Функціонали обміну-кореляції

6.2.1. Класифікація функціоналів: драбина Якова

Функціонали DFT класифікуються за "драбиною Якова" (Jacob's ladder), запропонованою Пердью:

1. **LDA/LSDA** (Local Density Approximation) — залежать тільки від $\rho(\mathbf{r})$
2. **GGA** (Generalized Gradient Approximation) — залежать від $\rho(\mathbf{r})$ та $\nabla\rho(\mathbf{r})$
3. **Meta-GGA** — додатково залежать від $\nabla^2\rho$ або кінетичної густини τ
4. **Hybrid** — включають частку точного обміну HF
5. **Double-hybrid** — включають і HF обмін, і MP2 кореляцію

6.2.2. LDA та LSDA функціонали

Локальне наближення густини

LDA базується на моделі однорідного електронного газу:

$$E_{xc}^{\text{LDA}}[\rho] = \int \rho(\mathbf{r}) \varepsilon_{xc}(\rho(\mathbf{r})) d\mathbf{r} \quad (6.7)$$

де $\varepsilon_{xc}(\rho)$ — обмінно-кореляційна енергія на електрон в однорідному газі густини ρ .

Обмінна частина (Slater/Dirac):

$$\varepsilon_x^{\text{LDA}}(\rho) = -\frac{3}{4} \left(\frac{3}{\pi} \right)^{1/3} \rho^{1/3} \quad (6.8)$$

Кореляційна частина: Використовуються параметризації (VWN, PW92).

LSDA для спін-поляризованих систем

Для систем з різними ρ_α та ρ_β :

$$E_{xc}^{\text{LSDA}}[\rho_\alpha, \rho_\beta] = \int \rho \varepsilon_{xc}(\rho_\alpha, \rho_\beta) d\mathbf{r} \quad (6.9)$$

`c code_1.py`

6.2.3. GGA функціонали

GGA функціонали враховують не тільки локальну густину, але й її градієнт:

$$E_{xc}^{\text{GGA}}[\rho] = \int f(\rho(\mathbf{r}), \nabla \rho(\mathbf{r})) d\mathbf{r} \quad (6.10)$$

Популярні GGA функціонали

PBE (Perdew-Burke-Ernzerhof, 1996) Найпопулярніший неемпіричний GGA функціонал:

`c code_2.py`

BLYP (Becke88 + Lee-Yang-Parr) Комбінація обміну Becke88 та кореляції LYP:

`c code_3.py`

BP86 (Becke88 + Perdew86) `c code_4.py`

Порівняння GGA функціоналів

`c code_5.py`

6.2.4. Meta-GGA функціонали

Meta-GGA функціонали включають додаткову інформацію про систему: кінетичну густину τ або лапласіан густини $\nabla^2\rho$:

$$E_{xc}^{\text{meta-GGA}}[\rho] = \int f(\rho, \nabla\rho, \tau) d\mathbf{r} \quad (6.11)$$

де кінетична густина орбіталей Кона-Шема:

$$\tau(\mathbf{r}) = \frac{1}{2} \sum_{i=1}^N |\nabla\psi_i(\mathbf{r})|^2 \quad (6.12)$$

TPSS (Tao-Perdew-Staroverov-Scuseria)

TPSS — один з перших успішних meta-GGA функціоналів (2003):

`c code_6.py`

M06-L (Minnesota 06 Local)

Високопараметризований meta-GGA функціонал для широкого спектру задач:

`c code_7.py`

SCAN (Strongly Constrained and Appropriately Normed)

Сучасний meta-GGA, що задовольняє всі відомі точні умови:

`c code_8.py`

6.2.5. Гібридні функціонали

Гібридні функціонали комбінують DFT обмін з точним (HF) обміном:

$$E_{xc}^{\text{hybrid}} = a \cdot E_x^{\text{HF}} + (1 - a) \cdot E_x^{\text{DFT}} + E_c^{\text{DFT}} \quad (6.13)$$

де a — частка HF обміну (зазвичай 0.2–0.3).

B3LYP (Becke 3-parameter Lee-Yang-Parr)

Найпопулярніший гібридний функціонал у хімії:

$$E_{xc}^{\text{B3LYP}} = E_x^{\text{LDA}} + 0.2(E_x^{\text{HF}} - E_x^{\text{LDA}}) + 0.72 \cdot E_x^{\text{B88}} + 0.81 \cdot E_c^{\text{LYP}} + 0.19 \cdot E_c^{\text{VWN}} \quad (6.14)$$

`c code_9.py`

PBE0 (PBE hybrid)

Неемпіричний гібрид з 25

$$E_{xc}^{PBE0} = 0.25 \cdot E_x^{HF} + 0.75 \cdot E_x^{PBE} + E_c^{PBE} \quad (6.15)$$

`c code_10.py`

CAM-B3LYP (Coulomb-Attenuated Method)

Функціонал з дальньодіючою корекцією (range-separated):

$$\frac{1}{r_{12}} = \frac{\alpha + \beta \cdot \text{erf}(\mu r_{12})}{r_{12}} + \frac{1 - [\alpha + \beta \cdot \text{erf}(\mu r_{12})]}{r_{12}} \quad (6.16)$$

`c code_11.py`

M06-2X та інші Minnesota функціонали

Родина M06 з різною часткою HF обміну:

- M06-L: 0% HF (meta-GGA)
- M06: 27% HF
- M06-2X: 54% HF (для кінетики, слабких взаємодій)
- M06-HF: 100% HF

`c code_12.py`

6.2.6. Порівняння різних рівнів теорії

`c code_13.py`

6.2.7. Вибір функціоналу: рекомендації

Таблиця 6.1. Рекомендовані функціонали для різних задач

Задача	Функціонал	Коментар
Швидкі розрахунки	PBE	Добра точність/швидкість
Енергії атомізації	B3LYP, PBE0	Стандарт у хімії
Перехідні метали	TPSSh, M06	Краще для d-електронів
Слабкі взаємодії	M06-2X, ωB97X-D	З дисперсією
Високоспінові стани	SCAN, TPSSh	Кращий баланс
Загальні розрахунки	PBE0	Універсальний вибір
Максимальна точність	CCSD(T)	Але дуже повільно

6.2.8. Практичний приклад: вплив функціоналу на властивості

`c code_14.py`

6.3. DFT розрахунки атомів

6.3.1. Базова структура DFT розрахунку

DFT розрахунки в PySCF використовують класи **RKS** (Restricted Kohn-Sham) та **UKS** (Unrestricted Kohn-Sham), аналогічні до RHF/UHF:

`c code_15.py`

6.3.2. Систематичний розрахунок атомів другого періоду

`c code_16.py`

6.3.3. Розрахунок атомів перехідних металів

Атоми перехідних металів — складна задача через багато близьких за енергією станів:

`c code_17.py`

6.3.4. Порівняння спінових станів

Для деяких атомів важливо порівняти різні спінові стани:

`c code_18.py`

6.3.5. Аналіз d-орбіталей перехідних металів

`c code_19.py`

6.3.6. Розрахунок важких атомів

Для важких атомів (4d, 5d, 4f, 5f) важливі релятивістські ефекти:

`c code_20.py`

6.3.7. Числові сітки в DFT

Точність DFT розрахунків залежить від якості числової сітки для інтегрування:

`c code_21.py`

6.3.8. Паралелізація DFT розрахунків

`c code_22.py`

6.4. Порівняння HF та DFT результатів

6.4.1. Систематичне порівняння енергій

c code_23.py

6.4.2. Порівняння енергій іонізації

c code_24.py

6.4.3. Електронна спорідненість

c code_25.py

6.4.4. Порівняння орбітальних енергій

c code_26.py

6.4.5. Забруднення спіном: HF vs DFT

c code_27.py

6.4.6. Час обчислень: HF vs DFT

c code_28.py

6.4.7. Загальні висновки HF vs DFT

Таблиця 6.2. Підсумкове порівняння HF та DFT методів

Критерій	Hartree-Fock	DFT
Точність енергій	Добра якісно	Краща кількісно
Енергії іонізації	Систематично завищені	Ближче до експерименту
Електронна спорідненість	Погана для аніонів	Значно краща
Орбітальні енергії	НОМО $\approx$ -IE (теорема Купманса)	Відхилення від теореми
Забруднення спіном	Присутнє (UHF)	Відсутнє (чисті DFT)
Швидкість	Середня	Швидше (чисті DFT) Повільніше (гібриди)
Перехідні метали	Складно конвергує	Зазвичай краще
Дисперсійні взаємодії	Відсутні	Потрібні корекції
Систематичність	Добра	Залежить від функціоналу

Рекомендації:

- Використовуйте **HF** для швидких якісних оцінок та як початок для post-HF методів
- Використовуйте **чисті DFT (PBE)** для великих систем, перехідних металів
- Використовуйте **гібридні DFT (B3LYP, PBE0)** для найкращого балансу точності та швидкості
- Для **аніонів** обов'язково використовуйте дифузні функції (aug-базиси)
- Для **важких атомів** враховуйте релятивістські ефекти

6.5. Вибір функціоналу для атомних систем

6.5.1. Тестовий набір даних

Для оцінки якості функціоналів використаємо стандартні атомні властивості:

`c code_29.py`

6.5.2. Рекомендації для різних елементів

Таблиця 6.3. Рекомендовані функціонали для різних груп елементів

Група елементів	Рекомендований	Альтернатива
H, He	HF, PBE0	Будь-який
Li–Ne (2 період)	PBE0, B3LYP	PBE, ω B97X-D
Na–Ar (3 період)	PBE0, B3LYP	TPSSh
3d метали (Sc–Zn)	TPSSh, M06	PBE, B3LYP
4d метали (Y–Cd)	TPSSh, PBE	M06
5d метали (La–Hg)	PBE, TPSSh	+ релятивістські
Лантаноїди	PBE, SCAN	+ SOC
Актиноїди	PBE	+ SOC + DFT+U

6.6. Практичні завдання

6.6.1. Завдання 1: Систематичне дослідження

`c code_30.py`

6.6.2. Завдання 2: Функціональна залежність

`c code_31.py`

6.6.3. Завдання 3: Конвергенція до базисної межі

`c code_32.py`

6.6.4. Завдання 4: Перехідні метали

`c code_33.py`

6.7. Резюме

У цьому розділі ми детально вивчили теорію функціоналу густини та її застосування до атомних систем:

- **Теоретичні основи** — теореми Хоенберга–Кона, рівняння Кона–Шема
- **Функціонали** — від простих LDA до складних гібридних і meta-GGA
- **Практичні розрахунки** — атоми різних періодів, перехідні метали
- **Порівняння з HF** — енергії, орбіталі, спіни, швидкість
- **Вибір методу** — рекомендації для різних задач

6.7.1. Ключові висновки

1. DFT зазвичай дає кращі результати для атомних енергій порівняно з HF
2. Гібридні функціонали (B3LYP, PBE0) — золота середина між точністю та швидкістю
3. Для перехідних металів краще використовувати meta-GGA (TPSS) або спеціалізовані функціонали (M06)
4. Вибір базису критичний: для аніонів потрібні дифузні функції
5. Чисті DFT не мають забруднення спіном на відміну від UHF
6. Якість числової сітки важлива для точних розрахунків
7. Для важких атомів необхідні релятивістські корекції

6.7.2. Типові помилки

1. **Неправильний спін** — завжди перевіряйте основний стан атома
2. **Недостатній базис** — для точних енергій використовуйте triple-zeta або більше
3. **Забування дифузних функцій** — критично для аніонів та збуджених станів
4. **Ігнорування симетрії** — може уповільнити розрахунок
5. **Погана конвергенція** — використовуйте level shift, змініть початкове наближення
6. **Неправильна сітка** — для точних результатів використовуйте `grids.level  $\geq 3$` .

6.7.3. Корисні посилання

- **Libxc** — бібліотека DFT функціоналів: <https://www.tddft.org/programs/libxc/>
- **NIST** — експериментальні дані атомів: <https://physics.nist.gov/PhysRefData/>
- **Basis Set Exchange** — база даних базисних наборів: <https://www.basissetexchange.org/>

6.8. DFT для діатомних молекул

Після вивчення атомних систем переходимо до молекул. Діатомні молекули — природний наступний крок, що дозволяє досліджувати хімічні зв'язки, потенційні криві та спектроскопічні властивості.

6.8.1. Базові DFT розрахунки молекул

Основна структура DFT розрахунку молекули аналогічна до атомної:

```
c code_34.py
```

6.8.2. Оптимізація геометрії

Для знаходження рівноважної структури молекули використовується оптимізація геометрії:

```
c code_35.py
```

6.8.3. Криві потенційної енергії

Побудова кривої потенційної енергії (PES) — фундаментальна характеристика хімічного зв'язку:

```
c code_36.py
```

6.8.4. Порівняння функціоналів для зв'язків

Різні функціонали по-різному описують хімічні зв'язки:

```
c code_37.py
```

6.8.5. Спектроскопічні константи

З кривої потенційної енергії можна отримати спектроскопічні константи:

- r_e — рівноважна довжина зв'язку
- D_e — енергія дисоціації
- ω_e — частота коливань
- $\omega_e x_e$ — ангармонічність

```
c code_38.py
```

6.8.6. Гетероядерні молекули

Гетероядерні молекули мають полярний зв'язок та дипольний момент:

`c code_39.py`

6.8.7. Молекули з кратними зв'язками

Дослідження молекул з подвійними та потрійними зв'язками:

`c code_40.py`

6.8.8. Порівняння HF та DFT для молекул

`c code_41.py`

6.8.9. Молекули перехідних металів

Диатомні молекули перехідних металів — складні багатоконфігураційні системи:

`c code_42.py`

6.8.10. Відкритошкаралупні молекули

Молекули з непарними електронами потребують UKS підходу:

`c code_43.py`

6.8.11. Дисперсійні корекції для слабких зв'язків

Стандартні DFT функціонали погано описують дисперсійні взаємодії. Потрібні корекції:

`c code_44.py`

6.8.12. Аналіз хвильових функцій

Детальний аналіз електронної структури молекул:

`c code_45.py`

6.8.13. Вплив базисного набору

Систематичне дослідження конвергенції до базисної межі:

`c code_46.py`

6.8.14. Дисоціація зв'язків

Проблема симетричного розриву зв'язку

Restricted методи (RKS) неправильно описують дисоціацію гомолітичного зв'язку:

[с code_47.py](#)

6.8.15. Збуджені стани: TD-DFT

Time-Dependent DFT для вертикальних збуджень:

[с code_48.py](#)

6.8.16. Порівняльна таблиця результатів

Таблиця 6.4. Порівняння методів для типових діатомних молекул

Молекула	Експери- мент	HF	PBE	B3LYP	PBE0
H ₂ r_e (Å)	0.741	0.735	0.748	0.744	0.742
H ₂ D_e (eV)	4.75	3.64	4.52	4.71	4.68
N ₂ r_e (Å)	1.098	1.065	1.104	1.098	1.091
N ₂ D_e (eV)	9.91	4.89	9.23	9.85	9.67
O ₂ r_e (Å)	1.208	1.162	1.229	1.216	1.208
CO r_e (Å)	1.128	1.097	1.137	1.130	1.124

6.8.17. Практичні рекомендації

- Для простих молекул (H₂, N₂, O₂):
 - Базовий розрахунок: PBE/cc-pVTZ
 - Висока точність: PBE0/aug-cc-pVQZ
 - Найкраща точність: CCSD(T)/aug-cc-pV5Z
- Для полярних молекул (CO, NO, HF):
 - Гібридні функціонали обов'язкові
 - Дифузні функції для дипольних моментів
- Для молекул перехідних металів:
 - TPSSh або M06 функціонали
 - Велика кількість спінових станів
 - Можливо, потрібен DFT+U
- Для слабких зв'язків:
 - Обов'язкові дисперсійні корекції (D3, D4)
 - Або ω B97X-D, M06-2X функціонали
- Для збуджених станів:
 - TD-DFT з range-separated функціоналами
 - Або багатоконфігураційні методи (CASSCF)

6.8.18. Типові помилки при розрахунках молекул

1. Неправильна симетрія:

- Завжди використовуйте симетрію молекули
- Перевіряйте point group командою `mol.symmetry = True`

2. Хибна мультиплетність:

- O_2 — триплет, не синглет!
- Перевіряйте спин по `mol.spin`

3. Недостатня сітка інтегрування:

- Для точних градієнтів: `grids.level = 4`
- Для оптимізації завжди перевіряйте конвергенцію по сітці

4. Погана початкова геометрія:

- Починайте з розумних відстаней
- Використовуйте експериментальні дані або попередні розрахунки

5. Ігнорування BSSE (Basis Set Superposition Error):

- Для енергій зв'язування використовуйте counterpoise корекцію
- Особливо важливо для малих базисів

6.8.19. Задачі для самостійної роботи

1. **Задача 1:** Побудуйте криві потенційної енергії молекули F_2 різними функціоналами (LDA, PBE, B3LYP, PBE0) та порівняйте з експериментом ($r_e = 1.412 \text{ \AA}$, $D_e = 1.66 \text{ eV}$).
2. **Задача 2:** Дослідіть вплив базисного набору на властивості CO: розрахуйте r_e , D_e , ω_e з базисами cc-pVDZ, cc-pVTZ, cc-pVQZ та екстраполуйте до межі.
3. **Задача 3:** Порівняйте RKS та UKS підходи для дисоціації H_2 . Поясніть різницю в поведінці енергії на великих відстанях.
4. **Задача 4:** Розрахуйте перші три збуджені стани N_2 методом TD-DFT з різними функціоналами. Порівняйте з експериментальними значеннями.
5. **Задача 5:** Дослідіть молекулу CuH методами PBE, TPSSh та B3LYP. Який функціонал краще описує зв'язок Cu–H?
6. **Задача 6:** Виконайте систематичне дослідження галогенів (F_2 , Cl_2 , Br_2 , I_2) з різними функціоналами. Який функціонал найкраще відтворює експериментальні енергії дисоціації?
7. **Задача 7:** Побудуйте benchmark для молекул H_2 , N_2 , CO з базисами від cc-pVDZ до aug-cc-pV5Z. Визначте, при якому базисі досягається конвергенція з точністю 1 mHa.
8. **Задача 8:** Розрахуйте тестовий набір G2 (8 молекул) функціоналами PBE, B3LYP, PBE0, M06-2X. Порівняйте MAE, RMSE та визначте найкращий функціонал.

6.8.20. Типові помилки при розрахунках молекул

1. Неправильна симетрія:

- Завжди використовуйте симетрію молекули
 - Перевіряйте point group командою `mol.symmetry = True`
2. **Хибна мультиплетність:**
- O_2 — триплет, не синглет!
 - Перевіряйте спин по `mol.spin`
3. **Недостатня сітка інтегрування:**
- Для точних градієнтів: `grids.level = 4`
 - Для оптимізації завжди перевіряйте конвергенцію по сітці
4. **Погана початкова геометрія:**
- Починайте з розумних відстаней
 - Використовуйте експериментальні дані або попередні розрахунки
5. **Ігнорування BSSE (Basis Set Superposition Error):**
- Для енергій зв'язування використовуйте counterpoise корекцію
 - Особливо важливо для малих базисів

6.8.21. Систематичне дослідження галогенів

Галогени (F_2 , Cl_2 , Br_2 , I_2) — цікава серія для тестування функціоналів:

`c code_49.py`

Особливості галогенів:

- F_2 має найслабкіший зв'язок через відштовхування lone pairs
- Для Br та I необхідні релятивістські псевдопотенціали
- Дисперсійні корекції важливі для важких галогенів
- LDA/GGA систематично переоцінюють енергії зв'язку

6.8.22. Benchmark: час обчислень

Практичне порівняння швидкості різних методів:

`c code_50.py`

Масштабування обчислювальної складності:

- **LDA/GGA:** $\mathcal{O}(N^3)$ — найшвидші
- **Гібриди:** $\mathcal{O}(N^4)$ — через точний обмін HF
- **Meta-GGA:** $\mathcal{O}(N^3)$ — але з більшою константою
- **Double-hybrid:** $\mathcal{O}(N^5)$ — через MP2 кореляцію

6.8.23. Тестовий набір G2

Систематична оцінка якості функціоналів на стандартному наборі:

`c code_51.py`

Метрики якості:

- **MAE** (Mean Absolute Error) — середня абсолютна помилка
- **RMSE** (Root Mean Square Error) — середньоквадратична помилка
- **Max error** — максимальна помилка

Типові результати для енергій дисоціації:

- LDA: MAE $\sim 0.5\text{--}1.0$ eV (переоцінює зв'язування)
- PBE: MAE $\sim 0.3\text{--}0.5$ eV
- B3LYP/PBE0: MAE $\sim 0.1\text{--}0.3$ eV (найкраще)
- M06-2X: MAE $\sim 0.2\text{--}0.4$ eV

6.8.24. Використання симетрії

Симетрія молекули може суттєво прискорити розрахунки:

`c code_52.py`

Переваги використання симетрії:

1. **Прискорення:** 2–10х залежно від точкової групи
2. **Менше пам'яті:** блокова структура матриць
3. **Класифікація орбіталей:** по незвідним представленням
4. **Відбір правил:** для спектроскопії

Точкові групи діатомних молекул:

- $D_{\infty h}$: гомоядерні (H_2 , N_2 , O_2)
- $C_{\infty v}$: гетероядерні (CO , HF , NO)

6.8.25. Резюме секції

У цій секції ми навчилися:

- Проводити DFT розрахунки діатомних молекул
- Будувати криві потенційної енергії
- Оптимізувати геометрію та обчислювати спектроскопічні константи
- Порівнювати різні функціонали для хімічних зв'язків
- Враховувати дисперсійні взаємодії
- Розраховувати збуджені стани методом TD-DFT
- Уникати типових помилок при молекулярних розрахунках

Ключовий висновок: DFT є ефективним інструментом для опису хімічних зв'язків у діатомних молекулах, але вибір функціоналу критично важливий і залежить від типу зв'язку та властивостей, що досліджуються.

Після обчислень електронної структури молекули природним кроком є обчислення її спостережуваних властивостей: коливальних спектрів, оптичних переходів, електричних та магнітних характеристик. PySCF надає потужні інструменти для розрахунку цих властивостей на рівні теорії Хартрі-Фока та пост-ХФ методів.

У попередніх розділах ви навчилися виконувати основні квантово-хімічні розрахунки: оптимізувати геометрію, обчислювати енергії, аналізувати хвильові функції. Тепер настав час перейти до **обчислення молекулярних властивостей** — фізичних величин, які можна безпосередньо порівняти з експериментом.

Що таке молекулярні властивості?

Молекулярні властивості — це спостережувані величини, які характеризують поведінку молекули у зовнішніх полях або при взаємодії з випромінюванням. На відміну від енергії чи геометрії, властивості безпосередньо вимірюються експериментально:

- **Коливальні спектри (ІЧ, Раман):** частоти, інтенсивності.
- **Електронні спектри (УФ-видимі):** довжини хвиль, сили осциляторів.
- **Електричні властивості:** дипольний момент, поляризованість.
- **Магнітні властивості:** ЯМР зсуви, g-тензор, константи спін-спінової взаємодії.
- **Оптична активність:** круговий дихроїзм, оптичне обертання.

Головна перевага обчислення властивостей: можна безпосередньо порівняти теорію з експериментом та перевірити якість обчислень!

Встановлення додаткових модулів PySCF

Базова інсталяція PySCF містить функціонал для розрахунку енергій, градієнтів та оптимізації геометрії. Для обчислення спектроскопічних та магнітних властивостей потрібні **додаткові модулі**.

Основні модулі для властивостей

1. `pyscf-properties` — головний модуль для властивостей:

```
pip install pyscf-properties
```

Цей модуль включає:

- `pyscf.prop.nmr` — ЯМР константи екранування
- `pyscf.prop.esr` — ЕПР g-тензор, константи надтонкої взаємодії
- `pyscf.prop.polarizability` — поляризованість, гіперполяризованості
- `pyscf.prop.magnetizability` — магнітна сприйнятливість

2. `pyscf-geomopt` — для оптимізації геометрії:

```
pip install pyscf-geomopt
```

3. `geometric` — покращена оптимізація (рекомендується):

```
pip install geometric
```

Перевірка встановлення

Після інсталяції перевірте, чи працюють модулі:

```
import pyscf
from pyscf import gto, scf
from pyscf.prop import nmr, esr # для магнітних властивостей
from pyscf.hessian import thermo # для коливальних властивостей
from pyscf import tddft # для електронних збуджень

print("PySCF версія:", pyscf.__version__)
print("Модулі успішно імпортовані!")
```

Якщо імпорт проходить без помилок — все готово для роботи!

Альтернатива: встановлення з conda

Для користувачів Anaconda/Miniconda:

```
conda install -c pyscf pyscf
conda install -c conda-forge geometric
pip install pyscf-properties # properties поки немає в conda
```

Концептуальна основа: теорія відгуку

Теорія відгуку (response theory) є потужним інструментом для обчислення молекулярних властивостей у квантовій механіці. Її суть полягає в аналізі реакції системи на зовнішні збурення: ми застосовуємо слабе зовнішнє поле (або збурення) і спостерігаємо, як змінюється енергія, хвильова функція чи інші observable величини. Цей підхід дозволяє систематично обчислювати похідні енергії по параметрах збурення, які безпосередньо відповідають фізичним властивостям, таким як поляризованість, магнітні моменти чи спектроскопічні переходи.

Загалом, енергія системи в присутності збурення λ (де λ може бути будь-яким параметром, наприклад, електричним або магнітним полем чи геометрією молекули тощо) розкладається в ряд Тейлора:

$$E(\lambda) = E_0 + \left. \frac{\partial E}{\partial \lambda} \right|_{\lambda=0} \lambda + \frac{1}{2!} \left. \frac{\partial^2 E}{\partial \lambda^2} \right|_{\lambda=0} \lambda^2 + \frac{1}{3!} \left. \frac{\partial^3 E}{\partial \lambda^3} \right|_{\lambda=0} \lambda^3 + \dots$$

Коефіцієнти при степенях λ є похідними енергії по збуренню в точці $\lambda = 0$ і визначають лінійний, квадратичний тощо відгук системи. Ці похідні інтерпретуються як фундаментальні властивості:

- Перша похідна $\left. \frac{\partial E}{\partial \lambda} \right|_{\lambda=0}$ — лінійний відгук (наприклад, дипольний момент).
- Друга похідна $\left. \frac{\partial^2 E}{\partial \lambda^2} \right|_{\lambda=0}$ — квадратичний відгук (наприклад, поляризованість).
- Вищі похідні — нелінійні ефекти (гіперполяризованість тощо).

Загальний принцип: будь-яка молекулярна властивість, пов'язана з відгуком на збурення, є похідною енергії (або хвильової функції) по відповідному параметру.

Приклад: розклад енергії в електричному полі

Для ілюстрації розглянемо конкретний випадок статичного однорідного електричного поля $\mathbf{E}$. Енергія системи розкладається як:

$$E(\mathbf{E}) = E_0 - \mu \cdot \mathbf{E} - \frac{1}{2} \sum_{ij} \alpha_{ij} E_i E_j - \frac{1}{6} \sum_{ijk} \beta_{ijk} E_i E_j E_k + \dots$$

де:

- E_0 — енергія системи без поля.
- μ — статичний дипольний момент (перша похідна $\left. -\frac{\partial E}{\partial \mathbf{E}} \right|_{\mathbf{E}=0}$).
- α_{ij} — тензор поляризованості (друга похідна $\left. -\frac{\partial^2 E}{\partial E_i \partial E_j} \right|_{\mathbf{E}=0}$).
- β_{ijk} — тензор першої гіперполяризованості (третя похідна $\left. -\frac{\partial^3 E}{\partial E_i \partial E_j \partial E_k} \right|_{\mathbf{E}=0}$).

Цей розклад є частковим випадком загальної теорії відгуку, де $\lambda \equiv \mathbf{E}$. Аналогічно, теорія застосовується до магнітних полів (для магнітної сприйнятливості), геометричних змін (для сил і гессіанів) чи часозалежних збурень (для спектроскопії).

Загальний принцип:

властивість = похідна енергії по зовнішньому полю.

Методи обчислення похідних

1. Числові похідні (finite differences):

$$\alpha_{xx} = -\frac{\partial^2 E}{\partial E_x^2} \approx \frac{E(E_x) + E(-E_x) - 2E(0)}{\Delta E_x^2}$$

Плюси: просто, універсально

Мінуси: неточно, повільно, багато розрахунків

2. Аналітичні похідні:

$$\alpha_{ij} = -\left. \frac{\partial^2 E}{\partial E_i \partial E_j} \right|_{E=0} = \text{складна формула через хвильову функцію}$$

Плюси: точно, швидко, один розрахунок

Мінуси: складна реалізація

3. Coupled-Perturbed методи (CP-HF, CP-KS):

Розв'язують рівняння для змін орбіталей під дією поля. Використовується в PySCF для більшості властивостей.

Рекомендація: завжди використовуйте аналітичні похідні, коли вони доступні!

Gauge-invariance для магнітних властивостей

Магнітні властивості (ЯМР, магнітна сприйнятливність) мають специфічну проблему: результат залежить від вибору **gauge** (калібрування) векторного потенціалу.

Проблема gauge origin

Магнітне поле $\mathbf{B}$ можна записати через векторний потенціал:

$$\mathbf{B} = \nabla \times \mathbf{A}$$

Але $\mathbf{A}$ не унікальний! Калібрувальне перетворення $\mathbf{A} \rightarrow \mathbf{A} + \nabla \chi$ не змінює $\mathbf{B}$.

Для скінченного базису результат **залежить від вибору початку координат** для $\mathbf{A}$ — це нефізично!

Рішення: GIAO (Gauge-Independent Atomic Orbitals)

У PySCF використовується метод GIAO (також називається IGLO, CSGT):

- Кожна атомна орбіталь має своє локальне калібрування.
- Результати не залежать від вибору початку координат.
- Швидка збіжність з розміром базису.
- **Стандарт** для розрахунків ЯМР властивостей.

В PySCF це працює автоматично — не потрібно нічого додатково налаштовувати!

```
from pyscf.prop import nmr

# GIAO використовується автоматично
nmr_calc = nmr.RHF(mf)
shielding = nmr_calc.kernel() # gauge-independent результат!
```

Базиси для розрахунку властивостей

Важливо: базиси для енергій $\neq$ базиси для властивостей!

Загальні вимоги

1. Дифузні функції (aug-, d-aug-):

Критично важливі для:

- Поляризовностей (опис деформації електронної густини)
- Магнітних властивостей (струми далеко від ядер)
- Анізотропних властивостей

Базиси: aug-cc-pVDZ, aug-cc-pVTZ, 6-311++G**

2. Високий angular momentum (поляризаційні функції):

Для точних властивостей потрібні d , f , іноді g функції.

3. Tight functions для ЯМР:

ЯМР екранування чутливе до густини на ядрі $\rho(\mathbf{R}_A)$. Спеціальні базиси: pcS-n, pcJ-n

Спеціалізовані базиси

Властивість	Рекомендований базис	Коментар
ІЧ частоти	6-31G*, 6-311G**	Невеликі базиси ОК
УФ-видимі спектри	aug-cc-pVDZ/TZ	Дифузні важливі
Поляризованість	aug-cc-pVDZ	Обов'язково aug-
ЯМР екранування	pcS-2, aug-cc-pVTZ	Tight + diffuse
J-константи	pcJ-2, pcJ-3	Спеціальний базис
ЕПР (g-тензор, HFC)	EPR-II, EPR-III	Tight s-функції

Практичні поради

- Для початку: **aug-cc-pVDZ** — універсальний вибір
- Для точних результатів: **aug-cc-pVTZ**
- Для великих систем: **6-311+G(2d,p)** — компроміс
- Для ЯМР: спеціалізовані базиси **pcS-n** кращі за aug-cc-pVnZ

Структура розділу

Розділ організовано за типами властивостей:

1. **Коливальні властивості (Секція ??):**
 - Гессіан та нормальні моди
 - ІЧ спектри (інтенсивності, правила відбору)
 - Раман спектри
 - Термохімія та ZPE
2. **Електронні властивості (Секція 7.1):**
 - Електронні збудження (TD-DFT)
 - УФ-видимі спектри
 - Флуоресценція та фосфоресценція
3. **Електричні властивості (Секція ??):**
 - Дипольний момент
 - Поляризованість (статична та динамічна)
 - Гіперполяризованості
 - Електростатичний потенціал
4. **Магнітні властивості (Секція ??):**
 - Магнітна сприйнятливості
 - ЯМР константи екранування та J-константи
 - ЕПР властивості (g-тензор, HFC)
 - Оптична активність (ORD, CD)

Практичні рекомендації перед початком

Типовий workflow розрахунку властивостей

1. Оптимізуйте геометрію:

```
from pyscf import gto, scf
from pyscf.geomopt.geometric_solver import optimize

mol = gto.M(atom='...', basis='6-31g*')
mf = scf.RHF(mol).run()
mol_eq = optimize(mf) # оптимізована геометрія
```

2. Перерахуйте з кращим базисом:

```
mol_prop = gto.M(atom=mol_eq.atom, basis='aug-cc-pvdz')
mf_prop = scf.RHF(mol_prop).run()
```

3. Обчисліть властивість:

```
from pyscf.prop import nmr
nmr_calc = nmr.RHF(mf_prop)
shielding = nmr_calc.kernel()
```

4. Проаналізуйте результат!

Типові помилки початківців

- Розрахунок властивостей без оптимізації геометрії
- Використання малих базисів (6-31G) для поляризовностей
- Забування про дифузні функції для анізотропних властивостей
- Розрахунок ЯМР без gauge-independent методу
- Порівняння абсолютних значень замість зсувів (для ЯМР)

Контрольний чек-лист

Перед розрахунком властивості перевірте:

- Геометрія оптимізована? (немає уявних частот)
- Базис підходить для цієї властивості?
- Метод підходить? (HF завищує поляризованість, потрібен DFT/MP2)
- Для магнітних: gauge-independent метод?
- SCF збігся? (check `mf.converged`)
- Для радикалів: перевірили $\langle S^2 \rangle$?

Корисні ресурси

- Документація PySCF: <https://pyscf.org/user.html>
- PySCF properties: <https://github.com/pyscf/properties>
- Basis Set Exchange: <https://www.basissetexchange.org/>
- NIST WebBook: експериментальні дані для порівняння

7.1. Електронні переходи та УФ-видимі спектри

7.1.1. Теорія збуджених станів

Поглинання фотона молекулою призводить до переходу електрона з основного електронного стану S_0 у збуджений S_n . Енергія переходу визначається різницею повних енергій:

$$\hbar\omega_n = E_n - E_0.$$

Однак у реальному спектрі спостерігаються не «чисті» електронні переходи, а *електронно-вібраційні*, тобто такі, де відбувається зміна не лише

електронного, а й вібраційного стану молекули:

$$|S_0, v''\rangle \longrightarrow |S_n, v'\rangle.$$

Через різницю форм потенціальних кривих $E_0(R)$ і $E_n(R)$ мінімальні відстані рівноваги $R_e^{(0)}$ і $R_e^{(n)}$ не збігаються. У момент поглинання фотона ядра практично не встигають зміститись — це **принцип Франка–Кондона**: електронний перехід відбувається при фіксованих координатах ядер. На діаграмі потенціальних кривих це означає *вертикальний перехід*:

$$R_{\text{збудж.}} = R_{\text{основ.}}$$

Рис. 7.1. Схематичне зображення принципу Франка–Кондона. Вертикальні переходи $S_0 \rightarrow S_n$ відбуваються без зміни координат ядер.

Згідно з наближенням Борна–Оппенгаймера, повна хвильова функція молекули розкладається на електронну та ядерну частини:

$$\Psi_{n,v}(r, R) = \psi_n(r; R) \chi_{n,v}(R),$$

де $\psi_n(r; R)$ описує електрони, а $\chi_{n,v}(R)$ — ядерний (вібраційний) рух.

Імовірність переходу між станами $|S_0, v''\rangle$ і $|S_n, v'\rangle$ визначається квадратом модуля дипольного матричного елемента:

$$I_{v' \leftarrow v''} \propto |\langle \Psi_{n,v'} | \hat{\mu} | \Psi_{0,v''} \rangle|^2.$$

Якщо електронний оператор $\hat{\mu}$ слабо залежить від координат ядер, вираз розкладається на електронну та вібраційну частини:

$$I_{v' \leftarrow v''} \propto \underbrace{|\langle \psi_n | \hat{\mu}_e | \psi_0 \rangle|^2}_{\text{електронний перехід}} \underbrace{|\langle \chi_{n,v'} | \chi_{0,v''} \rangle|^2}_{\text{фактор Франка–Кондона } F_{v',v''}}.$$

Другий множник

$$F_{v',v''} = |\langle \chi_{n,v'} | \chi_{0,v''} \rangle|^2$$

називається **фактором Франка–Кондона** і відображає перекриття хвильових функцій вібраційних рівнів. Він визначає, які з переходів $v'' \rightarrow v'$ будуть найінтенсивнішими: чим сильніше перекриття, тим більша інтенсивність.

Зв'язок із силою осцилятора

Сила осцилятора є мірою інтенсивності переходу в спектрі поглинання. Для кожного переходу $|S_0, v''\rangle \rightarrow |S_n, v'\rangle$ вона записується як:

$$f_{v',v''} = \frac{2}{3} \omega_{v',v''} |\langle \Psi_{n,v'} | \hat{\mu} | \Psi_{0,v''} \rangle|^2.$$

Підставляючи розклад із принципу Франка–Кондона, отримаємо:

$$f_{v',v''} \propto \underbrace{|\langle \psi_n | \hat{\mu}_e | \psi_0 \rangle|^2}_{\text{тип переходу}} \times \underbrace{F_{v',v''}}_{\text{вібраційне перекриття}}.$$

Отже:

- електронна частина визначає загальну інтенсивність переходу;
- вібраційна частина (фактор Франка–Кондона) розподіляє цю інтенсивність між коливальними рівнями.

Якщо потенціальні криві $E_0(R)$ і $E_n(R)$ мають близькі мінімуми, то головним буде перехід $v'' = 0 \rightarrow v' = 0$, і спектр має вузьку, симетричну смугу. При великому зсуві $R_e^{(n)} - R_e^{(0)}$ інтенсивність розподіляється по багатьох вібраційних лініях, утворюючи типову коливальну структуру у смузі поглинання.

7.1.2. TDDFT розрахунок для формальдегіду

Розглянемо молекулу формальдегіду H_2CO як приклад:

`c h2co_tddft.py`

Аналіз збуджень для H_2CO :

Перехід	Тип	λ (нм)	f	Характер
S_1	$n \rightarrow \pi^*$	355	0.001	Заборонений
S_2	$\pi \rightarrow \pi^*$	185	0.152	Дозволений
S_3	$n \rightarrow 3s$	172	0.021	Рідбергівський

Фізична інтерпретація:

- $n \rightarrow \pi^*$: неподілена пара O $\rightarrow$ антизв'язуюча МО C=O
- Низька сила осцилятора через симетрію
- $\pi \rightarrow \pi^*$: основна смуга поглинання в УФ
- Енергії залежать від функціоналу DFT

7.1.3. Вибір функціоналу для TDDFT

Різні функціонали дають різну точність для збуджень:

`c formaldehyde_functional_comparison.py`

Порівняння функціоналів (перехід $n \rightarrow \pi^*$):

Функціонал	λ (нм)	Похибка (нм)
B3LYP	355	+25
PBE0	342	+12
CAM-B3LYP	335	+5
ω B97X-D	332	+2
Експеримент	330	—

Рекомендації:

- Гібридні функціонали краще за чисті GGA
- Range-separated (CAM-B3LYP, ω B97X-D) найточніші
- Для переносу заряду обов'язково range-separated
- B3LYP часто завищує довжини хвиль

7.1.4. Візуалізація спектру

Для побудови спектру використовуємо гаусові або лоренцеві контури:

$$\varepsilon(\lambda) = \sum_n f_n \cdot \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(\lambda - \lambda_n)^2}{2\sigma^2}\right]$$

`c plot_uv_spectrum.py`

Рис. 7.2. УФ-спектр формальдегіду, розрахований методом TDDFT/CAM-B3LYP/aug-cc-pVDZ.

Параметри уширення:

- Типове $\sigma = 0.3\text{--}0.5$ eV для конденсованої фази
- $\sigma = 0.1\text{--}0.2$ eV для газової фази
- Експериментальне уширення враховує розподіл за T

7.1.5. Аналіз характеру переходів

TDDFT надає інформацію про орбіталі, задіяні у переході:

`c analyze_transitions.py`

Приклад виводу для S_1 стану H_2CO :

```
Excited State 1: 3.492 eV (355 nm) f=0.0012
  HOMO-1 -> LUMO      0.11 (1.2%)
  HOMO    -> LUMO      0.69 (47.6%)
  HOMO    -> LUMO+1    0.08 (0.6%)
```

Інтерпретація:

- Основний внесок: $\text{HOMO} \rightarrow \text{LUMO}$ (48%)
- HOMO — неподілена пара $n(\text{O})$
- LUMO — антизв'язуюча $\pi^*(\text{C}=\text{O})$
- Тип переходу: $n \rightarrow \pi^*$

Література

Основна література

1. *Jensen F.* Introduction to Computational Chemistry. — 3rd ed. — Wiley, 2017. — 661 p. — ISBN 1118825993.
2. *Levine I. N.* Quantum Chemistry. — 7th ed. — Pearson, 2014. — 714 p. — ISBN 978-0321803450.

Додаткові посилання

1. [Basis Sets](https://gaussian.com/basissets/). — URL: <https://gaussian.com/basissets/>.
2. *Ho M., Hernández-Perez J. M.* [Evaluation of Gaussian Molecular Integrals. I Overlap Integrals](#) // The Mathematica Journal. — 2012. — Vol. 14.
3. *Ho M., Hernández-Perez J. M.* [Evaluation of Gaussian Molecular Integrals. II. Kinetic-Energy Integrals](#) // The Mathematica Journal. — 2013. — Vol. 15.
4. *Ho M., Hernández-Perez J. M.* [Evaluation of Gaussian Molecular Integrals. III. Nuclear-Electron Attraction Integrals](#) // The Mathematica Journal. — 2014. — Vol. 16.
5. [Simple Quantum Chemistry: Hartree-Fock in Python](https://nznano.blogspot.com/2018/03/simple-quantum-chemistry-hartree-fock.html). — URL: <https://nznano.blogspot.com/2018/03/simple-quantum-chemistry-hartree-fock.html>.